Outline

What I can tell from the upload is that your current knowledge system is **already acting like a registry-driven library**, but it is still **organized like a path-driven folder tree**, and that is the part that will keep creating pain as it grows.

From `resolver.ts`, the system currently depends on a few brittle patterns:

- knowledge is resolved from `INDEX/knowledge.index.json`, `taxonomy.json`, and `bundles.index.json`
- routing and constraint maps are hardcoded in the resolver
- domain matching still leans heavily on `kid.path`
- the displayed “domain” is inferred from the parent folder name in the path
- taxonomy exists, but folder structure is still doing too much of the organizational work

That means adding new sections later will often force you to touch **folders, paths, routing rules, taxonomy, bundles, and matching logic** instead of just updating one clean registry layer.

The better system

You want to move from:

folder-based organization
to
registry-based organization with faceted classification

That gives you two big wins:

1. You can add new categories without moving files around.
2. One knowledge item can belong to multiple views without duplication.

The core rule

Your physical file layout should be **boring and stable**.
Your semantic organization should live in **metadata indexes**.

So instead of treating folders as meaning, treat folders as storage only.

Recommended model

Use 4 layers:

1. Canonical item layer

Each knowledge item is a single canonical record with a stable ID.

Example fields:

- `kid`
- `title`
- `artifact_type`
- `primary_pillar`
- `primary_domain`
- `secondary_domains`
- `capabilities`
- `lifecycle_stage`
- `audience`
- `maturity`
- `status`
- `tags`
- `depends_on`
- `supersedes`
- `related_kids`
- `path`

This becomes the source of truth.

2. Taxonomy layer

This defines the allowed classification values.

Not giant nested trees only. Use controlled vocabularies for:

- pillars
- domains
- subdomains
- artifact types
- lifecycle stages
- audiences
- capabilities
- maturity states

This should be data, not resolver logic.

3. View layer

Views are how humans browse the library.

Examples:

- by pillar
- by domain
- by lifecycle stage
- by artifact type
- by maturity
- by agent role
- by control plane responsibility
- by bundle/use case

A KID can appear in many views without being moved.

4. Bundle layer

Bundles are curated solution packs for execution or prompting.

Examples:

- `FULLSTACK_SAAS`
- `VERIFICATION_RUNTIME`
- `MAINTENANCE_SYSTEM`
- `CONTROL_PLANE_CORE`

Bundles should reference KIDs by ID only.

What is wrong with the current shape

Your resolver suggests you have this conceptual structure:

- pillar
- group
- domain
- bundle
- tags
- routing category
- constraints

That is not bad. The problem is that it is partially expressed in metadata and partially buried in code/path assumptions.

The biggest structural issue is this:

a knowledge item's meaning should not depend on where the file sits in the folder tree

Right now, `kid.path` is still functioning like taxonomy.

That will hurt when:

- you rename folders
- you split one domain into two
- one item belongs in multiple domains
- you add new cross-cutting categories like “control plane”, “operator UX”, “verification”, “runtime governance”

The system I’d use for Axion

Physical storage

Keep this flat and durable:

```
knowledge/
 INDEX/
   knowledge.index.json
   taxonomy/
     pillars.json
     domains.json
     artifact-types.json
     lifecycle-stages.json
     audiences.json
     capabilities.json
     maturity.json
     tags.json
   bundles/
     bundles.index.json
     FULLSTACK_SAAS.bundle.json
     MAINTENANCE_SYSTEM.bundle.json
     CONTROL_PLANE_CORE.bundle.json
   views/
     by-pillar.json
     by-domain.json
     by-lifecycle.json
     by-artifact-type.json
     by-role.json
 ITEMS/
   KID-ARC-001.md
   KID-OPS-014.md
   KID-VER-022.md
   KID-CP-007.md
```

This is much easier to maintain than deep domain folders.

Why this is better

- files never need to move when taxonomy changes
- IDs remain stable
- views can be regenerated automatically
- reorganizing the library becomes metadata work, not file surgery

Better classification scheme for your library

For Axion, I would classify each item across these axes:

Axis 1 — System pillar

Top-level ownership area.

Examples:

- Control Plane
- Agent Runtime
- Knowledge System
- Template System
- Verification System
- Maintenance System
- Operator UX
- Integrations
- Operations & Reliability
- Governance & Security

Axis 2 — Functional domain

Specific topic inside a pillar.

Examples:

- orchestration
- routing
- standards resolution
- artifact generation
- evidence/proofs
- policy
- audit
- observability

- approvals
- registries
- maintenance modes

Axis 3 — Artifact type

What kind of thing the item is.

Examples:

- standard
- spec
- contract
- schema
- template
- policy
- playbook
- workflow
- checklist
- registry entry
- design note

Axis 4 — Lifecycle stage

Where it fits in build/operate flow.

Examples:

- intake
- canonicalization
- planning
- generation
- verification
- packaging
- release
- maintenance
- governance

Axis 5 — Consumer

Who uses it.

Examples:

- agent

- control plane
- operator
- verifier
- maintainer
- runtime
- UI

Axis 6 — Stability

How safe it is to rely on.

Examples:

- draft
- reviewed
- verified
- golden
- deprecated

That makes the library expandable without rethinking the whole tree every time.

The key organizational shift

Do not organize mostly by “topic folders.”
Organize by:

stable items + metadata + generated views

That is the system that scales.

What I would change in your resolver design

Based on the uploaded code, I would make these changes first:

1. Stop deriving domain from path

This line of thinking is fragile:

- infer domain from parent folder
- match domains by substring in path

Instead, require explicit metadata:

- `primary_domain`

- `secondary_domains`

2. Move hardcoded routing maps into config

These should not live in the resolver source:

- `ROUTING_TO_BUNDLE`
- `CONSTRAINT_TO_DOMAINS`
- `ROUTING_CATEGORY_DOMAINS`

Put them in JSON registries so your library structure can evolve without code edits.

3. Separate taxonomy from navigation

Taxonomy defines allowed meanings.
Views define how humans browse.

Those are not the same thing and should not be fused.

4. Allow multi-home classification

A KID should be able to be:

- in one primary pillar
- in multiple domains
- in multiple bundles
- visible in several generated views

That removes duplication pressure.

5. Add explicit dependency metadata

This matters a lot in Axion.

Each KID should be able to declare:

- `depends_on`
- `required_by`
- `related_kids`
- `supersedes`
- `replaced_by`

That turns the library into a navigable system rather than a pile of docs.

The practical organization I recommend for you

Use this:

```
knowledge/
 INDEX/
   knowledge.index.json
   taxonomy/
   bundles/
   views/
 ITEMS/
   KID-*.md
```

And each item header should look like:


```
{
 "kid": "KID-CP-007",
 "title": "Run Orchestration Policy",
 "artifact_type": "policy",
 "primary_pillar": "control_plane",
 "primary_domain": "orchestration",
 "secondary_domains": ["governance", "runtime_control"],
 "lifecycle_stage": ["run_start", "stage_transition"],
 "audience": ["control_plane", "operator"],
 "maturity": "reviewed",
 "status": "active",
 "tags": ["gates", "reruns", "resume", "partial-run"],
 "depends_on": ["KID-CP-001", "KID-GOV-003"],
 "related_kids": ["KID-VER-011"],
 "path": "ITEMS/KID-CP-007.md"
}
```


Best browsing views to generate

For your use case, these matter most:

- by pillar
- by domain
- by lifecycle
- by artifact type
- by maturity
- by agent vs control plane
- by maintenance mode
- by bundle/use case

That would make updating far less painful.

My recommendation for naming

Use stable IDs that encode class, not folder:

- `KID-CP-001`
- `KID-OPS-014`
- `KID-VER-022`
- `KID-MAINT-005`

Then let titles be human-readable and changeable.

Do not rely on filenames for meaning beyond the stable ID.

Bottom line

Your current system is close, but it is still too dependent on **paths and resolver code**.
The cleaner long-term architecture is:

flat canonical items + strict metadata + generated views + config-driven routing

That is the structure that will let you add new sections, split domains, add cross-cutting categories, and reorganize the library without turning every update into cleanup work.

Also, I could only inspect the uploaded `resolver.ts` structure from the ZIP, not a full knowledge library tree, so this is an architecture recommendation based on the resolver design rather than a full audit of all library contents. There is also a separate planning file in the conversation context.

Update

I found the real issue already: the repo is carrying multiple organizational systems at once. The biggest one is `Axion/` with 17,000+ entries, while `App/`, `server/`, `.axion/`, and even a `Docs zip/` copy are also acting like parallel sources of truth.

Thought for 1m 59s

Yeah — now that I can see the real library, the problem is much clearer.

Your knowledge library is **not badly structured because it lacks categories**.
It is badly structured because it has **too many competing organizational layers at once**.

What is happening right now

Inside `Axion/libraries/knowledge/`, you currently have all of these mixed together:

- `PILLARS/` as the main content tree
- `INDEX/` as the registry layer
- `BUNDLES/` as packaging views
- `contracts/` as library rules
- `TEMPLATES/` as authoring scaffolds
- `POLICIES/` as governance rules
- `REUSE/` as licensing/reuse controls
- root-level `KNO-*` files as library doctrine
- `OUTPUTS/` as generated artifacts

That means the library is trying to be all of these at once:

- a content repository
- a taxonomy system
- a publishing system
- a rules engine
- a template library
- a governance layer
- a generated-output area

That is why it feels hard to update.

The biggest structural issue

Your current **human browsing model** is driven mostly by this path:

`PILLARS -> broad category -> subcategory -> domain -> content-type -> KID`

That works at small scale, but once the library grows, it causes 4 problems:

1. Path becomes meaning

A file's identity starts depending on where it lives.

That is fragile because:

- renaming folders becomes expensive
- one item can logically belong to multiple places
- adding new categories can force moving lots of files

2. Too much depth

A lot of your content lives several folders deep. That makes the library feel "organized" visually, but it increases maintenance cost.

3. Multiple source-of-truth layers

You already have:

- `knowledge.index.json`
- `knowledge_registry.v1.json`
- `taxonomy.json`
- `bundles.index.json`
- folder structure itself

That is too many systems carrying structure at once.

4. The index does not appear to fully mirror the content tree

You have **hundreds of content files under `PILLARS/`**, but the main index count is much smaller. That suggests one of these is true:

- not all items are indexed
- some items are placeholders only
- the registry and tree are drifting apart

That is a maintenance headache waiting to happen.

The better system

You need to separate the library into **4 clean layers**.

Layer 1 — Canonical content

This is the actual knowledge.

Only store:

- KIDs
- their content
- stable metadata

This should be the only place where knowledge items “live.”

Layer 2 — Taxonomy

This defines allowed classification values.

Examples:

- pillars
- domains
- content types
- tags
- maturity
- authority tier
- freshness model

Taxonomy should define meaning, not storage paths.

Layer 3 — Views

Views are for browsing.

Examples:

- by pillar
- by domain
- by content type
- by industry
- by stack

- by executor
- by risk class

These should be generated from metadata, not manually embodied in the folder tree.

Layer 4 — Bundles

Bundles are curated selection packs.

Examples:

- `FULLSTACK_SAAS`
- `WEB_APP_BASIC`
- `PRODUCTION`
- `EXTERNAL_AGENT`

Bundles should reference KIDs by ID, not depend on folder location.

What I would change

1. Stop using `PILLARS/` as the primary physical organization

Keep pillar as metadata, not as the main storage tree.

Right now your physical tree is doing too much semantic work.

2. Flatten the content storage

Instead of deeply nesting KIDs by pillar/domain/content-type, store canonical items in one stable place.

Example:

```
Axion/libraries/knowledge/
 ITEMS/
   KID-INDEDUC-CONCEPT-0001.md
   KID-INDEDUC-CHECK-0001.md
   KID-INDECOM-PATTERN-0001.md
```

Or at most a very shallow split:

```
ITEMS/
 concepts/
 patterns/
 checklists/
 standards/
```

That is enough.

3. Move all organization into metadata

Each KID should explicitly declare:

- `kid`
- `title`
- `primary_pillar`
- `primary_domain`
- `secondary_domains`
- `content_type`
- `industries`
- `stack_families`
- `tags`
- `authority_tier`
- `freshness_class`
- `status`
- `depends_on`
- `related_kids`

Then the system can generate any browsing view you want.

4. Convert `PILLARS/` into a generated view, not the canonical store

This is the big shift.

You can still have a pillar view in the UI.
You just should not make it the physical truth of the library.

5. Split “library content” from “library rules”

Right now `contracts/`, `POLICIES/`, `TEMPLATES/`, `REUSE/`, and `KNO-*` are all mixed into the same knowledge root.

That makes the root noisy.

These should become separate zones.

The structure I recommend

Here is the version I would use for Axion.

```
Axion/libraries/knowledge/
 README.md

 SYSTEM/
   doctrine/
     KNO-0_purpose.md
     KNO-1_unit_classes.md
     KNO-2_authority_tiers.md
     KNO-3_freshness_supersession.md
     KNO-4_retrieval_resolution_rules.md
     KNO-5_dependency_mapping.md
     KNO-6_proof_and_trust.md
     KNO-7_definition_of_done.md

   taxonomy/
     taxonomy.json
     tags.json
     quality_tiers.json

   registries/
     knowledge.index.json
     knowledge_registry.v1.json
     bundles.index.json
     sources.index.json
     deprecations.json

   contracts/
     KL-*.md
     *.schema.json
     *.json
```

```
  policies/
    citation_policy.md
    external_agent_policy.md
    plagiarism_ip_rules.md
    secrets_pii_handling.md
    use_policy.md

  templates/
    KID_frontmatter_template.md
    knowledge_item.md
    industry_playbook.md
    stack_playbook.md
    MVKL_domain_starter_set.md

  outputs/
    exports/
    selections/

CONTENT/
  ITEMS/
    KID-*.md

VIEWS/
  by_pillar/
  by_domain/
  by_industry/
  by_stack/
  by_content_type/

BUNDLES/
  by_executor/
  by_risk_class/
  by_run_profile/

REUSE/
  licenses/
  allowlist.json
  reuse_log.json
```

Why this is better

Root becomes understandable

Instead of one big mixed root, you get:

- `SYSTEM/` = how the library works
- `CONTENT/` = the actual knowledge
- `VIEWS/` = browsing projections
- `BUNDLES/` = curated packs
- `REUSE/` = licensing/compliance

That is much easier to reason about.

Adding new sections becomes easy

You do not need to invent a new place in a giant deep pillar tree.

You just:

- add a KID
- classify it in metadata
- let views/indexes pick it up

Reorganization becomes cheap

If you later want a new category like:

- Control Plane
- Verification Runtime
- Maintenance Operations
- Enterprise UX
- Agent Governance

you do not move files around.
You just update taxonomy and views.

The exact issue with your current pillar model

You currently have 3 big content families:

- `INDUSTRY_PLAYBOOKS`
- `IT_END_TO_END`
- `LANGUAGES_AND_LIBRARIES`

These are not the same kind of classification.

They mix:

- market verticals
- system/use-case structure
- technology stacks

That means your top level is mixing different axes of organization.

That is a taxonomy smell.

These should not all be siblings as the main physical tree.

They are better treated as separate metadata axes:

- `industry`
- `solution_domain`
- `technology_family`

Once you treat them as facets instead of folders, the library becomes much easier to scale.

My recommendation for your classification model

Every knowledge item should be classified on several axes, not just one tree.

Use these:

Primary axes

- `content_type`
- `domain`
- `industry`

- `stack_family`

Operational axes

- `authority_tier`
- `freshness_class`
- `status`
- `quality_tier`

Retrieval axes

- `tags`
- `bundle_refs`
- `executor_scope`
- `risk_class_scope`

Relationship axes

- `depends_on`
- `related_kids`
- `supersedes`
- `replaced_by`

That will outperform any folder tree.

What to keep from your current system

A lot of what you already have is worth keeping.

Keep:

- KID IDs
- bundle system
- taxonomy/index concept
- contracts/schemas
- policy layer
- reuse layer
- templates

Change:

- physical content storage model
- root-level grouping
- dependence on deep pillar folders
- overlap between index/registry/tree

The migration path I would use

Phase 1 — Clean the root

Move into 5 clear zones:

- `SYSTEM/`
- `CONTENT/`
- `VIEWS/`
- `BUNDLES/`
- `REUSE/`

Phase 2 — Flatten the KIDs

Move KID files from deep `PILLARS/` paths into `CONTENT/ITEMS/`.

Keep metadata intact.

Phase 3 — Make taxonomy authoritative

Make sure every classification comes from metadata, not file path.

Phase 4 — Generate views

Create derived views for:

- pillar
- industry
- stack
- domain
- content type

Phase 5 — De-emphasize path logic in the UI/resolver

The UI should browse via registry/filtering, not by walking folders.

My blunt recommendation

If your goal is:

> “updating it with new sections and categories won’t be a hassle”

then the answer is:

stop organizing the library primarily as a deep folder tree.

Use:

flat canonical knowledge items + strict metadata + generated views

That is the system that scales.

The simplest version of the answer

Your current library is hard to maintain because it is organized by **where files sit** instead of **what files are**.

The better system is:

- one canonical item store
- one taxonomy layer
- one registry layer
- multiple generated browsing views
- bundles as references, not storage locations

File System

Good. Based on the actual repo, here is the concrete Axion knowledge-library reorganization I'd use.

Right now the library has:

- `PILLARS/` as the dominant content tree
- `INDEX/` with `knowledge.index.json`, `knowledge_registry.v1.json`, `taxonomy.json`, `bundles.index.json`
- `BUNDLES/`
- `contracts/`
- `POLICIES/`
- `TEMPLATES/`
- `REUSE/`
- `OUTPUTS/`
- root `KNO-*` doctrine files

And the main structural pressure point is this:

- `PILLARS/` has the overwhelming majority of entries
- taxonomy has only 3 pillars, but those pillars mix different classification axes
- the index has 395 items, while the tree is much larger
- `INDUSTRY_PLAYBOOKS`, `IT_END_TO_END`, and `LANGUAGES_AND_LIBRARIES` are not the same kind of category

That is why expansion feels messy.

Before → after

Before

```
Axion/libraries/knowledge/
 PILLARS/
   INDUSTRY_PLAYBOOKS/
   IT_END_TO_END/
   LANGUAGES_AND_LIBRARIES/
 INDEX/
 BUNDLES/
 contracts/
 POLICIES/
 TEMPLATES/
 REUSE/
 OUTPUTS/
 KNO-*.md
```

After

```
Axion/libraries/knowledge/
 README.md

 SYSTEM/
   doctrine/
     KNO-0_purpose.md
     KNO-1_unit_classes.md
     KNO-2_authority_tiers.md
     KNO-3_freshness_supersession.md
     KNO-4_retrieval_resolution_rules.md
     KNO-5_dependency_mapping.md
     KNO-6_proof_and_trust.md
     KNO-7_definition_of_done.md

   taxonomy/
     taxonomy.json
     tags.json
     quality_tiers.json
     vocab.pillars.json
     vocab.domains.json
     vocab.content_types.json
     vocab.industries.json
     vocab.stack_families.json
     vocab.status.json

   registries/
```

```
    knowledge.index.json
    knowledge_registry.v1.json
    bundles.index.json
    sources.index.json
    deprecations.json
    aliases.index.json
    relationships.index.json

  contracts/
    *.schema.json
    KL-*.md

  policies/
    *.md

  templates/
    KID.template.md
    industry_playbook.template.md
    stack_playbook.template.md
    domain_starter_set.template.md

CONTENT/
  ITEMS/
    KID-*.md
  META/
    collections/
    aliases/
    redirects/

VIEWS/
  by_pillar/
  by_domain/
  by_content_type/
  by_industry/
  by_stack_family/
  by_status/
  by_authority/
  by_bundle/

BUNDLES/
  by_executor/
  by_risk_class/
  by_run_profile/
```

```
REUSE/
  licenses/
  allowlist.json
  reuse_log.json

OUTPUTS/
  exports/
  selections/
```

The main idea

The new system makes one thing canonical:

`CONTENT/ITEMS/`

Everything else becomes either:

- system behavior
- generated view
- curated bundle
- compliance/reuse layer

That means adding a new category later does not require moving hundreds of files.

Exact rules for what goes where

`SYSTEM/doctrine/`

Put all root `KNO-*` files here.

These are not normal knowledge items. They define how the knowledge library itself works.

Move:

- `KNO-0_purpose.md`

- `KNO-1_unit_classes.md`
- `KNO-2_authority_tiers.md`
- `KNO-3_freshness_supersession.md`
- `KNO-4_retrieval_resolution_rules.md`
- `KNO-5_dependency_mapping.md`
- `KNO-6_proof_and_trust.md`
- `KNO-7_definition_of_done.md`

`SYSTEM/taxonomy/`

Put controlled vocabularies here.

Move or split:

- `INDEX/taxonomy.json`
- `INDEX/tags.json`
- `INDEX/quality_tiers.json`

Also break `taxonomy.json` into smaller vocab files over time so one file does not become a junk drawer.

`SYSTEM/registries/`

Put machine-readable indexes here.

Move:

- `INDEX/knowledge.index.json`

- `INDEX/knowledge_registry.v1.json`
- `INDEX/bundles.index.json`
- `INDEX/sources.index.json`
- `INDEX/deprecations.json`

Add:

- `aliases.index.json`
- `relationships.index.json`

SYSTEM/contracts/

Move current `contracts/` here unchanged at first.

This keeps rules/schema separate from content.

SYSTEM/policies/

Move current `POLICIES/` here.

SYSTEM/templates/

Move current `TEMPLATES/` here.

CONTENT/ITEMS/

This becomes the canonical home for every KID.

All actual knowledge units go here, regardless of whether they are currently under:

- `INDUSTRY_PLAYBOOKS`

- `IT_END_TO_END`
- `LANGUAGES_AND_LIBRARIES`

VIEWS/

This is where you put generated browsing projections.

These are not source of truth.

Examples:

- a pillar view page
- a domain grouping file
- an industry index
- a stack-family index

BUNDLES/

Keep this as curated selection packs.

The important change is that bundles should point to KIDs by ID only, never by folder location.

REUSE/

Keep reuse/licensing isolated from knowledge content.

OUTPUTS/

Keep generated exports isolated from canonical content.

What changes conceptually

Your current top-level pillar model is doing too much work.

These 3 current pillars:

- `INDUSTRY_PLAYBOOKS`
- `IT_END_TO_END`
- `LANGUAGES_AND_LIBRARIES`

should stop being treated as equivalent physical top-level content classes.

They are really different axes:

- `INDUSTRY_PLAYBOOKS` → industry axis
- `IT_END_TO_END` → solution/domain axis
- `LANGUAGES_AND_LIBRARIES` → stack-family axis

So instead of storing content under those as the source of truth, you should classify every item across multiple axes.

The metadata model each KID should use

Each `KID-*.md` should have frontmatter like this:

```
kid: KID-XXXX
title: Example Title
content_type: concept
primary_domain: payments
secondary_domains:
 - compliance
 - data_modeling
industry_refs:
 - financial_services
stack_family_refs:
 - nodejs
 - postgres
pillar_refs:
 - solution_patterns
 - industry_knowledge
```

status: active
authority_tier: reviewed
freshness_class: versioned_stable
bundle_refs:
 - FULLSTACK_SAAS
tags:
 - reconciliation
 - ledger
 - settlement
depends_on:
 - KID-AAAA
related_kids:
 - KID-BBBB
supersedes: []
source_refs: []
legacy_path:
PILLARS/INDUSTRY_PLAYBOOKS/01_regulated_industries/financial_services/concepts/...

New pillar model

Do not keep the current 3-pillar model as the main structural spine.

Use pillars that represent semantic role, not storage origin.

I'd use something like:

industry_knowledge
solution_patterns
technology_knowledge
governance_and_quality
execution_and_operations

That lets one item belong to more than one pillar if needed.

Concrete migration rules

Rule 1

A KID's permanent identity is its `kid`, not its folder path.

Rule 2

No new canonical knowledge file goes under `PILLARS/`.

Rule 3

`PILLARS/` becomes read-only during migration, then eventually removed or regenerated as a view.

Rule 4

Bundles may reference only KID IDs, never path strings.

Rule 5

Views may group by:

- pillar
- domain
- industry
- stack family
- content type
- status
- authority tier

But views are never canonical storage.

Rule 6

Any metadata currently implied by folder location must become explicit frontmatter.

Rule 7

Every moved KID gets `legacy_path` for backwards traceability until migration is complete.

Rule 8

Every KID must declare exactly one `content_type`.

Rule 9

Every KID must declare at least one of:

- `primary_domain`
- `industry_refs`
- `stack_family_refs`

Rule 10

If a file is not a reusable knowledge unit, it does not belong in `CONTENT/ITEMS/`.

What to do with `_meta.md` files

Your `_meta.md` files in industry folders are useful, but they should not stay buried in deep content paths.

Convert them into one of two things:

- collection metadata in `CONTENT/META/collections/`
- generated view descriptors in `VIEWS/by_*`

So for example:

CONTENT/META/collections/industry.financial_services.md
CONTENT/META/collections/stack.react.md
CONTENT/META/collections/domain.authentication.md

Naming conventions I'd enforce

KIDs

Keep stable IDs.

Example:

- `KID-IND-FINSERV-CONCEPT-0001`
- `KID-SOL-AUTH-PATTERN-0007`
- `KID-TECH-REACT-CHECKLIST-0003`

If that is too long, keep the current IDs but make sure metadata is rich.

Content types

Use a controlled set only:

- concept
- pattern
- checklist
- workflow
- compliance
- data_model
- integration
- kpi_metric
- security_risk
- standard
- playbook
- reference

Right now your library already hints at many of these through folder names. Turn those into metadata enums.

The phased migration plan

Phase 1 — Root cleanup

Create:

- `SYSTEM/`
- `CONTENT/`
- `VIEWS/`

Do not move KIDs yet.

Phase 2 — Move non-content systems

Move:

- `KNO-*` → `SYSTEM/doctrine/`
- `INDEX/*` → `SYSTEM/registries/` and `SYSTEM/taxonomy/`
- `contracts/` → `SYSTEM/contracts/`
- `POLICIES/` → `SYSTEM/policies/`
- `TEMPLATES/` → `SYSTEM/templates/`

This alone will make the library feel cleaner.

Phase 3 — Freeze `PILLARS/`

Mark `PILLARS/` as legacy source during migration.

No new additions there.

Phase 4 — Start canonical item migration

Move KIDs into:

CONTENT/ITEMS/

One batch at a time:

- industry batch
- IT/domain batch
- language/library batch

Each moved file gets explicit frontmatter.

Phase 5 — Build generated views

Generate:

- `VIEWS/by_pillar/`
- `VIEWS/by_domain/`
- `VIEWS/by_industry/`
- `VIEWS/by_stack_family/`
- `VIEWS/by_content_type/`

Phase 6 — Rewrite resolver/index logic

Update your resolver so it never derives meaning from path first.

Resolution order should become:

1. explicit metadata

2. registry relationships
3. aliases
4. legacy path fallback only during transition

Phase 7 — Remove deep-tree dependency

Once indexes and views are stable, retire `PILLARS/` as canonical storage.

The minimum viable version

If you do not want a massive migration right away, do this smaller version first:

Step 1

Create:

```
SYSTEM/
CONTENT/
VIEWS/
```

Step 2

Move only:

- `KNO-*`
- `INDEX/`
- `contracts/`
- `POLICIES/`
- `TEMPLATES/`

Step 3

Leave `PILLARS/` in place temporarily.

Step 4

Change your rule going forward:

“All new KIDs go in `CONTENT/ITEMS/` with metadata. Views are generated.”

That alone stops the mess from growing.

My direct recommendation

Do not try to invent a “better folder hierarchy” inside `PILLARS/`.

That would only reorganize the mess without solving it.

The real fix is:

- make `CONTENT/ITEMS/` the canonical home
- make metadata the source of structure
- make `VIEWS/` the browsing layer
- make `SYSTEM/` the library-control layer

That is the setup that will let you add new sections and categories without turning every expansion into a rewrite.

Transfer

1. Create the new top-level structure

Create these first:

```
Axion/libraries/knowledge/
 README.md
 SYSTEM/
   doctrine/
   taxonomy/
   registries/
   contracts/
   policies/
   templates/
 CONTENT/
   ITEMS/
   META/
     collections/
     aliases/
     redirects/
 VIEWS/
   by_pillar/
   by_domain/
   by_content_type/
   by_industry/
   by_stack_family/
   by_status/
   by_authority/
   by_bundle/
 BUNDLES/
   by_executor/
   by_risk_class/
   by_run_profile/
 REUSE/
   licenses/
 OUTPUTS/
   exports/
   selections/
```

2. Move root library-governance files

Move all root `KNO-*` files into:

SYSTEM/doctrine/

Move

- `KNO-0_purpose.md`
- `KNO-1_unit_classes.md`
- `KNO-2_authority_tiers.md`
- `KNO-3_freshness_supersession.md`
- `KNO-4_retrieval_resolution_rules.md`
- `KNO-5_dependency_mapping.md`
- `KNO-6_proof_and_trust.md`
- `KNO-7_definition_of_done.md`

Result

These become library doctrine, not mixed with content.

3. Split `INDEX/` into taxonomy vs registry

Your current `INDEX/` is doing two jobs. Split it.

Move machine indexes into:

SYSTEM/registries/

Move

- `knowledge.index.json`
- `knowledge_registry.v1.json`
- `bundles.index.json`
- `sources.index.json`
- `deprecations.json`

Move vocabulary/classification files into:

SYSTEM/taxonomy/

Move

- `taxonomy.json`
- `tags.json`
- `quality_tiers.json`

Add later

- `vocab.pillars.json`
- `vocab.domains.json`
- `vocab.content_types.json`
- `vocab.industries.json`
- `vocab.stack_families.json`
- `vocab.status.json`

4. Move `contracts/`

Move current:

contracts/

to:

SYSTEM/contracts/

Do not reorganize internally yet. Just relocate it.

5. Move `POLICIES/`

Move current:

POLICIES/

to:

SYSTEM/policies/

6. Move `TEMPLATES/`

Move current:

TEMPLATES/

to:

SYSTEM/templates/

This keeps authoring scaffolds out of canonical content.

7. Keep `REUSE/`, but isolate its purpose

Keep it as:

REUSE/

or move contents under:

```
REUSE/licenses/
REUSE/allowlist.json
REUSE/reuse_log.json
```

The point is that reuse/licensing stays separate from content and system logic.

8. Keep `OUTPUTS/`, but make it explicitly non-canonical

Keep or normalize as:

```
OUTPUTS/
 exports/
 selections/
```

Nothing in `OUTPUTS/` should be treated as source-of-truth knowledge.

9. Freeze `PILLARS/`

Do not delete it yet.

Mark it as legacy:

PILLARS/ ← legacy canonical tree during migration only

Rule from this point on:

- no new content added here
- no new category design done here
- only migration out of it

10. Create canonical KID storage

Create:

CONTENT/ITEMS/

This becomes the permanent home for all actual knowledge units.

Every reusable knowledge item eventually goes here.

11. Migrate content from `PILLARS/` by batches

Do this in batches, not all at once.

Batch A

`PILLARS/INDUSTRY_PLAYBOOKS/`

Move all actual KID content files into:

CONTENT/ITEMS/

Batch B

`PILLARS/IT_END_TO_END/`

Move all actual KID content files into:

CONTENT/ITEMS/

Batch C

`PILLARS/LANGUAGES_AND_LIBRARIES/`

Move all actual KID content files into:

CONTENT/ITEMS/

12. What counts as a “content file”

Move only actual reusable knowledge units.

Examples:

- concepts
- patterns
- checklists
- workflows
- references
- standards
- playbooks
- guides

Do not move structural folder-helper files into `CONTENT/ITEMS/`.

13. What to do with `_meta.md`

Do not keep `_meta.md` as buried folder glue in the new system.

Convert each `_meta.md` into one of these:

Option A — collection metadata

Put into:

CONTENT/META/collections/

Examples:

- `industry.financial_services.md`
- `stack.react.md`
- `domain.authentication.md`

Option B — generated browsing descriptor

Put into:

VIEWS/by_industry/`
VIEWS/by_stack_family/`
VIEWS/by_domain/`

Use this if the file is really a browsing summary rather than source content.

14. Metadata every KID must have

When a file moves into `CONTENT/ITEMS/`, add explicit frontmatter.

Minimum required:

```
kid: KID-XXXX
title: Example Title
content_type: concept
primary_domain: authentication
pillar_refs:
 - solution_patterns
status: active
authority_tier: reviewed
tags:
 - oauth
 - session-management
legacy_path: PILLARS/IT_END_TO_END/...
```

Recommended full set:

```
kid: KID-XXXX
title: Example Title
content_type: concept
primary_domain: authentication
secondary_domains: []
industry_refs: []
stack_family_refs: []
pillar_refs: []
status: active
authority_tier: reviewed
freshness_class: versioned_stable
bundle_refs: []
```

tags: []
depends_on: []
related_kids: []
supersedes: []
source_refs: []
legacy_path: PILLARS/...

15. Replace folder meaning with metadata meaning

Today, a lot of meaning comes from the path.

That must stop.

For every moved file, explicitly capture what the old path implied:

- industry
- domain
- stack family
- content type
- grouping label

That becomes metadata instead of directory logic.

16. Create generated browsing views

Once KIDs are in `CONTENT/ITEMS/`, generate browsing indexes into:

VIEWS/by_pillar/
VIEWS/by_domain/
VIEWS/by_content_type/
VIEWS/by_industry/
VIEWS/by_stack_family/
VIEWS/by_status/
VIEWS/by_authority/
VIEWS/by_bundle/

These are for humans and UI browsing only.

They must not become the new canonical tree.

17. Normalize bundle references

Keep bundles, but clean their dependency model.

Rule:

- bundles reference KID IDs only
- bundles do not rely on folder locations
- bundles do not infer category from path

Move current bundle index data into:

SYSTEM/registries/bundles.index.json

And keep curated bundle definitions under:

BUNDLES/

18. Add relationship registries

Create these in:

SYSTEM/registries/

Add

- `aliases.index.json`
- `relationships.index.json`

Use them for:

- renamed KIDs

- alternate names
- supersession mapping
- dependency mapping
- redirects from legacy paths

19. Keep backward compatibility during migration

For a while, you will need both systems to work.

Use:

- `legacy_path` in frontmatter
- `aliases.index.json`
- `redirects/` in `CONTENT/META/redirects/`

This lets old references still resolve while the system transitions.

20. Retire `PILLARS/` in stages

Do not remove it immediately.

Use this sequence:

Stage 1

`PILLARS/` remains readable, but frozen

Stage 2

Resolver prefers:

1. item metadata
2. registries
3. aliases
4. legacy path fallback

Stage 3

All new content goes only to `CONTENT/ITEMS/`

Stage 4

`PILLARS/` becomes generated view or archival snapshot

21. New category rule going forward

When you want to add a new section in the future, do not create a new deep physical tree first.

Instead:

- add or extend a taxonomy value
- classify relevant KIDs with metadata
- generate a new view if needed

That is the main rule that keeps the system clean.

22. What your new primary classification axes should be

Instead of the old 3 physical pillars as storage, classify every item by facets.

Use these as the main axes:

- `content_type`
- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `status`
- `authority_tier`
- `bundle_refs`

This is much more scalable than deep folder nesting.

23. Recommended migration order

Do it in this order:

First

Move non-content systems:

- `KNO-*`
- `INDEX/`
- `contracts/`
- `POLICIES/`
- `TEMPLATES/`

Second

Create `CONTENT/ITEMS/` and the metadata standard

Third

Migrate one pillar family at a time:

- industry
- IT/domain
- languages/libraries

Fourth

Generate views

Fifth

Update resolver logic

Sixth

Retire `PILLARS/`

24. Minimum viable version

If you want the fastest cleanup with the least disruption, do only this first:

Create

- `SYSTEM/`
- `CONTENT/`

- `VIEWS/`

Move

- `KNO-*` → `SYSTEM/doctrine/`
- `INDEX/` → `SYSTEM/registries/` and `SYSTEM/taxonomy/`
- `contracts/` → `SYSTEM/contracts/`
- `POLICIES/` → `SYSTEM/policies/`
- `TEMPLATES/` → `SYSTEM/templates/`

Then enforce one new rule

All new knowledge items go into:

CONTENT/ITEMS/

with metadata.

That alone will stop the structure from getting worse.

25. The shortest version of the checklist

Move:

- library rules → `SYSTEM/`
- actual KIDs → `CONTENT/ITEMS/`
- browsing groupings → `VIEWS/`
- curated packs → `BUNDLES/`

- licensing/reuse → `REUSE/`
- generated artifacts → `OUTPUTS/`

Freeze:

- `PILLARS/`

Then make metadata, not folder path, define structure.

Phase 1 — move the non-content systems first

Phase 1 — move the non-content systems first

These are clean relocations.

Move root doctrine files

From:

Axion/libraries/knowledge/KNO-0_purpose.md
Axion/libraries/knowledge/KNO-1_unit_classes.md
Axion/libraries/knowledge/KNO-2_authority_tiers.md
Axion/libraries/knowledge/KNO-3_freshness_supersession.md
Axion/libraries/knowledge/KNO-4_retrieval_resolution_rules.md
Axion/libraries/knowledge/KNO-5_dependency_mapping.md
Axion/libraries/knowledge/KNO-6_proof_and_trust.md
Axion/libraries/knowledge/KNO-7_definition_of_done.md

To:

Axion/libraries/knowledge/SYSTEM/doctrine/

Split `INDEX/`

From:

Axion/libraries/knowledge/INDEX/

Move to:

`SYSTEM/registries/`

- `knowledge.index.json`
- `knowledge_registry.v1.json`
- `bundles.index.json`

- `sources.index.json`
- `deprecations.json`
- `changelog.md`

`SYSTEM/taxonomy/`

- `taxonomy.json`
- `tags.json`
- `quality_tiers.json`

Move contracts

From:

Axion/libraries/knowledge/contracts/

To:

Axion/libraries/knowledge/SYSTEM/contracts/

Move policies

From:

Axion/libraries/knowledge/POLICIES/

To:

Axion/libraries/knowledge/SYSTEM/policies/

Move templates

From:

Axion/libraries/knowledge/TEMPLATES/

To:

Axion/libraries/knowledge/SYSTEM/templates/

Keep these, but normalize them

Keep in place conceptually:

- `BUNDLES/`
- `REUSE/`
- `OUTPUTS/`

But reshape them to:

Axion/libraries/knowledge/BUNDLES/
Axion/libraries/knowledge/REUSE/
Axion/libraries/knowledge/OUTPUTS/

Phase 2 — freeze the old canonical content tree

Do not delete `PILLARS/` yet.

Mark it as legacy and stop adding anything new there.

Axion/libraries/knowledge/PILLARS/ ← frozen legacy tree

Create the new canonical store:

Axion/libraries/knowledge/CONTENT/ITEMS/
Axion/libraries/knowledge/CONTENT/META/collections/
Axion/libraries/knowledge/CONTENT/META/aliases/
Axion/libraries/knowledge/CONTENT/META/redirects/

Phase 3 — move `INDUSTRY_PLAYBOOKS`

This branch is smaller and cleaner, so it should go first.

Current major structure:

PILLARS/INDUSTRY_PLAYBOOKS/
01_regulated_industries/
02_commerce_and_operations/
03_media_and_creator_economy/
04_emerging_tech_industries/

It also has some parallel roots like:

- `finance/`
- `government_public_sector/`
- `healthcare/`
- `logistics_supply_chain/`
- `retail_ecommerce/`

That means the branch already contains two different organization styles at once.

What to move

Move all actual KID documents from:

PILLARS/INDUSTRY_PLAYBOOKS/**

into:

CONTENT/ITEMS/

What to convert

Convert folder `_meta.md` files into collection descriptors under:

CONTENT/META/collections/

Examples:

industry.financial_services.md
industry.healthcare.md
industry.government_public_sector.md
industry.logistics_supply_chain.md
industry.retail.md
industry.ecommerce.md
industry.saas_b2b.md

Metadata mapping for this branch

Old path meaning becomes metadata:

- `01_regulated_industries` → `pillar_refs: [industry_knowledge]`
- `financial_services`, `payments`, `healthcare`, `insurance`, etc. → `industry_refs`
- `workflows`, `compliance`, `data_models`, `integrations`, `kpis_metrics`, `security_risk` → `content_type` or `primary_domain`

So for example:

PILLARS/INDUSTRY_PLAYBOOKS/01_regulated_industries/payments/compliance/...

becomes a KID in `CONTENT/ITEMS/` with metadata like:

```
industry_refs:
 - payments
pillar_refs:
 - industry_knowledge
content_type: compliance
primary_domain: payments
legacy_path:
PILLARS/INDUSTRY_PLAYBOOKS/01_regulated_industries/payments/compliance/...
```

Phase 4 — move `IT_END_TO_END`

This is the largest system-domain branch and needs a stricter mapping.

Current major structure includes:

- `01_foundations`
- `02_data_and_storage`
- `03_backend_and_apis`
- `04_frontend_and_clients`
- `05_infrastructure_and_platform`
- `06_devops_and_reliability`
- `07_data_engineering_and_ai`

But it also has parallel trees like:

- `02_software_delivery`
- `03_data_systems`
- `04_platform_ops`

So this branch also mixes multiple taxonomies.

What to move

Move all actual KID documents from:

PILLARS/IT_END_TO_END/**

into:

CONTENT/ITEMS/

What to convert

Convert category-level `_meta` files into:

CONTENT/META/collections/

Examples:

domain.api_design.md
domain.authentication_and_sessions.md
domain.postgresql.md
domain.testing_strategies.md
domain.monitoring_and_alerting.md
domain.llm_applications.md
domain.kubernetes.md

Metadata mapping for this branch

Examples:

PILLARS/IT_END_TO_END/03_backend_and_apis/api_design/...

becomes:

pillar_refs:
 - solution_patterns
primary_domain: api_design
content_type: pattern
legacy_path: PILLARS/IT_END_TO_END/03_backend_and_apis/api_design/...
PILLARS/IT_END_TO_END/06_devops_and_reliability/monitoring_and_alerting/...

becomes:

pillar_refs:
 - execution_and_operations
primary_domain: monitoring_and_alerting
content_type: standard
legacy_path: PILLARS/IT_END_TO_END/06_devops_and_reliability/monitoring_and_alerting/...
PILLARS/IT_END_TO_END/07_data_engineering_and_ai/llm_applications/...

becomes:

pillar_refs:
 - solution_patterns
 - technology_knowledge
primary_domain: llm_applications
content_type: pattern
legacy_path: PILLARS/IT_END_TO_END/07_data_engineering_and_ai/llm_applications/...

Phase 5 — move `LANGUAGES_AND_LIBRARIES`

This branch is the clearest example of why a folder tree is hurting you, because it mixes:

- programming languages
- frameworks
- cloud/devops tools
- databases/ORMs
- blockchain stacks
- audio tech
- graphics tech
- streaming/realtime

These are not one taxonomy.

What to move

Move all actual KID documents from:

PILLARS/LANGUAGES_AND_LIBRARIES/**

into:

CONTENT/ITEMS/

What to convert

Convert category `_meta` files into:

CONTENT/META/collections/

Examples:

stack.javascript.md
stack.python.md
stack.react.md
stack.nextjs.md
stack.fastapi.md
stack.postgres_drivers.md
stack.docker.md
stack.aws.md
stack.ffmpeg.md
stack.unreal_engine.md

Metadata mapping for this branch

Examples:

PILLARS/LANGUAGES_AND_LIBRARIES/01_programming_languages/python/...

becomes:

stack_family_refs:
 - python
pillar_refs:
 - technology_knowledge
content_type: reference
primary_domain: python
legacy_path: PILLARS/LANGUAGES_AND_LIBRARIES/01_programming_languages/python/...
PILLARS/LANGUAGES_AND_LIBRARIES/02_web_frameworks/react/...

becomes:

stack_family_refs:
 - react
pillar_refs:
 - technology_knowledge
content_type: pattern
primary_domain: react
legacy_path: PILLARS/LANGUAGES_AND_LIBRARIES/02_web_frameworks/react/...
PILLARS/LANGUAGES_AND_LIBRARIES/05_cloud_and_devops_tooling/docker/...

becomes:

stack_family_refs:
 - docker
pillar_refs:
 - technology_knowledge

- execution_and_operations
content_type: reference
primary_domain: docker
legacy_path:
PILLARS/LANGUAGES_AND_LIBRARIES/05_cloud_and_devops_tooling/docker/...

Phase 6 — keep `BUNDLES`, but simplify what it means

Your current bundle tree is already fairly clean:

- `by_executor`
- `by_risk_class`
- `by_run_profile`

Keep that shape.

But enforce one rule:

Bundles may only point to KID IDs, never to old folder paths.

So keep:

Axion/libraries/knowledge/BUNDLES/by_executor/
Axion/libraries/knowledge/BUNDLES/by_risk_class/
Axion/libraries/knowledge/BUNDLES/by_run_profile/

But update definitions so they select from:

- `kid`
- `tags`
- `bundle_refs`
- `primary_domain`

- `pillar_refs`

not from physical directories.

Phase 7 — create browsing views

After migration, generate views into:

VIEWS/by_pillar/
VIEWS/by_domain/
VIEWS/by_content_type/
VIEWS/by_industry/
VIEWS/by_stack_family/
VIEWS/by_status/
VIEWS/by_authority/
VIEWS/by_bundle/

Concrete examples:

VIEWS/by_industry/financial_services.md
VIEWS/by_industry/healthcare.md
VIEWS/by_domain/authentication_and_sessions.md
VIEWS/by_domain/postgresql.md
VIEWS/by_stack_family/react.md
VIEWS/by_stack_family/python.md
VIEWS/by_pillar/industry_knowledge.md
VIEWS/by_pillar/technology_knowledge.md

These are generated browsing surfaces only.

Phase 8 — add migration support files

Create:

SYSTEM/registries/aliases.index.json
SYSTEM/registries/relationships.index.json
CONTENT/META/redirects/

Use them for:

- renamed KIDs
- old-path redirects
- supersession
- dependency mapping

The practical order I'd do this in

1. Move doctrine, index, contracts, policies, templates.
2. Create `CONTENT/ITEMS/` and the metadata standard.
3. Freeze `PILLARS/`.
4. Migrate `INDUSTRY_PLAYBOOKS`.
5. Migrate `IT_END_TO_END`.
6. Migrate `LANGUAGES_AND_LIBRARIES`.
7. Generate `VIEWS/`.
8. Update resolver logic.
9. Retire path-first lookup.

The direct move summary

Move these whole directories now:

- `INDEX/` → split across `SYSTEM/registries/` and `SYSTEM/taxonomy/`
- `contracts/` → `SYSTEM/contracts/`

- `POLICIES/` → `SYSTEM/policies/`
- `TEMPLATES/` → `SYSTEM/templates/`
- `KNO-*` → `SYSTEM/doctrine/`

Do not move these as-is:

- `PILLARS/` — migrate out of it item-by-item or batch-by-batch
- `BUNDLES/` — keep, but rewire references
- `REUSE/` — keep isolated
- `OUTPUTS/` — keep non-canonical

Directory

I’m keeping it operational and sequential so you can execute it without thinking through the structure every time.

1. Create the new skeleton first

```
mkdir -p Axion/libraries/knowledge/SYSTEM/doctrine
mkdir -p Axion/libraries/knowledge/SYSTEM/taxonomy
mkdir -p Axion/libraries/knowledge/SYSTEM/registries
mkdir -p Axion/libraries/knowledge/SYSTEM/contracts
mkdir -p Axion/libraries/knowledge/SYSTEM/policies
mkdir -p Axion/libraries/knowledge/SYSTEM/templates

mkdir -p Axion/libraries/knowledge/CONTENT/ITEMS
mkdir -p Axion/libraries/knowledge/CONTENT/META/collections
mkdir -p Axion/libraries/knowledge/CONTENT/META/aliases
mkdir -p Axion/libraries/knowledge/CONTENT/META/redirects

mkdir -p Axion/libraries/knowledge/VIEWS/by_pillar
mkdir -p Axion/libraries/knowledge/VIEWS/by_domain
mkdir -p Axion/libraries/knowledge/VIEWS/by_content_type
mkdir -p Axion/libraries/knowledge/VIEWS/by_industry
mkdir -p Axion/libraries/knowledge/VIEWS/by_stack_family
mkdir -p Axion/libraries/knowledge/VIEWS/by_status
mkdir -p Axion/libraries/knowledge/VIEWS/by_authority
mkdir -p Axion/libraries/knowledge/VIEWS/by_bundle

mkdir -p Axion/libraries/knowledge/BUNDLES/by_executor
mkdir -p Axion/libraries/knowledge/BUNDLES/by_risk_class
mkdir -p Axion/libraries/knowledge/BUNDLES/by_run_profile

mkdir -p Axion/libraries/knowledge/REUSE/licenses

mkdir -p Axion/libraries/knowledge/OUTPUTS/exports
mkdir -p Axion/libraries/knowledge/OUTPUTS/selections
```

2. Move doctrine files

```
mv Axion/libraries/knowledge/KNO-0_purpose.md Axion/libraries/knowledge/SYSTEM/doctrine/
mv Axion/libraries/knowledge/KNO-1_unit_classes.md
Axion/libraries/knowledge/SYSTEM/doctrine/
```

mv Axion/libraries/knowledge/KNO-2_authority_tiers.md
Axion/libraries/knowledge/SYSTEM/doctrine/
mv Axion/libraries/knowledge/KNO-3_freshness_supersession.md
Axion/libraries/knowledge/SYSTEM/doctrine/
mv Axion/libraries/knowledge/KNO-4_retrieval_resolution_rules.md
Axion/libraries/knowledge/SYSTEM/doctrine/
mv Axion/libraries/knowledge/KNO-5_dependency_mapping.md
Axion/libraries/knowledge/SYSTEM/doctrine/
mv Axion/libraries/knowledge/KNO-6_proof_and_trust.md
Axion/libraries/knowledge/SYSTEM/doctrine/
mv Axion/libraries/knowledge/KNO-7_definition_of_done.md
Axion/libraries/knowledge/SYSTEM/doctrine/

3. Split `INDEX/` into taxonomy and registries

Registries

mv Axion/libraries/knowledge/INDEX/knowledge.index.json
Axion/libraries/knowledge/SYSTEM/registries/
mv Axion/libraries/knowledge/INDEX/knowledge_registry.v1.json
Axion/libraries/knowledge/SYSTEM/registries/
mv Axion/libraries/knowledge/INDEX/bundles.index.json
Axion/libraries/knowledge/SYSTEM/registries/
mv Axion/libraries/knowledge/INDEX/sources.index.json
Axion/libraries/knowledge/SYSTEM/registries/
mv Axion/libraries/knowledge/INDEX/deprecations.json
Axion/libraries/knowledge/SYSTEM/registries/

Taxonomy

mv Axion/libraries/knowledge/INDEX/taxonomy.json
Axion/libraries/knowledge/SYSTEM/taxonomy/
mv Axion/libraries/knowledge/INDEX/tags.json Axion/libraries/knowledge/SYSTEM/taxonomy/
mv Axion/libraries/knowledge/INDEX/quality_tiers.json
Axion/libraries/knowledge/SYSTEM/taxonomy/

Optional if present

mv Axion/libraries/knowledge/INDEX/changelog.md
Axion/libraries/knowledge/SYSTEM/registries/

4. Move whole system folders

```
mv Axion/libraries/knowledge/contracts/* Axion/libraries/knowledge/SYSTEM/contracts/
mv Axion/libraries/knowledge/POLICIES/* Axion/libraries/knowledge/SYSTEM/policies/
mv Axion/libraries/knowledge/TEMPLATES/* Axion/libraries/knowledge/SYSTEM/templates/
```

After that, delete the emptied old folders only after verifying contents moved cleanly:

```
rmdir Axion/libraries/knowledge/contracts
rmdir Axion/libraries/knowledge/POLICIES
rmdir Axion/libraries/knowledge/TEMPLATES
```

5. Add migration support files now

Create these empty placeholders:

```
touch Axion/libraries/knowledge/SYSTEM/registries/aliases.index.json
touch Axion/libraries/knowledge/SYSTEM/registries/relationships.index.json
touch Axion/libraries/knowledge/CONTENT/META/redirects/README.md
touch Axion/libraries/knowledge/CONTENT/META/collections/README.md
touch Axion/libraries/knowledge/CONTENT/META/aliases/README.md
```

6. Freeze `PILLARS/`

Do not move it yet. Mark it clearly.

Create a note:

```
cat > Axion/libraries/knowledge/PILLARS/README_LEGACY_MIGRATION.md <<'EOF'
This tree is frozen during migration.
Do not add new canonical knowledge items here.
All new KIDs must go into CONTENT/ITEMS with explicit metadata.
EOF
```

7. Set the new rule for all future knowledge files

From this point forward, every new knowledge item goes here:

Axion/libraries/knowledge/CONTENT/ITEMS/

and not into `PILLARS/`.

8. Batch-migrate `INDUSTRY_PLAYBOOKS` first

Do not move whole folders blindly. Move actual KID content files and convert `_meta.md` separately.

Find KID files

```
find Axion/libraries/knowledge/PILLARS/INDUSTRY_PLAYBOOKS -type f -name "KID-*.md"
```

Move them into canonical storage

Example loop:

```
find Axion/libraries/knowledge/PILLARS/INDUSTRY_PLAYBOOKS -type f -name "KID-*.md"
-exec mv {} Axion/libraries/knowledge/CONTENT/ITEMS/ \;
```

Find `_meta.md` files

```
find Axion/libraries/knowledge/PILLARS/INDUSTRY_PLAYBOOKS -type f -name "_meta.md"
```

Move those to collection metadata

Do these one by one with renamed filenames, for example:

```
mv Axion/libraries/knowledge/PILLARS/INDUSTRY_PLAYBOOKS/finance/_meta.md \
  Axion/libraries/knowledge/CONTENT/META/collections/industry.finance.md

mv Axion/libraries/knowledge/PILLARS/INDUSTRY_PLAYBOOKS/healthcare/_meta.md \
  Axion/libraries/knowledge/CONTENT/META/collections/industry.healthcare.md

mv
Axion/libraries/knowledge/PILLARS/INDUSTRY_PLAYBOOKS/government_public_sector/_met
a.md \
  Axion/libraries/knowledge/CONTENT/META/collections/industry.government_public_sector.md
```

9. Then batch-migrate `IT_END_TO_END`

Move KIDs

find Axion/libraries/knowledge/PILLARS/IT_END_TO_END -type f -name "KID-*.md" -exec mv {} Axion/libraries/knowledge/CONTENT/ITEMS/ \;

Locate `_meta.md`

find Axion/libraries/knowledge/PILLARS/IT_END_TO_END -type f -name "_meta.md"

Rename them as domain collections

Examples:

```
mv
Axion/libraries/knowledge/PILLARS/IT_END_TO_END/03_backend_and_apis/api_design/_meta.md \
  Axion/libraries/knowledge/CONTENT/META/collections/domain.api_design.md

mv
Axion/libraries/knowledge/PILLARS/IT_END_TO_END/06_devops_and_reliability/monitoring_and_alerting/_meta.md \
  Axion/libraries/knowledge/CONTENT/META/collections/domain.monitoring_and_alerting.md

mv
Axion/libraries/knowledge/PILLARS/IT_END_TO_END/07_data_engineering_and_ai/llm_applications/_meta.md \
  Axion/libraries/knowledge/CONTENT/META/collections/domain.llm_applications.md
```

10. Then batch-migrate `LANGUAGES_AND_LIBRARIES`

Move KIDs

find Axion/libraries/knowledge/PILLARS/LANGUAGES_AND_LIBRARIES -type f -name "KID-*.md" -exec mv {} Axion/libraries/knowledge/CONTENT/ITEMS/ \;

Locate `_meta.md`

find Axion/libraries/knowledge/PILLARS/LANGUAGES_AND_LIBRARIES -type f -name "_meta.md"

Rename as stack collections

Examples:

```
mv
Axion/libraries/knowledge/PILLARS/LANGUAGES_AND_LIBRARIES/01_programming_languages/python/_meta.md \
  Axion/libraries/knowledge/CONTENT/META/collections/stack.python.md

mv
Axion/libraries/knowledge/PILLARS/LANGUAGES_AND_LIBRARIES/02_web_frameworks/react/_meta.md \
  Axion/libraries/knowledge/CONTENT/META/collections/stack.react.md

mv
Axion/libraries/knowledge/PILLARS/LANGUAGES_AND_LIBRARIES/05_cloud_and_devops_tooling/docker/_meta.md \
  Axion/libraries/knowledge/CONTENT/META/collections/stack.docker.md
```

11. Add frontmatter to moved KIDs

Every file in `CONTENT/ITEMS/` should get explicit metadata.

Minimum template:

```
---
kid: KID-XXXX
title: Replace Me
content_type: concept
primary_domain: replace_me
industry_refs: []
stack_family_refs: []
pillar_refs: []
status: active
authority_tier: reviewed
freshness_class: versioned_stable
bundle_refs: []
```

tags: []
depends_on: []
related_kids: []
supersedes: []
source_refs: []
legacy_path: PILLARS/...
---

12. Keep old path traceability

For each moved file, preserve the original location inside `legacy_path`.

That lets the resolver and your own cleanup process stay stable while the old tree still exists.

13. Update bundles to reference KIDs only

Audit all bundle definitions under:

Axion/libraries/knowledge/BUNDLES/

Replace anything path-based with:

- `kid`
- `bundle_refs`
- `tags`
- `primary_domain`
- `pillar_refs`

No bundle should depend on `PILLARS/...` paths anymore.

14. Generate view placeholders

Create the first browsing files:

```
touch Axion/libraries/knowledge/VIEWS/by_pillar/industry_knowledge.md
touch Axion/libraries/knowledge/VIEWS/by_pillar/solution_patterns.md
touch Axion/libraries/knowledge/VIEWS/by_pillar/technology_knowledge.md
touch Axion/libraries/knowledge/VIEWS/by_pillar/governance_and_quality.md
touch Axion/libraries/knowledge/VIEWS/by_pillar/execution_and_operations.md
```

And examples for the other view types:

```
touch Axion/libraries/knowledge/VIEWS/by_industry/financial_services.md
touch Axion/libraries/knowledge/VIEWS/by_industry/healthcare.md
touch Axion/libraries/knowledge/VIEWS/by_domain/api_design.md
touch Axion/libraries/knowledge/VIEWS/by_domain/monitoring_and_alerting.md
touch Axion/libraries/knowledge/VIEWS/by_stack_family/react.md
touch Axion/libraries/knowledge/VIEWS/by_stack_family/python.md
touch Axion/libraries/knowledge/VIEWS/by_content_type/concept.md
touch Axion/libraries/knowledge/VIEWS/by_content_type/pattern.md
```

15. Clean up old empty directories later

Only after all KIDs and relevant `_meta.md` files are migrated:

```
find Axion/libraries/knowledge/PILLARS -type d -empty
```

Then remove empty directories selectively.

Do not mass-delete until you confirm:

- no unresolved references remain
- no old-path bundle logic remains
- no resolver code still depends on folder shape

16. The order to actually execute

Run this in order:

1. create new skeleton
2. move doctrine files

3. split `INDEX/`
4. move contracts/policies/templates
5. add migration support files
6. freeze `PILLARS/`
7. migrate `INDUSTRY_PLAYBOOKS`
8. migrate `IT_END_TO_END`
9. migrate `LANGUAGES_AND_LIBRARIES`
10. add frontmatter cleanup
11. rewire bundles
12. generate views
13. clean old empty dirs
14. retire path-first resolution

17. The one caution

This command:

find ... -name "KID-*.md" -exec mv {} CONTENT/ITEMS/ \;

is only safe if all KID filenames are unique.

If they are not unique, you need a safer move script that:

- detects collisions
- renames with the `kid` value
- writes a redirect record

That is probably the next thing you should do before executing the mass move.

collision-safe migration script

Use this as the collision-safe migration script outline.

This avoids the biggest risk in flattening everything into `CONTENT/ITEMS/`:

- filename collisions
- accidental overwrite
- lost legacy paths
- inconsistent frontmatter
- broken lookup after moves

What the script should do

The script should:

1. scan `PILLARS/`
2. find every `KID-*.md`
3. read its frontmatter if present
4. determine the canonical filename from `kid`
5. detect collisions before moving
6. preserve original path as `legacy_path`
7. add missing metadata where needed
8. move the file into `CONTENT/ITEMS/`
9. write redirect and alias records
10. produce a migration report

Recommended script location

Put it here:

Axion/scripts/knowledge/migrate_kids_to_canonical.py

Safe migration rules

Rule 1

The target filename should come from the actual `kid`, not the old filename.

So if a file contains:

kid: KID-IND-FINSERV-COMP-0001

the target should be:

CONTENT/ITEMS/KID-IND-FINSERV-COMP-0001.md

Rule 2

If two files resolve to the same `kid`, do not move either automatically.

Flag them in a collision report.

Rule 3

If a file has no `kid`, do not move it automatically.

Put it into a review report.

Rule 4

Always preserve the original source path in metadata.

Rule 5

Never delete the source file on the first run unless you explicitly run in commit mode.

Start with dry-run only.

Folder outputs the script should write

Create these outputs:

Axion/libraries/knowledge/OUTPUTS/selections/migration_report.json
Axion/libraries/knowledge/OUTPUTS/selections/migration_collisions.json
Axion/libraries/knowledge/OUTPUTS/selections/migration_missing_kid.json
Axion/libraries/knowledge/OUTPUTS/selections/migration_moved.json

And update:

Axion/libraries/knowledge/SYSTEM/registries/aliases.index.json
Axion/libraries/knowledge/SYSTEM/registries/relationships.index.json

Suggested metadata behavior

When moving a file, ensure it has at least:

```
---
kid: KID-XXXX
title: Existing or inferred title
content_type: inferred_or_default
primary_domain: inferred_or_unknown
industry_refs: []
stack_family_refs: []
pillar_refs: []
status: active
authority_tier: reviewed
freshness_class: versioned_stable
bundle_refs: []
tags: []
depends_on: []
related_kids: []
supersedes: []
source_refs: []
legacy_path: PILLARS/...
---
```

If some of these are unknown, use conservative placeholders like:

- `content_type: reference`
- `primary_domain: unknown`

But record those in the report so they can be cleaned later.

Collision cases the script must catch

Case 1 — same filename, different KIDs

Example:

- `KID-auth.md`
- `KID-auth.md`

If their internal `kid` values differ, rename by actual `kid` and proceed.

Case 2 — different filenames, same KID

Example:

- `oauth-pattern.md`
- `auth-pattern-v2.md`

Both declare:

- `kid: KID-SOL-AUTH-PATTERN-0012`

This is a real collision. Stop and report it.

Case 3 — no frontmatter

Do not guess too aggressively. Report it.

Case 4 — malformed frontmatter

Do not auto-fix in the move step unless the repair is trivial.

Flag it.

Recommended script flow

Step 1 — scan source files

Walk:

Axion/libraries/knowledge/PILLARS/

and collect all markdown files matching:

- `KID-*.md`
- optionally all `*.md` if you want to inspect nonstandard names too

Step 2 — parse frontmatter

For each file:

- read YAML frontmatter
- extract `kid`
- extract existing metadata
- capture source path

Step 3 — infer fallback metadata

From path segments, infer:

- pillar hint

- domain hint
- industry hint
- stack hint
- content-type hint

But use those as migration defaults only, not truth.

Step 4 — compute destination path

Destination:

Axion/libraries/knowledge/CONTENT/ITEMS/{kid}.md

Step 5 — detect collisions

Build a map:

- `kid -> [source_paths]`

Anything with more than one source path is a collision.

Step 6 — dry-run report

Print:

- total files scanned
- valid KIDs
- missing KIDs
- malformed frontmatter
- collisions
- move candidates

Step 7 — commit mode

Only in `--apply` mode:

- write updated frontmatter
- move file
- write redirect record
- update alias registry
- update relationships registry

Redirect record format

For every moved file, create a redirect entry like:

```
{
 "kid": "KID-SOL-AUTH-PATTERN-0012",
 "legacy_path":
"PILLARS/IT_END_TO_END/03_backend_and_apis/authentication/oauth-pattern.md",
 "current_path": "CONTENT/ITEMS/KID-SOL-AUTH-PATTERN-0012.md"
}
```

Store these in:

CONTENT/META/redirects/

or aggregate them into a registry JSON.

Alias record format

In `aliases.index.json`, store things like:

```
{
 "KID-SOL-AUTH-PATTERN-0012": {
   "aliases": [
     "oauth-pattern",
     "auth-pattern-v2"
```

```
  ]
 }
}
```

Relationships registry format

In `relationships.index.json`, store things like:

```
{
 "KID-SOL-AUTH-PATTERN-0012": {
   "legacy_path":
"PILLARS/IT_END_TO_END/03_backend_and_apis/authentication/oauth-pattern.md",
   "collection_keys": [
     "domain.authentication",
     "pillar.solution_patterns"
   ]
 }
}
```

Python outline

Here is the script skeleton.

```
from __future__ import annotations

import argparse
import json
import re
import shutil
from dataclasses import dataclass, asdict
from pathlib import Path
from typing import Any

import yaml


ROOT = Path("Axion/libraries/knowledge")
SOURCE = ROOT / "PILLARS"
TARGET = ROOT / "CONTENT" / "ITEMS"
OUTPUT = ROOT / "OUTPUTS" / "selections"
ALIASES_PATH = ROOT / "SYSTEM" / "registries" / "aliases.index.json"
```

```
REL_PATH = ROOT / "SYSTEM" / "registries" / "relationships.index.json"


@dataclass
class FileRecord:
    source_path: str
    kid: str | None
    title: str | None
    metadata: dict[str, Any]
    inferred: dict[str, Any]
    destination_path: str | None
    issues: list[str]


def read_text(path: Path) -> str:
    return path.read_text(encoding="utf-8")


def write_text(path: Path, text: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(text, encoding="utf-8")


def parse_frontmatter(text: str) -> tuple[dict[str, Any], str]:
    if not text.startswith("---\n"):
        return {}, text
    parts = text.split("---\n", 2)
    if len(parts) < 3:
        return {}, text
    _, yaml_block, body = parts[0], parts[1], parts[2]
    try:
        data = yaml.safe_load(yaml_block) or {}
        if not isinstance(data, dict):
            return {}, text
        return data, body
    except Exception:
        return {}, text


def build_frontmatter(meta: dict[str, Any], body: str) -> str:
    yaml_block = yaml.safe_dump(meta, sort_keys=False, allow_unicode=True).strip()
    return f"---\n{yaml_block}\n---\n{body.lstrip()}"
```

```
def infer_from_path(path: Path) -> dict[str, Any]:
    parts = [p.lower() for p in path.parts]

    inferred = {
        "pillar_refs": [],
        "industry_refs": [],
        "stack_family_refs": [],
        "primary_domain": "unknown",
        "content_type": "reference",
    }

    if "industry_playbooks" in parts:
        inferred["pillar_refs"] = ["industry_knowledge"]
    elif "it_end_to_end" in parts:
        inferred["pillar_refs"] = ["solution_patterns"]
    elif "languages_and_libraries" in parts:
        inferred["pillar_refs"] = ["technology_knowledge"]

    content_type_candidates = {
        "concepts": "concept",
        "patterns": "pattern",
        "checklists": "checklist",
        "workflows": "workflow",
        "compliance": "compliance",
        "integrations": "integration",
        "references": "reference",
        "standards": "standard",
        "playbooks": "playbook",
        "security_risk": "security_risk",
        "data_models": "data_model",
        "kpis_metrics": "kpi_metric",
    }

    for part in parts:
        if part in content_type_candidates:
            inferred["content_type"] = content_type_candidates[part]

    return inferred


def normalize_metadata(meta: dict[str, Any], inferred: dict[str, Any], legacy_path: str) -> dict[str,
Any]:
    out = dict(meta)
```

```
    out.setdefault("kid", meta.get("kid"))
    out.setdefault("title", meta.get("title", "Untitled"))
    out.setdefault("content_type", inferred["content_type"])
    out.setdefault("primary_domain", inferred["primary_domain"])
    out.setdefault("industry_refs", [])
    out.setdefault("stack_family_refs", [])
    out.setdefault("pillar_refs", inferred["pillar_refs"])
    out.setdefault("status", "active")
    out.setdefault("authority_tier", "reviewed")
    out.setdefault("freshness_class", "versioned_stable")
    out.setdefault("bundle_refs", [])
    out.setdefault("tags", [])
    out.setdefault("depends_on", [])
    out.setdefault("related_kids", [])
    out.setdefault("supersedes", [])
    out.setdefault("source_refs", [])
    out["legacy_path"] = legacy_path

    return out


def load_json(path: Path) -> Any:
    if not path.exists():
        return {}
    return json.loads(path.read_text(encoding="utf-8"))


def save_json(path: Path, data: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, indent=2), encoding="utf-8")


def scan_files() -> list[FileRecord]:
    records: list[FileRecord] = []

    for path in SOURCE.rglob("*.md"):
        if path.name == "_meta.md":
            continue

        text = read_text(path)
        meta, body = parse_frontmatter(text)
        inferred = infer_from_path(path)
        issues: list[str] = []
```

```
        kid = meta.get("kid")
        title = meta.get("title")

        if not kid:
            issues.append("missing_kid")

        dest = str(TARGET / f"{kid}.md") if kid else None

        records.append(
            FileRecord(
                source_path=str(path),
                kid=kid,
                title=title,
                metadata=meta,
                inferred=inferred,
                destination_path=dest,
                issues=issues,
            )
        )

    return records


def detect_collisions(records: list[FileRecord]) -> dict[str, list[str]]:
    by_kid: dict[str, list[str]] = {}
    for r in records:
        if not r.kid:
            continue
        by_kid.setdefault(r.kid, []).append(r.source_path)

    return {kid: paths for kid, paths in by_kid.items() if len(paths) > 1}


def apply_moves(records: list[FileRecord], collisions: dict[str, list[str]]) -> list[dict[str, str]]:
    moved: list[dict[str, str]] = []
    aliases = load_json(ALIASES_PATH)
    relationships = load_json(REL_PATH)

    for r in records:
        if not r.kid or r.kid in collisions:
            continue

        source = Path(r.source_path)
        target = Path(r.destination_path)
```

```
        text = read_text(source)
        meta, body = parse_frontmatter(text)

        normalized = normalize_metadata(meta, r.inferred, r.source_path)
        new_text = build_frontmatter(normalized, body)

        write_text(target, new_text)

        alias_bucket = aliases.setdefault(r.kid, {"aliases": []})
        stem = source.stem
        if stem != r.kid and stem not in alias_bucket["aliases"]:
            alias_bucket["aliases"].append(stem)

        relationships[r.kid] = {
            "legacy_path": r.source_path,
            "current_path": str(target),
            "collection_keys": [],
        }

        moved.append(
            {
                "kid": r.kid,
                "legacy_path": r.source_path,
                "current_path": str(target),
            }
        )

    save_json(ALIASES_PATH, aliases)
    save_json(REL_PATH, relationships)
    return moved


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--apply", action="store_true")
    args = parser.parse_args()

    OUTPUT.mkdir(parents=True, exist_ok=True)

    records = scan_files()
    collisions = detect_collisions(records)

    missing_kid = [asdict(r) for r in records if "missing_kid" in r.issues]
    collision_report = [{"kid": k, "paths": v} for k, v in collisions.items()]
```

```
    all_report = [asdict(r) for r in records]

    save_json(OUTPUT / "migration_report.json", all_report)
    save_json(OUTPUT / "migration_collisions.json", collision_report)
    save_json(OUTPUT / "migration_missing_kid.json", missing_kid)

    if args.apply:
        moved = apply_moves(records, collisions)
        save_json(OUTPUT / "migration_moved.json", moved)
    else:
        save_json(OUTPUT / "migration_moved.json", [])

    print(f"Scanned: {len(records)}")
    print(f"Missing kid: {len(missing_kid)}")
    print(f"Collisions: {len(collision_report)}")
    print("Mode:", "apply" if args.apply else "dry-run")


if __name__ == "__main__":
    main()
```

How to run it

Dry run first:

python Axion/scripts/knowledge/migrate_kids_to_canonical.py

Then inspect:

- `migration_report.json`
- `migration_collisions.json`
- `migration_missing_kid.json`

Only after that, commit mode:

python Axion/scripts/knowledge/migrate_kids_to_canonical.py --apply

One improvement I strongly recommend

Do not use plain `shutil.move` for the first real pass.

Write the canonical file to `CONTENT/ITEMS/` first and leave the original in place.

That gives you:

- non-destructive migration
- easier diff review
- safer rollback

Then once verified, you can do a second cleanup pass that removes or archives originals.

Best practice for first migration pass

Use this sequence:

1. scan
2. detect collisions
3. write canonical copies only
4. update registries
5. verify resolver behavior
6. archive or remove legacy originals later

That is much safer than immediate move/delete.

What this gives you

After this script, you will have:

- flattened canonical KIDs
- collision detection

- preserved legacy paths
- alias records
- migration reports
- a safe bridge from old tree to new tree

companion script

Use this as the companion `_meta.md` conversion plan.

This handles the second risky part of the migration:

- buried collection meaning
- folder summaries tied to old paths
- inconsistent `_meta.md` naming
- losing browsing context when flattening KIDs

The goal is to convert `_meta.md` files from path glue into explicit collection descriptors.

What `_meta.md` should become

In the new system, `_meta.md` files should not stay inside deep legacy trees.

Each one should become one of these:

- `CONTENT/META/collections/*.md` for canonical collection metadata
- `VIEWS/by_*/...` for generated browse pages

For migration, treat them first as:

Axion/libraries/knowledge/CONTENT/META/collections/

What the script should do

The script should:

1. scan `PILLARS/` for `_meta.md`
2. infer the collection type from the path
3. generate a stable collection key

4. convert the file into a normalized collection descriptor
5. preserve the old location as `legacy_path`
6. write the result into `CONTENT/META/collections/`
7. produce a report of ambiguous cases

Recommended script location

Axion/scripts/knowledge/migrate_meta_to_collections.py

New collection filename rules

Use stable filenames based on collection type.

Examples:

- `industry.financial_services.md`
- `industry.healthcare.md`
- `domain.api_design.md`
- `domain.monitoring_and_alerting.md`
- `stack.react.md`
- `stack.python.md`
- `pillar.industry_knowledge.md`

Do not preserve `_meta.md` as the final filename.

Collection descriptor shape

Each converted collection file should have frontmatter like this:

```
---
collection_key: industry.financial_services
collection_type: industry
title: Financial Services
status: active
legacy_path: PILLARS/INDUSTRY_PLAYBOOKS/finance/_meta.md
source_branch: INDUSTRY_PLAYBOOKS
scope:
 industry_refs:
   - financial_services
 primary_domain: null
 stack_family_refs: []
 pillar_refs:
   - industry_knowledge
related_views:
 - VIEWS/by_industry/financial_services.md
related_kids: []
notes: []
---
```

Then keep the existing body text from `_meta.md` underneath.

How to infer collection type

If path includes `INDUSTRY_PLAYBOOKS`

Default to:

- `collection_type: industry`

If path includes `IT_END_TO_END`

Default to:

- `collection_type: domain`

If path includes `LANGUAGES_AND_LIBRARIES`

Default to:

- `collection_type: stack_family`

If `_meta.md` sits at a very high branch level

It may instead represent:

- pillar
- category
- subdomain

Those should be flagged for review if inference is weak.

Ambiguity rules

The script should not force certainty where the path is unclear.

Flag for review when:

- the path is too shallow
- the collection name is generic like `01_foundations`
- multiple interpretations are plausible
- the body conflicts with the inferred type

Examples likely needing review:

- `01_foundations`
- `02_commerce_and_operations`

- `04_platform_ops`

Output reports

Write these:

Axion/libraries/knowledge/OUTPUTS/selections/meta_migration_report.json
Axion/libraries/knowledge/OUTPUTS/selections/meta_migration_ambiguous.json
Axion/libraries/knowledge/OUTPUTS/selections/meta_migration_written.json

Suggested normalization rules

Convert branch prefixes into semantic refs

Examples:

- `INDUSTRY_PLAYBOOKS` → `pillar_refs: [industry_knowledge]`
- `IT_END_TO_END` → `pillar_refs: [solution_patterns]`
- `LANGUAGES_AND_LIBRARIES` → `pillar_refs: [technology_knowledge]`

Strip numeric ordering prefixes

Examples:

- `01_regulated_industries` → `regulated_industries`
- `03_backend_and_apis` → `backend_and_apis`

These should not become user-facing collection names as-is unless you explicitly want them.

Convert slugs to readable titles

Examples:

- `financial_services` → `Financial Services`
- `monitoring_and_alerting` → `Monitoring and Alerting`
- `nextjs` → `Next.js`

Python outline

```
from __future__ import annotations

import argparse
import json
import re
from dataclasses import dataclass, asdict
from pathlib import Path
from typing import Any

import yaml


ROOT = Path("Axion/libraries/knowledge")
SOURCE = ROOT / "PILLARS"
TARGET = ROOT / "CONTENT" / "META" / "collections"
OUTPUT = ROOT / "OUTPUTS" / "selections"


@dataclass
class MetaRecord:
    source_path: str
    collection_key: str | None
    collection_type: str | None
    title: str | None
    inferred_scope: dict[str, Any]
    destination_path: str | None
    issues: list[str]


def read_text(path: Path) -> str:
    return path.read_text(encoding="utf-8")
```

```
def write_text(path: Path, text: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(text, encoding="utf-8")


def parse_frontmatter(text: str) -> tuple[dict[str, Any], str]:
    if not text.startswith("---\n"):
        return {}, text
    parts = text.split("---\n", 2)
    if len(parts) < 3:
        return {}, text
    try:
        meta = yaml.safe_load(parts[1]) or {}
        if not isinstance(meta, dict):
            return {}, text
        return meta, parts[2]
    except Exception:
        return {}, text


def slug_to_title(slug: str) -> str:
    parts = slug.replace("-", "_").split("_")
    return " ".join(p.capitalize() for p in parts if p)


def strip_numeric_prefix(value: str) -> str:
    return re.sub(r"^\d+_", "", value)


def infer_collection(path: Path) -> tuple[str | None, str | None, dict[str, Any], list[str]]:
    parts = list(path.parts)
    lower_parts = [p.lower() for p in parts]
    issues: list[str] = []

    inferred_scope = {
        "industry_refs": [],
        "stack_family_refs": [],
        "pillar_refs": [],
        "primary_domain": None,
    }

    branch = None
    if "industry_playbooks" in lower_parts:
        branch = "INDUSTRY_PLAYBOOKS"
```

```
    inferred_scope["pillar_refs"] = ["industry_knowledge"]
elif "it_end_to_end" in lower_parts:
    branch = "IT_END_TO_END"
    inferred_scope["pillar_refs"] = ["solution_patterns"]
elif "languages_and_libraries" in lower_parts:
    branch = "LANGUAGES_AND_LIBRARIES"
    inferred_scope["pillar_refs"] = ["technology_knowledge"]

if not branch:
    return None, None, inferred_scope, ["unknown_branch"]

path_after_branch = lower_parts[lower_parts.index(branch.lower()) + 1 : -1]
cleaned = [strip_numeric_prefix(p) for p in path_after_branch if p]

if not cleaned:
    return None, None, inferred_scope, ["path_too_shallow"]

leaf = cleaned[-1]

if branch == "INDUSTRY_PLAYBOOKS":
    collection_type = "industry"
    collection_key = f"industry.{leaf}"
    inferred_scope["industry_refs"] = [leaf]
elif branch == "IT_END_TO_END":
    collection_type = "domain"
    collection_key = f"domain.{leaf}"
    inferred_scope["primary_domain"] = leaf
else:
    collection_type = "stack_family"
    collection_key = f"stack.{leaf}"
    inferred_scope["stack_family_refs"] = [leaf]

generic_names = {
    "foundations",
    "commerce_and_operations",
    "platform_ops",
    "backend_and_apis",
    "frontend_and_clients",
    "data_and_storage",
}
if leaf in generic_names:
    issues.append("ambiguous_generic_collection")

return collection_key, collection_type, inferred_scope, issues
```

```
def build_descriptor(
    record: MetaRecord,
    original_meta: dict[str, Any],
    body: str,
) -> str:
    title = record.title or slug_to_title(record.collection_key.split(".", 1)[1])

    frontmatter = {
        "collection_key": record.collection_key,
        "collection_type": record.collection_type,
        "title": title,
        "status": "active",
        "legacy_path": record.source_path,
        "scope": record.inferred_scope,
        "related_views": [],
        "related_kids": [],
        "notes": [],
    }

    if original_meta:
        for key in ("status", "notes", "related_kids"):
            if key in original_meta and original_meta[key] not in (None, "", []):
                frontmatter[key] = original_meta[key]

    yaml_block = yaml.safe_dump(frontmatter, sort_keys=False, allow_unicode=True).strip()
    return f"---\n{yaml_block}\n---\n{body.lstrip()}"


def scan_meta_files() -> list[MetaRecord]:
    records: list[MetaRecord] = []

    for path in SOURCE.rglob("_meta.md"):
        text = read_text(path)
        original_meta, body = parse_frontmatter(text)
        collection_key, collection_type, scope, issues = infer_collection(path)

        title = original_meta.get("title") if isinstance(original_meta, dict) else None
        dest = str(TARGET / f"{collection_key}.md") if collection_key else None

        records.append(
            MetaRecord(
                source_path=str(path),
```

```
            collection_key=collection_key,
            collection_type=collection_type,
            title=title,
            inferred_scope=scope,
            destination_path=dest,
            issues=issues,
        )
    )

  return records


def detect_collection_collisions(records: list[MetaRecord]) -> dict[str, list[str]]:
  mapping: dict[str, list[str]] = {}
  for r in records:
    if not r.collection_key:
      continue
    mapping.setdefault(r.collection_key, []).append(r.source_path)
  return {k: v for k, v in mapping.items() if len(v) > 1}


def save_json(path: Path, data: Any) -> None:
  path.parent.mkdir(parents=True, exist_ok=True)
  path.write_text(json.dumps(data, indent=2), encoding="utf-8")


def apply(records: list[MetaRecord], collisions: dict[str, list[str]]) -> list[dict[str, str]]:
  written: list[dict[str, str]] = []

  for r in records:
    if not r.collection_key or r.collection_key in collisions:
      continue

    source = Path(r.source_path)
    text = read_text(source)
    original_meta, body = parse_frontmatter(text)
    output_text = build_descriptor(r, original_meta, body)

    target = Path(r.destination_path)
    write_text(target, output_text)

    written.append(
      {
        "collection_key": r.collection_key,
```

```
            "legacy_path": r.source_path,
            "current_path": str(target),
        }
    )

    return written


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--apply", action="store_true")
    args = parser.parse_args()

    OUTPUT.mkdir(parents=True, exist_ok=True)

    records = scan_meta_files()
    collisions = detect_collection_collisions(records)

    ambiguous = [
        asdict(r)
        for r in records
        if r.issues or not r.collection_key or not r.collection_type
    ]

    save_json(OUTPUT / "meta_migration_report.json", [asdict(r) for r in records])
    save_json(
        OUTPUT / "meta_migration_ambiguous.json",
        ambiguous + [{"collection_key": k, "paths": v} for k, v in collisions.items()],
    )

    if args.apply:
        written = apply(records, collisions)
    else:
        written = []

    save_json(OUTPUT / "meta_migration_written.json", written)

    print(f"Scanned: {len(records)}")
    print(f"Ambiguous: {len(ambiguous)}")
    print(f"Collisions: {len(collisions)}")
    print("Mode:", "apply" if args.apply else "dry-run")


if __name__ == "__main__":
```

main()

How to run it

Dry run first:

python Axion/scripts/knowledge/migrate_meta_to_collections.py

Then inspect:

- `meta_migration_report.json`
- `meta_migration_ambiguous.json`

Then apply:

python Axion/scripts/knowledge/migrate_meta_to_collections.py --apply

Good review sequence

Run the KID migration first, then the `_meta.md` migration.

Review in this order:

1. `migration_collisions.json`
2. `migration_missing_kid.json`
3. `meta_migration_ambiguous.json`
4. spot-check `CONTENT/ITEMS/`
5. spot-check `CONTENT/META/collections/`

resolver rewrite plan

Next is the resolver rewrite plan.

This is the part that makes the new organization actually work.
Until the resolver stops depending on path shape, the old tree still controls too much.

Goal

Change resolution so the system answers:

- what this item is
- where it belongs
- what bundle/view it appears in
- what it replaces or depends on

from **metadata and registries first**, not from folder location.

New resolution order

Use this exact priority:

1. Canonical item metadata

Primary source of truth:

- `CONTENT/ITEMS/KID-*.md`
- frontmatter on the item itself

This should answer:

- `kid`
- `title`

- `content_type`
- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `status`
- `authority_tier`
- `bundle_refs`

2. System registries

Second source:

- `SYSTEM/registries/knowledge.index.json`
- `SYSTEM/registries/knowledge_registry.v1.json`
- `SYSTEM/registries/aliases.index.json`
- `SYSTEM/registries/relationships.index.json`
- `SYSTEM/registries/bundles.index.json`

These should answer:

- item lookup
- aliases
- redirects

- dependency links
- bundle membership
- deprecations/supersession

3. Collection descriptors

Third source:

- `CONTENT/META/collections/*.md`

These help with:

- browse grouping
- collection summaries
- inherited context for views

But they should not override item-level metadata unless explicitly designed to.

4. Generated views

Fourth source:

- `VIEWS/by_*`

These are navigation only, not truth.

5. Legacy path fallback

Last resort only during migration:

- old `PILLARS/...` path parsing

This should exist only as a temporary compatibility layer.

What the resolver should stop doing

Remove these behaviors over time:

- deriving domain from folder names
- inferring meaning from parent directories first
- routing by `kid.path` substring matches
- treating path layout as taxonomy
- coupling bundle selection to directory structure

That is the old system.

What the resolver should do instead

The resolver should answer queries using a structured pipeline.

Resolver pipeline

Step 1 — identify target

Given an input like:

- KID
- alias
- old filename
- old path
- tag/domain query
- bundle query

normalize it into a candidate lookup key.

Examples:

- `KID-SOL-AUTH-PATTERN-0012`
- `oauth-pattern`
- `PILLARS/IT_END_TO_END/.../oauth-pattern.md`
- `react`
- `financial_services`

Step 2 — resolve direct KID

If the input is already a KID:

- go directly to `CONTENT/ITEMS/{kid}.md`
- confirm metadata exists
- enrich from registries

Step 3 — resolve alias

If not a direct KID:

- check `aliases.index.json`

Map:

- filename aliases
- old slugs
- shorthand names

- legacy identifiers

Step 4 — resolve redirect / legacy path

If input matches an old path or old location:

- check `relationships.index.json`
- map `legacy_path -> current_path` or `legacy_path -> kid`

Step 5 — resolve by metadata query

If the user is browsing by concept like:

- industry
- domain
- stack
- content type
- status
- authority

search the item registry and frontmatter fields directly.

Step 6 — enrich with collection context

Once an item is found, optionally attach:

- collection summaries
- related views
- related kids

- bundle references

Core resolver functions

I would break it into these functions.

`resolveKid(input)`

Returns a canonical KID if the input is:

- KID
- alias
- legacy path
- legacy filename

`loadItem(kid)`

Loads:

- markdown body
- frontmatter
- canonical path

`loadRelationships(kid)`

Loads:

- dependencies
- related items
- supersedes

- replaced_by
- legacy path

`loadCollectionsForItem(item)`

Finds matching collection descriptors from:

- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`

`loadBundlesForItem(item)`

Returns:

- explicit `bundle_refs`
- registry-driven membership if any

`searchItems(filters)`

Searches items by:

- content type
- domain
- industry
- stack
- pillar

- status
- authority tier
- tags

Suggested data contracts

aliases.index.json

```
{
 "KID-SOL-AUTH-PATTERN-0012": {
   "aliases": [
     "oauth-pattern",
     "auth-pattern-v2"
   ]
 }
}
```

relationships.index.json

```
{
 "KID-SOL-AUTH-PATTERN-0012": {
   "legacy_path":
"PILLARS/IT_END_TO_END/03_backend_and_apis/authentication/oauth-pattern.md",
   "current_path": "CONTENT/ITEMS/KID-SOL-AUTH-PATTERN-0012.md",
   "depends_on": ["KID-SOL-IDENTITY-CONCEPT-0004"],
   "related_kids": ["KID-SOL-AUTH-CHECKLIST-0007"],
   "supersedes": []
 }
}
```

knowledge.index.json

This should become a fast lookup table, not a second competing truth.

Example entry:

```
{
 "kid": "KID-SOL-AUTH-PATTERN-0012",
```

"title": "OAuth Integration Pattern",
"path": "CONTENT/ITEMS/KID-SOL-AUTH-PATTERN-0012.md",
"content_type": "pattern",
"primary_domain": "authentication",
"industry_refs": [],
"stack_family_refs": [],
"pillar_refs": ["solution_patterns"],
"status": "active",
"authority_tier": "reviewed",
"tags": ["oauth", "identity", "session-management"]
}

Resolver module layout

I'd split it like this:

```
Axion/src/knowledge/
 resolver/
   resolveKid.ts
   loadItem.ts
   loadRelationships.ts
   loadCollections.ts
   searchItems.ts
   searchBundles.ts
   legacyFallback.ts
   types.ts
   index.ts
```

Recommended TypeScript types

```
export type KnowledgeItem = {
 kid: string;
 title: string;
 path: string;
 content_type: string;
 primary_domain: string;
 secondary_domains?: string[];
 industry_refs: string[];
 stack_family_refs: string[];
 pillar_refs: string[];
 status: string;
 authority_tier: string;
```

```
 freshness_class?: string;
 bundle_refs: string[];
 tags: string[];
 depends_on?: string[];
 related_kids?: string[];
 supersedes?: string[];
 legacy_path?: string;
};

export type AliasIndex = Record<string, { aliases: string[] }>;

export type RelationshipEntry = {
 legacy_path?: string;
 current_path: string;
 depends_on?: string[];
 related_kids?: string[];
 supersedes?: string[];
};

export type RelationshipIndex = Record<string, RelationshipEntry>;
```

Resolution behavior examples

Example 1 — direct KID

Input:
`KID-SOL-AUTH-PATTERN-0012`

Resolver:

1. check item exists in `CONTENT/ITEMS`
2. load metadata
3. merge relationships
4. return canonical result

Example 2 — alias

Input:
`oauth-pattern`

Resolver:

1. alias lookup
2. map to KID
3. load item
4. return canonical result

Example 3 — old path

Input:

`PILLARS/IT_END_TO_END/03_backend_and_apis/authentication/oauth-pattern.md`

Resolver:

1. relationships/redirect lookup
2. map to current KID
3. return canonical item

Example 4 — browse query

Input:
`react`

Resolver:

1. search `primary_domain`, `stack_family_refs`, `tags`, aliases
2. return matching KIDs
3. optionally attach collection `stack.react`

Migration-safe resolver behavior

During transition, the resolver should expose where the result came from.

Example:


```
{
 "kid": "KID-SOL-AUTH-PATTERN-0012",
 "resolved_from": "alias",
 "matched_input": "oauth-pattern",
 "legacy_path":
"PILLARS/IT_END_TO_END/03_backend_and_apis/authentication/oauth-pattern.md",
 "current_path": "CONTENT/ITEMS/KID-SOL-AUTH-PATTERN-0012.md"
}
```


That will make debugging much easier.

Search behavior after the rewrite

Instead of searching folders, search these fields:

- `kid`
- `title`
- `tags`
- `primary_domain`
- `secondary_domains`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `bundle_refs`

- aliases

That gives you true faceted retrieval.

What to keep temporarily

Keep a `legacyFallback.ts` module for now.

It can:

- parse old paths
- infer old categories
- redirect to new KIDs when possible

But treat it as temporary and low-priority.

Rewrite phases

Phase 1

Add the new resolver modules without deleting the old resolver.

Phase 2

Make direct KID resolution prefer `CONTENT/ITEMS`.

Phase 3

Add alias and relationship resolution.

Phase 4

Add collection-aware browse/search.

Phase 5

Make legacy path parsing fallback-only.

Phase 6

Remove path-derived taxonomy logic.

Definition of done for the resolver rewrite

You are done when:

- direct KID lookup no longer depends on `PILLARS/`
- alias lookup works
- old paths redirect cleanly
- bundles do not depend on folder shape
- searches operate on metadata fields
- views are generated from metadata
- path parsing is only fallback compatibility

Best next implementation order

1. create `aliases.index.json` and `relationships.index.json`
2. update `knowledge.index.json` entries to point to `CONTENT/ITEMS`
3. write `resolveKid`
4. write `loadItem`
5. write `legacy path -> kid` redirect logic

6. write `searchItems`

7. swap the old resolver to use the new pipeline

TypeScript implementation

This is the bridge from architecture to actual code shape.

Target module layout

```
Axion/src/knowledge/
 resolver/
   types.ts
   paths.ts
   registry.ts
   resolveKid.ts
   loadItem.ts
   loadRelationships.ts
   loadCollections.ts
   searchItems.ts
   legacyFallback.ts
   index.ts
```

1. `types.ts`

This holds the core data contracts.

```
export type KnowledgeFrontmatter = {
 kid: string;
 title: string;
 content_type: string;
 primary_domain: string;
 secondary_domains?: string[];
 industry_refs: string[];
 stack_family_refs: string[];
 pillar_refs: string[];
 status: string;
 authority_tier: string;
 freshness_class?: string;
 bundle_refs: string[];
 tags: string[];
 depends_on?: string[];
 related_kids?: string[];
 supersedes?: string[];
 source_refs?: string[];
 legacy_path?: string;
};
```

```
export type KnowledgeItem = {
 kid: string;
 path: string;
 body: string;
 frontmatter: KnowledgeFrontmatter;
};

export type AliasIndex = Record<
 string,
 {
   aliases: string[];
 }
>;

export type RelationshipEntry = {
 legacy_path?: string;
 current_path: string;
 depends_on?: string[];
 related_kids?: string[];
 supersedes?: string[];
 replaced_by?: string[];
 collection_keys?: string[];
};

export type RelationshipIndex = Record<string, RelationshipEntry>;

export type KnowledgeIndexEntry = {
 kid: string;
 title: string;
 path: string;
 content_type: string;
 primary_domain: string;
 industry_refs: string[];
 stack_family_refs: string[];
 pillar_refs: string[];
 status: string;
 authority_tier: string;
 bundle_refs?: string[];
 tags?: string[];
};

export type KnowledgeIndex = KnowledgeIndexEntry[];
```

```
export type CollectionDescriptor = {
 collection_key: string;
 collection_type: string;
 title: string;
 status: string;
 legacy_path?: string;
 scope: {
   industry_refs?: string[];
   stack_family_refs?: string[];
   pillar_refs?: string[];
   primary_domain?: string | null;
 };
 related_views?: string[];
 related_kids?: string[];
 notes?: string[];
 body?: string;
};

export type ResolveSource =
 | "kid"
 | "alias"
 | "legacy_path"
 | "legacy_filename"
 | "search_match";

export type ResolveResult = {
 kid: string;
 resolved_from: ResolveSource;
 matched_input: string;
 current_path: string;
 legacy_path?: string;
};

export type SearchFilters = {
 text?: string;
 content_type?: string;
 primary_domain?: string;
 industry_refs?: string[];
 stack_family_refs?: string[];
 pillar_refs?: string[];
 status?: string;
 authority_tier?: string;
 bundle_refs?: string[];
 tags?: string[];
```

```
 limit?: number;
};

export type SearchResult = {
 item: KnowledgeIndexEntry;
 score: number;
 matched_fields: string[];
};
```

2. paths.ts

This centralizes all filesystem locations.

```
import path from "node:path";

export const KNOWLEDGE_ROOT = path.resolve("Axion/libraries/knowledge");

export const CONTENT_ITEMS_DIR = path.join(KNOWLEDGE_ROOT, "CONTENT", "ITEMS");
export const COLLECTIONS_DIR = path.join(KNOWLEDGE_ROOT, "CONTENT", "META",
"collections");

export const REGISTRIES_DIR = path.join(KNOWLEDGE_ROOT, "SYSTEM", "registries");

export const KNOWLEDGE_INDEX_PATH = path.join(REGISTRIES_DIR,
"knowledge.index.json");
export const ALIASES_INDEX_PATH = path.join(REGISTRIES_DIR, "aliases.index.json");
export const RELATIONSHIPS_INDEX_PATH = path.join(REGISTRIES_DIR,
"relationships.index.json");

export const LEGACY_PILLARS_DIR = path.join(KNOWLEDGE_ROOT, "PILLARS");
```

3. registry.ts

This module loads and caches indexes.

```
import fs from "node:fs";
import {
 ALIASES_INDEX_PATH,
 KNOWLEDGE_INDEX_PATH,
 RELATIONSHIPS_INDEX_PATH,
} from "./paths";
```

```
import type {
 AliasIndex,
 KnowledgeIndex,
 RelationshipIndex,
} from "./types";

function readJsonFile<T>(filePath: string, fallback: T): T {
 if (!fs.existsSync(filePath)) return fallback;
 const raw = fs.readFileSync(filePath, "utf8");
 return JSON.parse(raw) as T;
}

let knowledgeIndexCache: KnowledgeIndex | null = null;
let aliasIndexCache: AliasIndex | null = null;
let relationshipIndexCache: RelationshipIndex | null = null;

export function loadKnowledgeIndex(): KnowledgeIndex {
 if (!knowledgeIndexCache) {
   knowledgeIndexCache = readJsonFile<KnowledgeIndex>(KNOWLEDGE_INDEX_PATH, []);
 }
 return knowledgeIndexCache;
}

export function loadAliasIndex(): AliasIndex {
 if (!aliasIndexCache) {
   aliasIndexCache = readJsonFile<AliasIndex>(ALIASES_INDEX_PATH, {});
 }
 return aliasIndexCache;
}

export function loadRelationshipIndex(): RelationshipIndex {
 if (!relationshipIndexCache) {
   relationshipIndexCache = readJsonFile<RelationshipIndex>(RELATIONSHIPS_INDEX_PATH,
{});
 }
 return relationshipIndexCache;
}

export function clearResolverCaches(): void {
 knowledgeIndexCache = null;
 aliasIndexCache = null;
 relationshipIndexCache = null;
}
```

4. `resolveKid.ts`

This is the most important module.

Resolution order:

1. direct KID
2. alias
3. legacy path
4. legacy filename
5. fail

```
import path from "node:path";
import { loadAliasIndex, loadKnowledgeIndex, loadRelationshipIndex } from "./registry";
import type { ResolveResult } from "./types";

function isKid(value: string): boolean {
 return /^KID-[A-Z0-9-_]+$/i.test(value.trim());
}

function normalizeInput(value: string): string {
 return value.trim();
}

export function resolveKid(input: string): ResolveResult | null {
 const normalized = normalizeInput(input);
 const knowledgeIndex = loadKnowledgeIndex();
 const aliasIndex = loadAliasIndex();
 const relationshipIndex = loadRelationshipIndex();

 if (isKid(normalized)) {
   const match = knowledgeIndex.find((entry) => entry.kid === normalized);
   if (match) {
     const rel = relationshipIndex[match.kid];
     return {
       kid: match.kid,
       resolved_from: "kid",
       matched_input: input,
```

```
      current_path: match.path,
      legacy_path: rel?.legacy_path,
    };
  }
}

for (const [kid, bucket] of Object.entries(aliasIndex)) {
  if (bucket.aliases.includes(normalized)) {
    const match = knowledgeIndex.find((entry) => entry.kid === kid);
    if (!match) return null;
    const rel = relationshipIndex[kid];
    return {
      kid,
      resolved_from: "alias",
      matched_input: input,
      current_path: match.path,
      legacy_path: rel?.legacy_path,
    };
  }
}

for (const [kid, rel] of Object.entries(relationshipIndex)) {
  if (rel.legacy_path === normalized) {
    return {
      kid,
      resolved_from: "legacy_path",
      matched_input: input,
      current_path: rel.current_path,
      legacy_path: rel.legacy_path,
    };
  }
}

const inputBasename = path.basename(normalized, path.extname(normalized));
for (const [kid, rel] of Object.entries(relationshipIndex)) {
  const legacyBase = rel.legacy_path
    ? path.basename(rel.legacy_path, path.extname(rel.legacy_path))
    : null;

  if (legacyBase && legacyBase === inputBasename) {
    return {
      kid,
      resolved_from: "legacy_filename",
      matched_input: input,
```

```
      current_path: rel.current_path,
      legacy_path: rel.legacy_path,
    };
  }
}

 return null;
}
```

5. `loadItem.ts`

This loads the canonical item from `CONTENT/ITEMS`.

```
import fs from "node:fs";
import matter from "gray-matter";
import { CONTENT_ITEMS_DIR } from "./paths";
import { resolveKid } from "./resolveKid";
import type { KnowledgeFrontmatter, KnowledgeItem } from "./types";
import path from "node:path";

export function loadItem(input: string): KnowledgeItem | null {
 const resolved = resolveKid(input);
 if (!resolved) return null;

 const fullPath = path.isAbsolute(resolved.current_path)
   ? resolved.current_path
   : path.resolve(resolved.current_path);

 if (!fs.existsSync(fullPath)) return null;

 const raw = fs.readFileSync(fullPath, "utf8");
 const parsed = matter(raw);

 return {
   kid: resolved.kid,
   path: fullPath,
   body: parsed.content,
   frontmatter: parsed.data as KnowledgeFrontmatter,
 };
}

export function loadItemByKid(kid: string): KnowledgeItem | null {
```

```
 const filePath = path.join(CONTENT_ITEMS_DIR, `${kid}.md`);
 if (!fs.existsSync(filePath)) return null;

 const raw = fs.readFileSync(filePath, "utf8");
 const parsed = matter(raw);

 return {
   kid,
   path: filePath,
   body: parsed.content,
   frontmatter: parsed.data as KnowledgeFrontmatter,
 };
}
```

6. loadRelationships.ts

```
import { loadRelationshipIndex } from "./registry";
import type { RelationshipEntry } from "./types";

export function loadRelationships(kid: string): RelationshipEntry | null {
 const relIndex = loadRelationshipIndex();
 return relIndex[kid] ?? null;
}
```

7. loadCollections.ts

This matches collections by item metadata.

```
import fs from "node:fs";
import path from "node:path";
import matter from "gray-matter";
import { COLLECTIONS_DIR } from "./paths";
import type { CollectionDescriptor, KnowledgeItem } from "./types";

function loadAllCollections(): CollectionDescriptor[] {
 if (!fs.existsSync(COLLECTIONS_DIR)) return [];

 return fs
   .readdirSync(COLLECTIONS_DIR)
   .filter((file) => file.endsWith(".md"))
   .map((file) => {
```

```
    const fullPath = path.join(COLLECTIONS_DIR, file);
    const raw = fs.readFileSync(fullPath, "utf8");
    const parsed = matter(raw);
    return {
      ...(parsed.data as Omit<CollectionDescriptor, "body">),
      body: parsed.content,
    };
  });
}

export function loadCollectionsForItem(item: KnowledgeItem): CollectionDescriptor[] {
 const collections = loadAllCollections();
 const fm = item.frontmatter;

 return collections.filter((collection) => {
   const scope = collection.scope ?? {};

   const industryMatch =
     (scope.industry_refs ?? []).some((v) => fm.industry_refs.includes(v));

   const stackMatch =
     (scope.stack_family_refs ?? []).some((v) => fm.stack_family_refs.includes(v));

   const pillarMatch =
     (scope.pillar_refs ?? []).some((v) => fm.pillar_refs.includes(v));

   const domainMatch =
     scope.primary_domain != null && scope.primary_domain === fm.primary_domain;

   return industryMatch || stackMatch || pillarMatch || domainMatch;
 });
}
```

8. searchItems.ts

This should search metadata fields, not folders.

```
import { loadAliasIndex, loadKnowledgeIndex } from "./registry";
import type { KnowledgeIndexEntry, SearchFilters, SearchResult } from "./types";

function includesAll(haystack: string[] | undefined, needles: string[] | undefined): boolean {
 if (!needles || needles.length === 0) return true;
```

```
 if (!haystack) return false;
 return needles.every((needle) => haystack.includes(needle));
}

function scoreTextMatch(entry: KnowledgeIndexEntry, aliases: string[], text: string): { score:
number; fields: string[] } {
 const q = text.toLowerCase();
 let score = 0;
 const fields: string[] = [];

 if (entry.kid.toLowerCase().includes(q)) {
   score += 10;
   fields.push("kid");
 }
 if (entry.title.toLowerCase().includes(q)) {
   score += 8;
   fields.push("title");
 }
 if (entry.primary_domain.toLowerCase().includes(q)) {
   score += 6;
   fields.push("primary_domain");
 }
 if (entry.tags?.some((tag) => tag.toLowerCase().includes(q))) {
   score += 5;
   fields.push("tags");
 }
 if (entry.stack_family_refs.some((v) => v.toLowerCase().includes(q))) {
   score += 5;
   fields.push("stack_family_refs");
 }
 if (entry.industry_refs.some((v) => v.toLowerCase().includes(q))) {
   score += 5;
   fields.push("industry_refs");
 }
 if (entry.pillar_refs.some((v) => v.toLowerCase().includes(q))) {
   score += 4;
   fields.push("pillar_refs");
 }
 if (aliases.some((v) => v.toLowerCase().includes(q))) {
   score += 7;
   fields.push("aliases");
 }

 return { score, fields };
```

```
}

export function searchItems(filters: SearchFilters): SearchResult[] {
 const knowledgeIndex = loadKnowledgeIndex();
 const aliasIndex = loadAliasIndex();

 const results: SearchResult[] = [];

 for (const entry of knowledgeIndex) {
   if (filters.content_type && entry.content_type !== filters.content_type) continue;
   if (filters.primary_domain && entry.primary_domain !== filters.primary_domain) continue;
   if (filters.status && entry.status !== filters.status) continue;
   if (filters.authority_tier && entry.authority_tier !== filters.authority_tier) continue;
   if (!includesAll(entry.industry_refs, filters.industry_refs)) continue;
   if (!includesAll(entry.stack_family_refs, filters.stack_family_refs)) continue;
   if (!includesAll(entry.pillar_refs, filters.pillar_refs)) continue;
   if (!includesAll(entry.bundle_refs, filters.bundle_refs)) continue;
   if (!includesAll(entry.tags, filters.tags)) continue;

   const aliases = aliasIndex[entry.kid]?.aliases ?? [];
   let score = 1;
   let matched_fields: string[] = [];

   if (filters.text) {
     const scored = scoreTextMatch(entry, aliases, filters.text);
     if (scored.score === 0) continue;
     score += scored.score;
     matched_fields = scored.fields;
   }

   results.push({ item: entry, score, matched_fields });
 }

 results.sort((a, b) => b.score - a.score);

 return results.slice(0, filters.limit ?? 25);
}
```

9. `legacyFallback.ts`

This should be explicitly limited and temporary.

```
import path from "node:path";
import type { SearchFilters } from "./types";

export function inferFiltersFromLegacyPath(input: string): SearchFilters | null {
 const normalized = input.replace(/\\/g, "/").toLowerCase();

 if (!normalized.includes("/pillars/")) return null;

 const filters: SearchFilters = {};

 if (normalized.includes("/industry_playbooks/")) {
   filters.pillar_refs = ["industry_knowledge"];
 }
 if (normalized.includes("/it_end_to_end/")) {
   filters.pillar_refs = ["solution_patterns"];
 }
 if (normalized.includes("/languages_and_libraries/")) {
   filters.pillar_refs = ["technology_knowledge"];
 }

 const base = path.basename(normalized, path.extname(normalized));
 if (base && base !== "_meta") {
   filters.text = base;
 }

 return filters;
}
```

10. `index.ts`

This becomes the public resolver API.

```
import { loadCollectionsForItem } from "./loadCollections";
import { loadItem } from "./loadItem";
import { loadRelationships } from "./loadRelationships";
import { resolveKid } from "./resolveKid";
import { searchItems } from "./searchItems";

export function resolveKnowledge(input: string) {
 const resolved = resolveKid(input);
 if (!resolved) return null;
```

```
 const item = loadItem(resolved.kid);
 if (!item) return null;

 const relationships = loadRelationships(resolved.kid);
 const collections = loadCollectionsForItem(item);

 return {
   resolved,
   item,
   relationships,
   collections,
 };
}

export {
 resolveKid,
 loadItem,
 loadRelationships,
 loadCollectionsForItem,
 searchItems,
};
```

11. Implementation notes

Use `gray-matter`

That is the cleanest way to parse markdown frontmatter in TS.

Install:

npm install gray-matter

Keep `knowledge.index.json` lightweight

Do not duplicate full body content into the index.

The index should be search metadata only.

Do not make collections authoritative

Collections are contextual and navigational.
Item frontmatter should win if there is conflict.

12. The first practical rollout

Implement in this order:

Step 1

Add:

- `types.ts`
- `paths.ts`
- `registry.ts`

Step 2

Add:

- `resolveKid.ts`
- `loadItem.ts`
- `loadRelationships.ts`

At this point you have direct lookup working.

Step 3

Add:

- `searchItems.ts`

At this point metadata search works.

Step 4

Add:

- `loadCollections.ts`
- `legacyFallback.ts`

At this point migration compatibility is in place.

Step 5

Switch existing callers from old path-based resolver logic to `resolveKnowledge()`.

13. What your existing resolver should be replaced with

Your old resolver likely mixes:

- path parsing
- taxonomy inference
- bundle routing
- direct file assumptions

Replace that with this flow:

```
const result = resolveKnowledge(input);

if (!result) {
 // optionally try legacy fallback search here
}
```

That turns the resolver into a clean orchestration layer rather than a folder interpreter.

14. Definition of done for code

You are done with the first implementation pass when:

- `resolveKid("KID-...")` works
- `resolveKid("old-alias")` works
- `resolveKid("old/path/file.md")` works through registry redirect
- `searchItems({ text: "react" })` returns canonical items
- `loadItem()` reads from `CONTENT/ITEMS`
- no normal resolution path requires `PILLARS/`

JSON contracts

Here are the locked JSON contracts.

These should be treated as the minimum stable schemas the resolver reads from.

1. `knowledge.index.json`

Purpose:
 fast searchable registry of canonical items.

This is not the full knowledge content.
 It is the lookup/search layer.

Shape


```
[
 {
   "kid": "KID-SOL-AUTH-PATTERN-0012",
   "title": "OAuth Integration Pattern",
   "path": "Axion/libraries/knowledge/CONTENT/ITEMS/KID-SOL-AUTH-PATTERN-0012.md",
   "content_type": "pattern",
   "primary_domain": "authentication",
   "secondary_domains": ["identity"],
   "industry_refs": [],
   "stack_family_refs": [],
   "pillar_refs": ["solution_patterns"],
   "status": "active",
   "authority_tier": "reviewed",
   "freshness_class": "versioned_stable",
   "bundle_refs": ["FULLSTACK_SAAS"],
   "tags": ["oauth", "identity", "session-management"]
 }
]
```


Required fields

- `kid`
- `title`

- `path`
- `content_type`
- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `status`
- `authority_tier`

Optional fields

- `secondary_domains`
- `freshness_class`
- `bundle_refs`
- `tags`

Rules

- one entry per canonical KID
- `path` must point only to `CONTENT/ITEMS/...`
- no duplicate `kid`
- no body content stored here

- arrays should always be arrays, never null

2. `aliases.index.json`

Purpose:
map alternate names to canonical KIDs.

Recommended shape

Use alias-to-kid as the primary lookup shape because resolution is faster.

```
{
 "oauth-pattern": "KID-SOL-AUTH-PATTERN-0012",
 "auth-pattern-v2": "KID-SOL-AUTH-PATTERN-0012",
 "react-query-guide": "KID-TECH-REACT-REFERENCE-0041"
}
```

Optional richer companion shape

If you want auditability, keep a second reverse map or enrich the same file later, but for the resolver this flat map is better.

Example richer alternative:

```
{
 "_meta": {
   "version": "1.0.0"
 },
 "aliases": {
   "oauth-pattern": "KID-SOL-AUTH-PATTERN-0012",
   "auth-pattern-v2": "KID-SOL-AUTH-PATTERN-0012",
   "react-query-guide": "KID-TECH-REACT-REFERENCE-0041"
 }
}
```

If you do this richer shape, update the code accordingly.
My recommendation is the flat map first.

Rules

- keys must be normalized slugs
- values must be valid canonical KIDs
- one alias points to one KID only
- never use alias as a source of truth for metadata

3. `relationships.index.json`

Purpose:
hold non-primary structural relationships and backward-compatibility routing.

Shape


```
{
 "KID-SOL-AUTH-PATTERN-0012": {
   "legacy_path":
"PILLARS/IT_END_TO_END/03_backend_and_apis/authentication/oauth-pattern.md",
   "current_path":
"Axion/libraries/knowledge/CONTENT/ITEMS/KID-SOL-AUTH-PATTERN-0012.md",
   "depends_on": ["KID-SOL-IDENTITY-CONCEPT-0004"],
   "related_kids": ["KID-SOL-AUTH-CHECKLIST-0007"],
   "supersedes": [],
   "replaced_by": [],
   "collection_keys": ["domain.authentication", "pillar.solution_patterns"]
 }
}
```


Required fields per entry

- `current_path`

Optional fields

- `legacy_path`

- `depends_on`
- `related_kids`
- `supersedes`
- `replaced_by`
- `collection_keys`

Rules

- keyed by canonical `kid`
- `current_path` must point to canonical item location
- arrays should always be arrays, never null
- do not duplicate primary metadata here unless necessary

4. `bundles.index.json`

Purpose:
 define curated bundle membership without relying on path structure.

Shape


```
{
 "FULLSTACK_SAAS": {
   "title": "Fullstack SaaS",
   "description": "Core bundle for production SaaS application delivery.",
   "selection_mode": "explicit",
   "kid_refs": [
     "KID-SOL-AUTH-PATTERN-0012",
     "KID-SOL-API-STANDARD-0008",
     "KID-OPS-MONITORING-STANDARD-0006"
   ],
   "filters": {
```

```
    "pillar_refs": ["solution_patterns", "execution_and_operations"],
    "status": ["active", "reviewed"]
  }
 }
}
```

Bundle modes

Use one of:

- `explicit`
- `filter`
- `hybrid`

`explicit`

Bundle is directly defined by `kid_refs`.

`filter`

Bundle is generated by filters.

`hybrid`

Bundle has both fixed `kid_refs` and additional `filters`.

Rules

- bundle IDs are stable keys
- `kid_refs` contains canonical KIDs only
- filters use metadata fields, not paths

5. `deprecations.json`

Purpose:
track deprecated items and their replacements cleanly.

Shape

```
{
 "KID-SOL-AUTH-PATTERN-0005": {
   "status": "deprecated",
   "replaced_by": "KID-SOL-AUTH-PATTERN-0012",
   "reason": "Merged into canonical OAuth pattern rewrite.",
   "effective_date": "2026-03-11"
 }
}
```

Rules

- only deprecated KIDs belong here
- replacement should point to canonical active KID when available

6. `sources.index.json`

Purpose:
track provenance source references at registry level.

Shape

```
{
 "SRC-OAUTH-RFC": {
   "title": "OAuth 2.0 RFC",
   "type": "standard",
   "locator": "internal-or-external-reference",
   "status": "active"
 }
}
```

This can stay lightweight until you need deeper provenance.

7. `relationships.index.json` vs frontmatter

This split matters.

Keep in frontmatter

Use frontmatter for item-owned metadata:

- `kid`
- `title`
- `content_type`
- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `status`
- `authority_tier`
- `freshness_class`
- `bundle_refs`
- `tags`

Keep in relationships registry

Use registry for cross-item and migration data:

- `legacy_path`
- `current_path`
- `depends_on`
- `related_kids`
- `supersedes`
- `replaced_by`
- `collection_keys`

That keeps ownership clean.

8. Locked enum recommendations

These should be controlled through taxonomy files, but here is the minimum stable set.

`content_type`

- `concept`
- `pattern`
- `checklist`
- `workflow`
- `reference`
- `standard`
- `playbook`

- integration
- compliance
- data_model
- security_risk
- kpi_metric

status

- draft
- active
- deprecated
- archived

authority_tier

- draft
- reviewed
- verified
- golden

freshness_class

- volatile

- `periodic_review`
- `versioned_stable`
- `historical`

9. Best final form for resolver loading

Use these exact load expectations:

`knowledge.index.json`

Array of entries.

`aliases.index.json`

Flat object of:

```
alias -> kid
```

`relationships.index.json`

Object of:

```
kid -> relationship entry
```

That gives you the cleanest read patterns in TypeScript.

10. Updated TypeScript types to match the locked JSON

Your earlier alias type should be changed if you use the flat alias map.

```
export type AliasIndex = Record<string, string>;

export type RelationshipEntry = {
 legacy_path?: string;
 current_path: string;
```

```
 depends_on?: string[];
 related_kids?: string[];
 supersedes?: string[];
 replaced_by?: string[];
 collection_keys?: string[];
};

export type RelationshipIndex = Record<string, RelationshipEntry>;

export type KnowledgeIndexEntry = {
 kid: string;
 title: string;
 path: string;
 content_type: string;
 primary_domain: string;
 secondary_domains?: string[];
 industry_refs: string[];
 stack_family_refs: string[];
 pillar_refs: string[];
 status: string;
 authority_tier: string;
 freshness_class?: string;
 bundle_refs?: string[];
 tags?: string[];
};
```

11. Updated `resolveKid.ts` alias logic

If you use the flat alias map, replace the alias block with this:

```
const aliasedKid = aliasIndex[normalized];
if (aliasedKid) {
 const match = knowledgeIndex.find((entry) => entry.kid === aliasedKid);
 if (!match) return null;

 const rel = relationshipIndex[aliasedKid];
 return {
   kid: aliasedKid,
   resolved_from: "alias",
   matched_input: input,
   current_path: match.path,
   legacy_path: rel?.legacy_path,
```

```
 };
}
```

12. Updated `searchItems.ts` alias logic

If aliases are flat, you need reverse lookup per item:

```
const aliasesForKid = Object.entries(aliasIndex)
 .filter(([, kid]) => kid === entry.kid)
 .map(([alias]) => alias);
```

That works, though later you may want a precomputed reverse map for speed.

13. My recommendation

Lock these first:

- `knowledge.index.json` as array
- `aliases.index.json` as flat alias-to-kid map
- `relationships.index.json` as kid-to-relationship map

That is the cleanest minimal contract for the rewrite.

Then keep everything else secondary until the resolver is stable.

knowledge.index.json

Use this as the `knowledge.index.json` builder spec.

This is the missing piece that keeps the migration scripts, registry files, and resolver from drifting apart.

Purpose

The builder's job is to generate one reliable searchable registry from the canonical source:

- `CONTENT/ITEMS/*.md`

It should not treat old `PILLARS/` paths as truth.
At most, it can use them during migration as fallback metadata or for validation.

Source of truth hierarchy

The builder should use this order:

1. Canonical item frontmatter

Primary truth:

- `CONTENT/ITEMS/KID-*.md`

2. Relationship registry

Supplemental truth:

- `SYSTEM/registries/relationships.index.json`

Used for:

- `legacy_path`
- dependency relationships

- migration mapping
- collection references

3. Alias registry

Supplemental truth:

- `SYSTEM/registries/aliases.index.json`

Used for:

- validation only
- reverse lookup support if needed

4. Taxonomy

Validation truth:

- `SYSTEM/taxonomy/*.json`

Used to validate:

- allowed `content_type`
- allowed `status`
- allowed `authority_tier`
- allowed `pillar_refs`
- allowed domain/industry/stack refs

Builder output

The builder should generate:

Axion/libraries/knowledge/SYSTEM/registries/knowledge.index.json

Optional companion outputs:

Axion/libraries/knowledge/OUTPUTS/selections/knowledge_index_report.json
Axion/libraries/knowledge/OUTPUTS/selections/knowledge_index_errors.json
Axion/libraries/knowledge/OUTPUTS/selections/knowledge_index_warnings.json

What the builder scans

Scan only:

Axion/libraries/knowledge/CONTENT/ITEMS/*.md

Do not build the main index directly from:

- `PILLARS/`
- `VIEWS/`
- `BUNDLES/`
- `OUTPUTS/`
- `SYSTEM/templates/`

Those are not canonical items.

Builder inclusion rule

Include a file in `knowledge.index.json` only if:

- it is a markdown file in `CONTENT/ITEMS/`
- it has valid frontmatter

- it has a valid `kid`
- it passes minimum required metadata validation

Otherwise:

- exclude it from the final index
- write it to the error report

Required frontmatter fields

Each item must provide:

- `kid`
- `title`
- `content_type`
- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `status`
- `authority_tier`

Optional but recommended:

- `secondary_domains`

- `freshness_class`
- `bundle_refs`
- `tags`
- `legacy_path`

Output contract per entry

Each output entry should be exactly this shape:


```
{
 "kid": "KID-SOL-AUTH-PATTERN-0012",
 "title": "OAuth Integration Pattern",
 "path": "Axion/libraries/knowledge/CONTENT/ITEMS/KID-SOL-AUTH-PATTERN-0012.md",
 "content_type": "pattern",
 "primary_domain": "authentication",
 "secondary_domains": ["identity"],
 "industry_refs": [],
 "stack_family_refs": [],
 "pillar_refs": ["solution_patterns"],
 "status": "active",
 "authority_tier": "reviewed",
 "freshness_class": "versioned_stable",
 "bundle_refs": ["FULLSTACK_SAAS"],
 "tags": ["oauth", "identity", "session-management"]
}
```


Builder normalization rules

Rule 1 — path

Always write canonical path:

Axion/libraries/knowledge/CONTENT/ITEMS/{kid}.md

Do not preserve source scan path if the filename differs.

Rule 2 — arrays

Force these fields to arrays:

- `secondary_domains`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `bundle_refs`
- `tags`

Never allow:

- `null`
- scalar strings in place of arrays

Rule 3 — trim strings

Trim all string fields.

Rule 4 — empty defaults

If optional arrays are missing, write `[]`.

Rule 5 — deterministic ordering

Sort final index by:

1. `kid`

This makes diffs stable.

Validation rules

Hard-fail errors

Exclude item from final index if any of these occur:

- missing `kid`
- duplicate `kid`
- invalid frontmatter parse
- missing required field
- `kid` does not match filename when filename is expected canonical
- invalid enum value for `content_type`
- invalid enum value for `status`
- invalid enum value for `authority_tier`

Soft warnings

Include item, but write warning if:

- `primary_domain` is `unknown`
- `title` looks placeholder-like
- `tags` is empty
- `bundle_refs` is empty
- no corresponding relationships entry exists
- no alias exists for older known filename

- `legacy_path` is missing during migration phase

Duplicate handling

If two files declare the same `kid`:

- neither should silently win
- both go to `knowledge_index_errors.json`
- builder exits non-zero in strict mode

Taxonomy validation

The builder should validate values against taxonomy vocab files.

At minimum validate:

- `content_type`
- `status`
- `authority_tier`
- `pillar_refs`

Later also validate:

- `primary_domain`
- `industry_refs`
- `stack_family_refs`

Suggested script location

Axion/scripts/knowledge/build_knowledge_index.py

Script modes

Use 3 modes.

`--check`

Validation only.

- scan items
- report errors/warnings
- do not write index

`--write`

Generate and write:

- `knowledge.index.json`
- reports

`--strict`

Fails on warnings as well as errors.

Recommended CLI examples

python Axion/scripts/knowledge/build_knowledge_index.py --check
python Axion/scripts/knowledge/build_knowledge_index.py --write
python Axion/scripts/knowledge/build_knowledge_index.py --write --strict

Builder report shapes

`knowledge_index_report.json`


```
{
 "scanned_files": 395,
 "indexed_items": 389,
 "error_count": 6,
 "warning_count": 24
}
```


`knowledge_index_errors.json`


```
[
 {
   "path": "Axion/libraries/knowledge/CONTENT/ITEMS/KID-BAD-0001.md",
   "kid": "KID-BAD-0001",
   "errors": ["missing_required_field:title", "invalid_status:live"]
 }
]
```


`knowledge_index_warnings.json`


```
[
 {
   "path": "Axion/libraries/knowledge/CONTENT/ITEMS/KID-SOL-FOO-0003.md",
   "kid": "KID-SOL-FOO-0003",
   "warnings": ["primary_domain_unknown", "empty_tags"]
 }
]
```


Python implementation outline

```
from __future__ import annotations

import argparse
import json
from dataclasses import dataclass, asdict
from pathlib import Path
from typing import Any
```

```
import yaml


ROOT = Path("Axion/libraries/knowledge")
ITEMS_DIR = ROOT / "CONTENT" / "ITEMS"
REGISTRIES_DIR = ROOT / "SYSTEM" / "registries"
TAXONOMY_DIR = ROOT / "SYSTEM" / "taxonomy"
OUTPUT_DIR = ROOT / "OUTPUTS" / "selections"

INDEX_PATH = REGISTRIES_DIR / "knowledge.index.json"
REPORT_PATH = OUTPUT_DIR / "knowledge_index_report.json"
ERRORS_PATH = OUTPUT_DIR / "knowledge_index_errors.json"
WARNINGS_PATH = OUTPUT_DIR / "knowledge_index_warnings.json"


REQUIRED_FIELDS = [
  "kid",
  "title",
  "content_type",
  "primary_domain",
  "industry_refs",
  "stack_family_refs",
  "pillar_refs",
  "status",
  "authority_tier",
]


@dataclass
class ValidationIssue:
  path: str
  kid: str | None
  errors: list[str]
  warnings: list[str]


def read_text(path: Path) -> str:
  return path.read_text(encoding="utf-8")


def load_json(path: Path, fallback: Any) -> Any:
  if not path.exists():
      return fallback
  return json.loads(path.read_text(encoding="utf-8"))
```

```
def save_json(path: Path, data: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, indent=2), encoding="utf-8")


def parse_frontmatter(text: str) -> tuple[dict[str, Any], str]:
    if not text.startswith("---\n"):
        return {}, text
    parts = text.split("---\n", 2)
    if len(parts) < 3:
        return {}, text
    try:
        meta = yaml.safe_load(parts[1]) or {}
        if not isinstance(meta, dict):
            return {}, text
        return meta, parts[2]
    except Exception:
        return {}, text


def ensure_list(meta: dict[str, Any], key: str) -> None:
    value = meta.get(key, [])
    if value is None:
        meta[key] = []
    elif isinstance(value, list):
        meta[key] = value
    else:
        meta[key] = [value]


def normalize_entry(meta: dict[str, Any], path: Path) -> dict[str, Any]:
    for key in [
        "secondary_domains",
        "industry_refs",
        "stack_family_refs",
        "pillar_refs",
        "bundle_refs",
        "tags",
    ]:
        ensure_list(meta, key)

    entry = {
```

```
        "kid": str(meta["kid"]).strip(),
        "title": str(meta["title"]).strip(),
        "path": str(ITEMS_DIR / f"{str(meta['kid']).strip()}.md"),
        "content_type": str(meta["content_type"]).strip(),
        "primary_domain": str(meta["primary_domain"]).strip(),
        "secondary_domains": meta.get("secondary_domains", []),
        "industry_refs": meta.get("industry_refs", []),
        "stack_family_refs": meta.get("stack_family_refs", []),
        "pillar_refs": meta.get("pillar_refs", []),
        "status": str(meta["status"]).strip(),
        "authority_tier": str(meta["authority_tier"]).strip(),
        "freshness_class": str(meta["freshness_class"]).strip() if meta.get("freshness_class") else
None,
        "bundle_refs": meta.get("bundle_refs", []),
        "tags": meta.get("tags", []),
    }

    if not entry["secondary_domains"]:
        entry.pop("secondary_domains")
    if not entry["bundle_refs"]:
        entry.pop("bundle_refs")
    if not entry["tags"]:
        entry.pop("tags")
    if not entry["freshness_class"]:
        entry.pop("freshness_class")

    return entry


def load_taxonomy_values() -> dict[str, set[str]]:
    taxonomy = load_json(TAXONOMY_DIR / "taxonomy.json", {})
    return {
        "content_type": set(taxonomy.get("content_types", [])),
        "status": set(taxonomy.get("status", [])),
        "authority_tier": set(taxonomy.get("authority_tiers", [])),
        "pillar_refs": set(taxonomy.get("pillars", [])),
    }


def validate_meta(meta: dict[str, Any], path: Path, taxonomy_values: dict[str, set[str]]) ->
ValidationIssue:
    errors: list[str] = []
    warnings: list[str] = []
```

```
    kid = meta.get("kid")

    for field in REQUIRED_FIELDS:
        if field not in meta:
            errors.append(f"missing_required_field:{field}")

    if kid and path.name != f"{kid}.md":
        errors.append("filename_kid_mismatch")

    if "content_type" in meta and taxonomy_values["content_type"] and meta["content_type"] not
in taxonomy_values["content_type"]:
        errors.append(f"invalid_content_type:{meta['content_type']}")

    if "status" in meta and taxonomy_values["status"] and meta["status"] not in
taxonomy_values["status"]:
        errors.append(f"invalid_status:{meta['status']}")

    if "authority_tier" in meta and taxonomy_values["authority_tier"] and meta["authority_tier"] not
in taxonomy_values["authority_tier"]:
        errors.append(f"invalid_authority_tier:{meta['authority_tier']}")

    for pillar in meta.get("pillar_refs", []) or []:
        if taxonomy_values["pillar_refs"] and pillar not in taxonomy_values["pillar_refs"]:
            errors.append(f"invalid_pillar_ref:{pillar}")

    if meta.get("primary_domain") == "unknown":
        warnings.append("primary_domain_unknown")

    if not meta.get("tags"):
        warnings.append("empty_tags")

    if not meta.get("bundle_refs"):
        warnings.append("empty_bundle_refs")

    title = str(meta.get("title", "")).strip().lower()
    if title in {"untitled", "replace me", "example title"}:
        warnings.append("placeholder_title")

    return ValidationIssue(
        path=str(path),
        kid=str(kid) if kid else None,
        errors=errors,
        warnings=warnings,
    )
```

```
def build_index(strict: bool, write: bool) -> int:
    taxonomy_values = load_taxonomy_values()
    issues: list[ValidationIssue] = []
    entries: list[dict[str, Any]] = []

    seen_kids: dict[str, str] = {}

    for path in sorted(ITEMS_DIR.glob("*.md")):
        text = read_text(path)
        meta, _ = parse_frontmatter(text)

        if not meta:
            issues.append(ValidationIssue(str(path), None, ["invalid_or_missing_frontmatter"], []))
            continue

        issue = validate_meta(meta, path, taxonomy_values)

        kid = meta.get("kid")
        if kid:
            if kid in seen_kids:
                issue.errors.append(f"duplicate_kid_with:{seen_kids[kid]}")
            else:
                seen_kids[kid] = str(path)

        issues.append(issue)

        if issue.errors:
            continue

        entries.append(normalize_entry(meta, path))

    entries.sort(key=lambda x: x["kid"])

    errors = [asdict(i) for i in issues if i.errors]
    warnings = [asdict(i) for i in issues if i.warnings]

    report = {
        "scanned_files": len(list(ITEMS_DIR.glob("*.md"))),
        "indexed_items": len(entries),
        "error_count": len(errors),
        "warning_count": len(warnings),
    }
```

```
    if write:
        save_json(INDEX_PATH, entries)
        save_json(REPORT_PATH, report)
        save_json(ERRORS_PATH, errors)
        save_json(WARNINGS_PATH, warnings)

    has_errors = len(errors) > 0
    has_warnings = len(warnings) > 0

    if has_errors:
        return 1
    if strict and has_warnings:
        return 2
    return 0


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--check", action="store_true")
    parser.add_argument("--write", action="store_true")
    parser.add_argument("--strict", action="store_true")
    args = parser.parse_args()

    exit_code = build_index(strict=args.strict, write=args.write)

    if args.check and not args.write:
        pass

    raise SystemExit(exit_code)


if __name__ == "__main__":
    main()
```

Recommended follow-on rule

After this builder exists, stop hand-editing `knowledge.index.json`.

Make it generated only.

That removes one of the biggest sources of drift.

Operational workflow

Use this workflow every time canonical items change:

1. edit `CONTENT/ITEMS/*.md`
2. run migration/cleanup scripts if needed
3. run `build_knowledge_index.py --write`
4. fix any reported errors
5. then run resolver/search

That makes the index a derived artifact, not another manual database.

Definition of done for the builder

You are done when:

- every canonical item can be indexed from `CONTENT/ITEMS`
- `knowledge.index.json` is fully generated
- invalid items are excluded with explicit error output
- warnings are visible but non-blocking unless strict mode is used
- resolver reads only the generated index, not ad hoc scans

companion builder spec

Use this as the companion builder spec for `aliases.index.json` and `relationships.index.json`.

Once these three registries are generated the same way—

- `knowledge.index.json`
- `aliases.index.json`
- `relationships.index.json`

—you stop having three partially manual systems drifting apart.

Goal

Make both registries derived from canonical content plus controlled migration inputs.

That means:

- item metadata stays in `CONTENT/ITEMS/*.md`
- alias and relationship registries are generated artifacts
- manual edits happen only in clearly designated override files when needed

Source of truth hierarchy

For `aliases.index.json`

Primary inputs:

1. `CONTENT/ITEMS/*.md`
2. canonical filename

3. `legacy_path` in frontmatter
4. optional manual alias overrides

For `relationships.index.json`

Primary inputs:

1. `CONTENT/ITEMS/*.md`
2. frontmatter relationship fields
3. `legacy_path` in frontmatter
4. collection matching rules
5. optional manual relationship overrides

Recommended generated vs manual split

Do not force every relationship or alias to be manually edited in the final generated registry.

Use this pattern:

Generated outputs

- `SYSTEM/registries/aliases.index.json`
- `SYSTEM/registries/relationships.index.json`

Optional manual inputs

- `SYSTEM/registries/aliases.manual.json`
- `SYSTEM/registries/relationships.manual.json`

The builder merges canonical/generated data with manual overrides.

That gives you flexibility without turning the generated files into hand-maintained clutter.

1. `aliases.index.json` builder spec

Purpose

Map every useful alternate lookup token to a canonical KID.

Final output shape

Use the flat shape:

```
{
 "oauth-pattern": "KID-SOL-AUTH-PATTERN-0012",
 "auth-pattern-v2": "KID-SOL-AUTH-PATTERN-0012",
 "react-query-guide": "KID-TECH-REACT-REFERENCE-0041"
}
```

Alias sources

The builder should generate aliases from these sources.

Source A — canonical filename stem

For:
`CONTENT/ITEMS/KID-SOL-AUTH-PATTERN-0012.md`

Do not alias the exact KID to itself here.
That is direct resolution, not aliasing.

Source B — frontmatter title slug

Example:
`OAuth Integration Pattern` → `oauth-integration-pattern`

Source C — legacy filename stem

From:
`legacy_path: PILLARS/.../oauth-pattern.md`

Generate:

- `oauth-pattern`

Source D — optional explicit alias field

If you allow it in frontmatter:

```
aliases:
 - oauth-pattern
 - auth-pattern-v2
```

Source E — manual alias overrides

From:
`aliases.manual.json`

Example:

```
{
 "oauth2-pattern": "KID-SOL-AUTH-PATTERN-0012"
}
```

Alias normalization rules

Every alias key should be normalized to a slug:

- lowercase
- trim whitespace
- replace spaces/underscores with hyphens
- strip file extensions
- collapse repeated hyphens

Examples:

- `OAuth Pattern` → `oauth-pattern`

- `oauth_pattern.md` → `oauth-pattern`
- `React Query Guide` → `react-query-guide`

Alias collision rules

If the same alias resolves to multiple KIDs:

- do not pick one automatically
- exclude that alias from final output
- write it to an alias collision report
- fail in strict mode

Example bad case:

- `auth-pattern` → `KID-A`
- `auth-pattern` → `KID-B`

Builder outputs

Generate:

Axion/libraries/knowledge/SYSTEM/registries/aliases.index.json
Axion/libraries/knowledge/OUTPUTS/selections/aliases_report.json
Axion/libraries/knowledge/OUTPUTS/selections/aliases_collisions.json

`aliases_report.json`


```
{
 "items_scanned": 389,
 "aliases_generated": 612,
 "collisions": 4
}
```

`aliases_collisions.json`


```
[
 {
   "alias": "auth-pattern",
   "kids": [
     "KID-SOL-AUTH-PATTERN-0012",
     "KID-SOL-AUTH-PATTERN-0005"
   ]
 }
]
```


2. `relationships.index.json` builder spec

Purpose

Generate the cross-item and migration registry.

Final output shape


```
{
 "KID-SOL-AUTH-PATTERN-0012": {
   "legacy_path":
"PILLARS/IT_END_TO_END/03_backend_and_apis/authentication/oauth-pattern.md",
   "current_path":
"Axion/libraries/knowledge/CONTENT/ITEMS/KID-SOL-AUTH-PATTERN-0012.md",
   "depends_on": ["KID-SOL-IDENTITY-CONCEPT-0004"],
   "related_kids": ["KID-SOL-AUTH-CHECKLIST-0007"],
   "supersedes": [],
   "replaced_by": [],
   "collection_keys": ["domain.authentication", "pillar.solution_patterns"]
 }
}
```


Relationship sources

Source A — frontmatter on canonical item

Read directly from each item:

- `depends_on`
- `related_kids`
- `supersedes`
- `legacy_path`

Source B — canonical path

Always generate:

- `current_path`

Source C — inferred collection keys

Build from metadata:

- `primary_domain` → `domain.{value}`
- each `industry_refs` → `industry.{value}`
- each `stack_family_refs` → `stack.{value}`
- each `pillar_refs` → `pillar.{value}`

Source D — manual overrides

From:
`relationships.manual.json`

Use for:

- forced redirects
- non-frontmatter relationships
- temporary migration patches

Relationship generation rules

current_path

Always write:

Axion/libraries/knowledge/CONTENT/ITEMS/{kid}.md

legacy_path

Copy from frontmatter if present.

depends_on

Take from frontmatter, normalize to array, de-duplicate.

related_kids

Take from frontmatter, normalize to array, de-duplicate.

supersedes

Take from frontmatter, normalize to array.

replaced_by

Usually empty unless manual override or deprecation logic provides it.

collection_keys

Always generate from item metadata.

Example:

primary_domain: authentication
industry_refs: []
stack_family_refs: [react]
pillar_refs: [solution_patterns, technology_knowledge]

becomes:

```
"collection_keys": [
 "domain.authentication",
 "stack.react",
 "pillar.solution_patterns",
 "pillar.technology_knowledge"
]
```

Sort these for stable diffs.

Relationship validation rules

Hard errors:

- KID missing from canonical content
- `current_path` target missing
- referenced `depends_on` KID does not exist
- referenced `related_kids` KID does not exist
- referenced `supersedes` KID does not exist

Warnings:

- missing `legacy_path` during migration phase
- empty relationship arrays
- item has no collection keys
- `supersedes` exists but replaced item not marked deprecated elsewhere

Builder outputs

Generate:

Axion/libraries/knowledge/SYSTEM/registries/relationships.index.json
Axion/libraries/knowledge/OUTPUTS/selections/relationships_report.json
Axion/libraries/knowledge/OUTPUTS/selections/relationships_errors.json
Axion/libraries/knowledge/OUTPUTS/selections/relationships_warnings.json

3. Recommended frontmatter additions

To make these builders work cleanly, allow these optional frontmatter fields in canonical items:

aliases: []
depends_on: []
related_kids: []
supersedes: []
legacy_path: PILLARS/...

That is enough for the initial builders.

4. Builder execution order

Run builders in this order:

Step 1

`build_knowledge_index.py`

This validates canonical items first.

Step 2

`build_aliases_index.py`

Uses canonical items already validated.

Step 3

`build_relationships_index.py`

Uses canonical items and optionally validated index output.

That sequence avoids garbage-in/garbage-out.

5. `build_aliases_index.py` outline

Recommended location:

Axion/scripts/knowledge/build_aliases_index.py

Core behavior:

- scan `CONTENT/ITEMS/*.md`
- parse frontmatter
- collect alias candidates
- normalize candidates
- detect collisions
- merge manual overrides
- write `aliases.index.json`

Pseudo-shape:

```
for each canonical item:
  read kid
  collect aliases from:
    - title slug
    - legacy filename stem
    - frontmatter aliases
  normalize each alias
  skip alias if alias == kid.lower()
  add alias -> kid candidate

merge aliases.manual.json

detect collisions
remove collided aliases from final map
write outputs
```

6. `build_relationships_index.py` outline

Recommended location:

Axion/scripts/knowledge/build_relationships_index.py

Core behavior:

- scan `CONTENT/ITEMS/*.md`
- parse frontmatter
- build relationship entry per KID
- compute `current_path`
- compute `collection_keys`
- validate referenced KIDs
- merge manual overrides
- write `relationships.index.json`

Pseudo-shape:

```
for each canonical item:
  read frontmatter
  kid = meta["kid"]

  entry = {
    "current_path": canonical_path,
    "legacy_path": meta.get("legacy_path"),
    "depends_on": normalized array,
    "related_kids": normalized array,
    "supersedes": normalized array,
    "replaced_by": [],
    "collection_keys": generated collection keys
  }
```

merge relationships.manual.json
validate all referenced KIDs exist
write outputs

7. Suggested manual override shapes

`aliases.manual.json`

```
{
 "oauth2-pattern": "KID-SOL-AUTH-PATTERN-0012",
 "oauth-flow-guide": "KID-SOL-AUTH-PATTERN-0012"
}
```

`relationships.manual.json`

```
{
 "KID-SOL-AUTH-PATTERN-0012": {
   "replaced_by": [],
   "collection_keys": [
     "domain.authentication",
     "pillar.solution_patterns"
   ]
 }
}
```

Keep manual overrides narrow.
Do not duplicate the whole generated entry unless necessary.

8. Stable generation rules

Apply these to all three builders:

- sort by KID or alias key
- normalize all arrays
- de-duplicate arrays
- never write null arrays

- prefer explicit metadata over inferred values
- generated output is not hand-edited

9. Definition of done

You are done when:

- `knowledge.index.json` is generated from canonical items
- `aliases.index.json` is generated from canonical items plus overrides
- `relationships.index.json` is generated from canonical items plus overrides
- resolver reads only these generated registries
- no normal resolution path depends on `PILLARS/`

registry build pipeline

Use this as the registry build pipeline outline.

This is the piece that turns all the separate scripts into one repeatable maintenance workflow for the knowledge library.

Goal

One command should be able to:

1. validate canonical items
2. build `knowledge.index.json`
3. build `aliases.index.json`
4. build `relationships.index.json`
5. write summary reports
6. fail clearly when the library is broken

That gives you a proper maintenance loop instead of a bunch of disconnected scripts.

Recommended pipeline entrypoint

Put the orchestrator here:

Axion/scripts/knowledge/build_registries.py

What the pipeline owns

It should orchestrate these steps in order:

Step 1 — canonical item validation

Check all files in:

Axion/libraries/knowledge/CONTENT/ITEMS/

Validate:

- frontmatter exists
- required fields exist
- enums are valid
- filenames match KIDs
- duplicate KIDs do not exist

This is the gatekeeper step.

Step 2 — knowledge index build

Generate:

Axion/libraries/knowledge/SYSTEM/registries/knowledge.index.json

This is the searchable canonical registry.

Step 3 — alias index build

Generate:

Axion/libraries/knowledge/SYSTEM/registries/aliases.index.json

This is the alternate lookup registry.

Step 4 — relationships index build

Generate:

Axion/libraries/knowledge/SYSTEM/registries/relationships.index.json

This is the dependency, redirect, and collection registry.

Step 5 — summary and health reporting

Write a final summary report that says:

- how many items scanned
- how many indexed
- how many aliases generated
- how many relationship entries generated
- how many errors
- how many warnings
- whether the build passed

Recommended outputs

Keep all reports here:

Axion/libraries/knowledge/OUTPUTS/selections/

Generate at minimum:

knowledge_index_report.json
knowledge_index_errors.json
knowledge_index_warnings.json

aliases_report.json
aliases_collisions.json

relationships_report.json
relationships_errors.json
relationships_warnings.json

registry_build_summary.json

Pipeline modes

Use 3 modes.

`--check`

Validation and dry analysis only.
No registry files are written.

Use this when auditing the library.

`--write`

Runs the full pipeline and writes generated registries and reports.

`--strict`

Treat warnings as failures.

Good for CI or release-quality checks.

Recommended CLI

```
python Axion/scripts/knowledge/build_registries.py --check
python Axion/scripts/knowledge/build_registries.py --write
python Axion/scripts/knowledge/build_registries.py --write --strict
```

Pipeline success model

Pass

- no hard errors
- warnings allowed unless `--strict`

Fail

- validation errors
- duplicate KIDs

- alias collisions in strict mode or if you choose to hard-fail them
- broken relationship references
- missing canonical files

Recommended build order

This order matters.

1. validate canonical items

Do not build anything if canonical items are fundamentally broken.

2. build knowledge index

This is the base registry all later steps assume is trustworthy.

3. build alias index

Alias generation depends on canonical item identity being stable.

4. build relationships index

Relationship generation depends on canonical KIDs being known and valid.

5. aggregate summary

Collect all counts, warnings, and failures into one final report.

Orchestrator responsibilities

The pipeline script should not duplicate all the builder logic itself.

It should call the dedicated builders.

Recommended scripts:

Axion/scripts/knowledge/build_knowledge_index.py
Axion/scripts/knowledge/build_aliases_index.py
Axion/scripts/knowledge/build_relationships_index.py
Axion/scripts/knowledge/build_registries.py

So `build_registries.py` becomes the conductor, not the whole orchestra.

Recommended summary contract

`registry_build_summary.json`


```
{
 "status": "pass",
 "strict_mode": false,
 "steps": {
   "validate_canonical_items": {
     "status": "pass",
     "scanned_files": 389,
     "errors": 0,
     "warnings": 12
   },
   "build_knowledge_index": {
     "status": "pass",
     "indexed_items": 389,
     "errors": 0,
     "warnings": 12
   },
   "build_aliases_index": {
     "status": "pass",
     "aliases_generated": 611,
     "collisions": 2
   },
   "build_relationships_index": {
     "status": "pass",
     "relationship_entries": 389,
     "errors": 0,
     "warnings": 9
   }
 },
 "final_counts": {
   "canonical_items": 389,
   "knowledge_index_entries": 389,
```

"alias_entries": 609,
"relationship_entries": 389
}
}

Recommended pass/fail semantics by step

Canonical validation

Hard fail on:

- missing required fields
- invalid frontmatter
- duplicate KIDs
- bad filename/KID mismatch
- invalid enums

Knowledge index

Hard fail on:

- invalid canonical item set
- duplicate index entry
- impossible canonical path

Alias index

Warn or fail on:

- alias collisions

- malformed manual override
- alias pointing to missing KID

My recommendation:

- collision = warning in normal mode
- collision = fail in strict mode

Relationships index

Hard fail on:

- missing referenced KID in `depends_on`
- missing referenced KID in `related_kids`
- missing canonical target path

Warn on:

- missing `legacy_path`
- empty relationship sets
- empty collection keys

Python orchestrator outline

```
from __future__ import annotations

import argparse
import json
from pathlib import Path

from build_knowledge_index import build_index
from build_aliases_index import build_aliases
```

```
from build_relationships_index import build_relationships


ROOT = Path("Axion/libraries/knowledge")
OUTPUT_DIR = ROOT / "OUTPUTS" / "selections"
SUMMARY_PATH = OUTPUT_DIR / "registry_build_summary.json"


def save_json(path: Path, data) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, indent=2), encoding="utf-8")


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--check", action="store_true")
    parser.add_argument("--write", action="store_true")
    parser.add_argument("--strict", action="store_true")
    args = parser.parse_args()

    write = args.write
    strict = args.strict

    summary = {
        "status": "pass",
        "strict_mode": strict,
        "steps": {},
        "final_counts": {},
    }

    knowledge_result = build_index(strict=strict, write=write, return_summary=True)
    summary["steps"]["build_knowledge_index"] = knowledge_result

    if knowledge_result["status"] == "fail":
        summary["status"] = "fail"
        save_json(SUMMARY_PATH, summary)
        raise SystemExit(1)

    alias_result = build_aliases(strict=strict, write=write, return_summary=True)
    summary["steps"]["build_aliases_index"] = alias_result

    if alias_result["status"] == "fail":
        summary["status"] = "fail"
        save_json(SUMMARY_PATH, summary)
```

```
        raise SystemExit(1)

    relationship_result = build_relationships(strict=strict, write=write, return_summary=True)
    summary["steps"]["build_relationships_index"] = relationship_result

    if relationship_result["status"] == "fail":
        summary["status"] = "fail"
        save_json(SUMMARY_PATH, summary)
        raise SystemExit(1)

    summary["final_counts"] = {
        "canonical_items": knowledge_result.get("indexed_items", 0),
        "knowledge_index_entries": knowledge_result.get("indexed_items", 0),
        "alias_entries": alias_result.get("aliases_generated", 0),
        "relationship_entries": relationship_result.get("relationship_entries", 0),
    }

    save_json(SUMMARY_PATH, summary)
    raise SystemExit(0)


if __name__ == "__main__":
    main()
```

Better function contract for each builder

Each builder should return a structured summary object like:

```
{
 "status": "pass",
 "indexed_items": 389,
 "errors": 0,
 "warnings": 12
}
```

or

```
{
 "status": "fail",
 "errors": 3,
 "warnings": 7
}
```

That makes orchestration much cleaner than parsing printed output.

Maintenance workflow

This should become your standard operating loop.

Normal update loop

1. add or edit canonical items in `CONTENT/ITEMS`
2. run migration scripts if needed
3. run `build_registries.py --write`
4. inspect reports
5. fix errors
6. rerun
7. then use resolver/search/UI

CI/release loop

Use:

python Axion/scripts/knowledge/build_registries.py --write --strict

That gives you a gate for bad library updates.

What this solves

This pipeline removes the biggest mess-causing behaviors:

- hand-editing registries
- index drift

- alias drift
- relationship drift
- partial updates that only fix one layer
- silent breakage after reorganization

Definition of done

You have a stable maintenance system when:

- canonical items are the only real content source
- registries are always generated
- builders are repeatable
- failures are explicit
- resolver reads only generated registries plus canonical items
- the old `PILLARS/` tree is no longer part of the normal build path

knowledge library governance ruleset

Use this as the knowledge library governance ruleset.

This is the layer that keeps the new structure from collapsing back into a mess after you clean it up.

Purpose

The governance ruleset defines:

- what can be added
- where it can be added
- what metadata is required
- when taxonomy is allowed to change
- when aliases are allowed
- when relationships are required
- how items are deprecated
- what counts as a valid update

Without this, the system will drift again even if the folder structure is better.

Core governance principle

The library must be governed by this order:

1. canonical item content
2. controlled taxonomy
3. generated registries
4. generated views

5. legacy compatibility only as fallback

That means no one should be “fixing” the library by editing views, path trees, or generated registries directly.

Governance zones

Treat the library as 5 governed zones.

Zone 1 — Canonical content

CONTENT/ITEMS/

Contains:

- all actual KIDs

This is where reusable knowledge lives.

Zone 2 — System control

SYSTEM/

Contains:

- doctrine
- taxonomy
- registries
- contracts
- policies
- templates

This is where the library’s operating rules live.

Zone 3 — Views

VIEWS/

Contains:

- generated browse structures

These are not canonical.

Zone 4 — Bundles

BUNDLES/

Contains:

- curated selections
- run-profile packs
- executor packs

These must reference KIDs or metadata only.

Zone 5 — Legacy compatibility

PILLARS/

Temporary migration zone only.

No new canonical content should be created here.

Rule set

1. Rules for adding a new KID

A new knowledge item may be added only if all of these are true:

- it is a reusable knowledge unit
- it belongs in canonical content, not templates or views

- it has a stable `kid`
- it has valid frontmatter
- it fits an existing `content_type` or an approved new one
- it fits an existing classification model
- it passes registry build validation

Required frontmatter

Every new KID must include:

- `kid`
- `title`
- `content_type`
- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `status`
- `authority_tier`

Recommended:

- `bundle_refs`
- `tags`

- `depends_on`
- `related_kids`
- `legacy_path` if migrated
- `aliases` if needed

New KID acceptance rule

A KID is not valid until:

- it exists in `CONTENT/ITEMS/`
- it passes builder validation
- it appears in `knowledge.index.json`

2. Rules for editing an existing KID

An existing KID may be edited when:

- its content improves clarity, correctness, or completeness
- its metadata becomes more accurate
- its taxonomy classification becomes more explicit
- its relationships become more accurate

But editing a KID must not silently change:

- the `kid`
- the canonical semantic identity
- the deprecation state without explicit handling

- the current path model outside `CONTENT/ITEMS`

Breaking-change rule

If the semantic identity of a KID changes substantially, do not mutate it in place.

Instead:

- create a new KID
- link with `supersedes` or `replaced_by`
- deprecate the old KID if needed

3. Rules for taxonomy changes

Taxonomy is controlled and must not be changed casually.

A taxonomy value may be added only when:

- existing values cannot express the concept
- the new value will apply to more than one item or one temporary case
- the new value is semantically distinct and not just a synonym
- it does not duplicate an existing value under another name

Allowed taxonomy changes

- add new `content_type`
- add new `pillar`
- add new domain
- add new industry

- add new stack family
- add new status/authority enum only with caution

Taxonomy review questions

Before adding a taxonomy value, answer:

- is this truly a new category or just a tag?
- is this stable enough to deserve controlled vocabulary status?
- will more than one KID use it?
- does it belong as taxonomy, tag, or bundle instead?

Taxonomy anti-drift rule

Do not add taxonomy values just to avoid thinking about classification.

If a concept is too narrow, use:

- `tags`
- bundle membership
- collection descriptors

not a new top-level taxonomy value.

4. Rules for aliases

Aliases are allowed only to support retrieval, migration, or common naming.

A new alias is justified when:

- the old filename or slug is still referenced

- users commonly search a non-canonical term
- a legacy path or legacy title needs backward compatibility
- the canonical title is not the most likely search term

Aliases must not be used to create parallel identities for the same KID.

Alias rules

- alias must normalize to a unique target KID
- alias cannot collide with another canonical lookup term
- alias is for lookup only, not metadata
- alias should be short, useful, and stable

Alias anti-sprawl rule

Do not generate dozens of vanity aliases.

Only keep aliases that are:

- historically important
- practically useful
- common search entry points

5. Rules for relationships

Relationships are required when the knowledge item has structural dependencies.

`depends_on` is required when:

the item assumes understanding or use of another KID

`related_kids` is required when:

the item is meaningfully connected but not dependent

`supersedes` is required when:

the new item replaces or absorbs an older one

`replaced_by` is usually generated or managed through deprecation flow

Relationship minimum rule

A KID with meaningful dependency structure should not remain relationship-empty just because the author skipped that work.

6. Rules for bundle membership

Bundles should not be treated as storage categories.

A KID may be included in a bundle when:

- it is relevant to a repeatable use case
- it belongs in an execution pack
- it belongs in a role/executor pack
- it belongs in a run-profile pack

Bundle rules

- bundle membership must be based on KID or metadata
- bundles cannot depend on folder location
- bundles do not define the KID's identity

- bundle membership can change without moving the file

7. Rules for collection descriptors

Collection descriptors exist to describe browse groupings and library context.

They may be created when:

- a browse grouping needs a summary
- a domain/industry/stack collection needs guidance
- there is useful context shared across many KIDs

Collection descriptors must not replace item-level metadata.

Collection rule

If an item's identity depends on a collection descriptor to be understandable, the item metadata is too weak.

8. Rules for views

Views are generated navigation surfaces only.

A view may be added when:

- there is a stable browsing need
- the view can be generated from metadata
- the view helps operators or agents navigate the library

Views must not become manual storage trees.

View rule

No canonical KID is authored directly under `VIEWS/`.

9. Rules for deprecation

A KID should be deprecated when:

- it is outdated and no longer recommended
- it has been replaced by a better canonical item
- it duplicates a stronger canonical item
- its taxonomy or structure is obsolete

Deprecation requirements

When deprecating a KID:

- set `status: deprecated`
- add deprecation entry in `deprecations.json`
- specify `replaced_by` when applicable
- preserve alias/legacy redirect if still needed
- do not silently remove it from the registry without record

Archive rule

Deprecated does not mean deleted.

Deletion should be rare and usually follow a governed archival process.

10. Rules for moving or renaming

Canonical KIDs should almost never need path moves once in `CONTENT/ITEMS/`.

The only normal move pattern is:

- migration from legacy tree to canonical store

Rename rule

Do not rename a file arbitrarily.

If canonical naming is tied to `kid`, the filename should remain:

{kid}.md

That makes moves and renames mostly unnecessary.

11. Rules for generated registries

These files are generated artifacts:

- `knowledge.index.json`
- `aliases.index.json`
- `relationships.index.json`

They must not be hand-edited except for temporary debugging in a non-authoritative environment.

Registry rule

If a registry looks wrong, fix:

- canonical item metadata
- manual override file
- builder logic

Do not patch the generated output and call it done.

12. Rules for manual overrides

Manual override files are allowed only where generation cannot fully express intent.

Examples:

- `aliases.manual.json`
- `relationships.manual.json`

Override rule

Overrides must be:

- minimal
- documented
- reviewed
- non-duplicative of canonical frontmatter

Overrides are not a second content system.

13. Rules for quality gates

A library update is valid only if it passes the registry build pipeline.

Minimum gate:

- canonical validation passes
- no duplicate KIDs
- no invalid enum values
- no broken relationship references
- no alias collisions in strict mode

Stronger gate

For higher-quality updates also require:

- no placeholder titles
- no unknown primary domains
- tags present where useful
- relationship fields present where expected

14. Rules for ownership

Every update should have a clear ownership reason.

You need to know:

- who added it
- why it was added
- what it changes
- whether it changes taxonomy
- whether it changes bundle selection
- whether it introduces deprecation

This can be lightweight, but it should exist.

15. Rules for library update types

Every update should be classified as one of these:

Type A — content update

Improves or corrects existing KID content.

Type B — new KID

Adds a new canonical item.

Type C — taxonomy update

Adds or changes a controlled classification value.

Type D — relationship update

Changes dependencies, related links, or supersession.

Type E — alias update

Adds or removes lookup aliases.

Type F — bundle update

Changes bundle membership or bundle logic.

Type G — deprecation update

Marks item deprecated or replaced.

Type H — migration/compatibility update

Maintains old-path compatibility or legacy redirects.

This makes changes easier to review and reason about.

16. Rules for valid library updates

A library update counts as valid only if:

- it belongs to one or more update types
- it modifies the correct governed zone

- it includes required metadata changes
- it passes the build pipeline
- it does not bypass controlled taxonomy
- it does not introduce hidden parallel structure

17. Anti-patterns to forbid

These are the behaviors that will rot the library again.

Forbidden pattern 1

Adding new canonical items under `PILLARS/`

Forbidden pattern 2

Hand-editing generated registries as the permanent fix

Forbidden pattern 3

Creating new taxonomy values for one-off cases

Forbidden pattern 4

Encoding meaning only in file paths

Forbidden pattern 5

Using bundles as storage categories

Forbidden pattern 6

Using views as canonical content homes

Forbidden pattern 7

Silently overwriting a KID's semantic identity

Forbidden pattern 8

Leaving migrated items without explicit metadata because "the path already tells us"

18. Governance checklist for every update

Before accepting any library change, check:

- Is this a canonical content change, taxonomy change, alias change, relationship change, bundle change, or deprecation?
- Did the change happen in the correct zone?
- Does the KID have valid metadata?
- Are taxonomy changes truly necessary?
- Are relationships declared where needed?
- Will registries regenerate cleanly?
- Does this create hidden duplication or parallel structure?

19. Minimum policy statement

You can phrase the library policy like this:

Canonical knowledge lives only in `CONTENT/ITEMS`.
Classification is defined by metadata and controlled taxonomy.
Registries and views are generated artifacts.
Legacy paths are compatibility only.
No update is valid until the registry pipeline passes.

That should be your operating law.

20. Definition of done for governance

The governance layer is working when:

- new items enter through canonical rules only
- taxonomy changes are deliberate and rare
- aliases are useful and controlled
- relationships are explicit
- deprecations are traceable
- registries are generated
- path structure is no longer carrying semantic truth
- the library can grow without needing structural surgery every time

change-request template

Use this as the change-request template for knowledge library updates.

This is what turns governance from “good ideas” into a repeatable operating process.

Purpose

Every library update should be submitted in one consistent shape so you can tell:

- what is changing
- why it is changing
- what zone it touches
- whether it changes taxonomy
- whether it changes canonical content
- whether it changes aliases, relationships, bundles, or deprecations
- what must be validated before accepting it

Without this, updates become ad hoc again.

Where this should live

Put the template in:

Axion/libraries/knowledge/SYSTEM/templates/knowledge_change_request.md

You can also create a JSON companion later, but start with markdown.

The template

```
# Knowledge Library Change Request

## 1. Change summary
- Change title:
- Change owner:
```

- Date:
- Update type:
 - [ ] Type A — content update
 - [ ] Type B — new KID
 - [ ] Type C — taxonomy update
 - [ ] Type D — relationship update
 - [ ] Type E — alias update
 - [ ] Type F — bundle update
 - [ ] Type G — deprecation update
 - [ ] Type H — migration/compatibility update

2. Objective
Describe what problem this change solves and why it is needed.

3. Affected zone(s)
- [ ] CONTENT/ITEMS
- [ ] SYSTEM/taxonomy
- [ ] SYSTEM/registries
- [ ] SYSTEM/contracts
- [ ] SYSTEM/policies
- [ ] SYSTEM/templates
- [ ] VIEWS
- [ ] BUNDLES
- [ ] REUSE
- [ ] Legacy compatibility only

4. Affected KIDs
List all canonical KIDs affected by this update.

- Added:
- Updated:
- Deprecated:
- Replaced:
- Removed from active use:

5. Canonical content impact
Explain the content change.

- Is a new KID being created?
- Is an existing KID being revised?
- Is semantic identity changing?
- Does this require a new KID instead of mutating an old one?

6. Metadata impact

List metadata fields being added or changed.

- `content_type`:
- `primary_domain`:
- `industry_refs`:
- `stack_family_refs`:
- `pillar_refs`:
- `status`:
- `authority_tier`:
- `bundle_refs`:
- `tags`:
- `depends_on`:
- `related_kids`:
- `supersedes`:
- `legacy_path`:
- `aliases`:

7. Taxonomy impact
- Does this change add a new taxonomy value?
- If yes, which taxonomy set?
- Why can existing taxonomy not express this?
- Will more than one KID use the new value?
- Is this really taxonomy, or should it be a tag/bundle/collection instead?

8. Alias impact
- Are new aliases being introduced?
- Why are they needed?
- Are they migration aliases, search aliases, or legacy-name aliases?
- Could any alias collide with another KID?

9. Relationship impact
- New `depends_on`:
- New `related_kids`:
- New `supersedes`:
- New `replaced_by`:
- Legacy redirects affected:

10. Bundle impact
- Bundles added to:
- Bundles removed from:
- Does this change affect bundle selection logic?
- Is bundle membership metadata-driven or explicit?

11. Deprecation impact

- Is any KID being deprecated?
- Replacement KID:
- Reason:
- Effective date:
- Backward compatibility needed?

12. Migration / compatibility impact
- Does this change preserve old paths?
- Does it require alias redirects?
- Does it depend on legacy fallback behavior?
- Can legacy compatibility later be removed?

13. Validation checklist
- [ ] Canonical item frontmatter is valid
- [ ] Filename matches KID
- [ ] Required metadata fields are present
- [ ] Taxonomy values are valid
- [ ] Relationship references point to real KIDs
- [ ] Alias collisions checked
- [ ] Bundle references checked
- [ ] Deprecation records updated if needed
- [ ] Registry build pipeline passes
- [ ] No generated registry was hand-edited as the real fix

14. Risk check
What could go wrong if this change is wrong?

- Retrieval risk:
- Taxonomy drift risk:
- Backward compatibility risk:
- Duplicate-content risk:
- Bundle-selection risk:

15. Acceptance decision
- Status:
  - [ ] Approved
  - [ ] Approved with revisions
  - [ ] Rejected
- Reviewer:
- Notes:

How to use it

Every update should start by selecting one or more update types.

That matters because different update types require different scrutiny.

Required review rules by change type

Type A — content update

Reviewer checks:

- content improved without changing semantic identity
- no hidden need for a new KID
- metadata still fits

Type B — new KID

Reviewer checks:

- KID is actually needed
- not a duplicate of an existing item
- metadata is complete
- classification is correct

Type C — taxonomy update

Reviewer checks:

- this is truly taxonomy, not a tag
- value is reusable
- it does not duplicate an existing concept

Type D — relationship update

Reviewer checks:

- links are structurally correct
- referenced KIDs exist
- supersession logic makes sense

Type E — alias update

Reviewer checks:

- alias is useful
- alias is not vanity
- no collision risk

Type F — bundle update

Reviewer checks:

- bundle change is justified
- membership is not path-based
- bundle is not becoming a storage category

Type G — deprecation update

Reviewer checks:

- deprecation is explicit
- replacement path is clear
- compatibility handling exists

Type H — migration/compatibility update

Reviewer checks:

- temporary compatibility is justified
- it does not become permanent accidental complexity

Minimum required fields for approval

A change request should not be accepted unless it clearly states:

- update type
- objective
- affected zones
- affected KIDs
- metadata impact
- taxonomy impact if any
- relationship impact if any
- validation status

Short-form version

For small updates, you can use a compact version.

```
# Knowledge Change Request — Short Form

## Summary
- Title:
- Owner:
- Update type:
- Affected KID(s):
```

What is changing?
-

Why?
-

Metadata / taxonomy / alias / relationship impact
-

Validation
- [ ] Pipeline passes
- [ ] No duplicate KIDs
- [ ] No invalid taxonomy
- [ ] No broken relationships

Decision
- [ ] Approved
- [ ] Rejected

Suggested policy

Use the full template when the change involves:

- new KIDs
- taxonomy changes
- deprecations
- migration logic
- multiple affected items

Use the short form only for:

- minor content corrections
- small metadata cleanup
- low-risk relationship updates

Best practice

Do not treat this as bureaucratic paperwork.
Treat it as the structured reasoning layer that prevents bad library drift.

A good change request should make it obvious:

- whether the change is legitimate
- whether it belongs in the right zone
- whether it creates future maintenance burden
- whether it should really be a new KID, new taxonomy value, alias, or just a tag

KID authoring template

Use this as the KID authoring template.

This is what makes new canonical items start clean instead of needing repair later.

Purpose

Every new KID should begin with:

- valid frontmatter
- stable identity
- predictable structure
- enough metadata for indexing, search, bundles, and relationships
- enough body structure to be useful without becoming bloated

This template is for:

Axion/libraries/knowledge/CONTENT/ITEMS/

Where this should live

Put the template in:

Axion/libraries/knowledge/SYSTEM/templates/KID.template.md

The template

```
---
kid: KID-PLACEHOLDER-0001
title: Replace Me
content_type: concept
primary_domain: replace_me
secondary_domains: []
industry_refs: []
stack_family_refs: []
pillar_refs: []
```

status: draft
authority_tier: draft
freshness_class: versioned_stable
bundle_refs: []
tags: []
aliases: []
depends_on: []
related_kids: []
supersedes: []
source_refs: []
legacy_path: null

Summary
One short paragraph explaining what this KID is, what it covers, and when it should be used.

Purpose
Explain why this knowledge item exists and what problem it solves.

Scope
State what is included and what is not included.

In scope
-

Out of scope
-

Core content
Put the actual reusable knowledge here.

Rules / guidance
List the main rules, recommendations, or operating guidance.

Examples
Add concrete examples, scenarios, patterns, or anti-patterns where useful.

Related knowledge
- Depends on:
- Related:
- Supersedes:

Usage notes
Explain how this item should be used by:

- operators
- agents
- control plane
- verifiers
if relevant.

Maintenance notes
Document anything that affects future updates:
- volatility
- review cadence
- likely future changes
- migration notes

Required frontmatter fields

These must always be present:

- `kid`
- `title`
- `content_type`
- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `status`
- `authority_tier`

Recommended frontmatter defaults

Use these defaults unless you have a reason not to:

status: draft
authority_tier: draft
freshness_class: versioned_stable
secondary_domains: []
industry_refs: []
stack_family_refs: []
pillar_refs: []
bundle_refs: []
tags: []
aliases: []
depends_on: []
related_kids: []
supersedes: []
source_refs: []
legacy_path: null

Frontmatter field guidance

`kid`

Stable canonical ID.

Rules:

- never reuse an old KID for a different semantic item
- once assigned, it should remain stable
- filename should match it exactly

Example:

kid: KID-SOL-AUTH-PATTERN-0012

`title`

Human-readable name.

Rules:

- should be specific
- should describe the item's actual content
- avoid placeholders like "Example Title"

`content_type`

Use only controlled values.

Recommended set:

- `concept`
- `pattern`
- `checklist`
- `workflow`
- `reference`
- `standard`
- `playbook`
- `integration`
- `compliance`
- `data_model`
- `security_risk`
- `kpi_metric`

`primary_domain`

Main conceptual domain.

Examples:

- `authentication`
- `api_design`
- `monitoring_and_alerting`
- `financial_services`
- `react`

`secondary_domains`

Optional supporting domains.

Use only when the item clearly spans multiple domains.

`industry_refs`

Use for industry classification.

Examples:

- `financial_services`
- `healthcare`
- `government_public_sector`

`stack_family_refs`

Use for technology classification.

Examples:

- `react`
- `python`
- `docker`
- `postgres`

`pillar_refs`

Use semantic pillars, not old path branches.

Recommended pillars:

- `industry_knowledge`
- `solution_patterns`
- `technology_knowledge`
- `governance_and_quality`
- `execution_and_operations`

`status`

Recommended values:

- `draft`
- `active`
- `deprecated`
- `archived`

`authority_tier`

Recommended values:

- `draft`
- `reviewed`
- `verified`
- `golden`

`freshness_class`

Recommended values:

- `volatile`
- `periodic_review`
- `versioned_stable`
- `historical`

`bundle_refs`

Only include if the item is known to belong in curated bundles.

`tags`

Use for search enrichment, not core taxonomy.

`aliases`

Only include useful alternate lookup names.

`depends_on`

Use when understanding or applying this KID requires another KID.

`related_kids`

Use for strong relationships that are not strict dependencies.

`supersedes`

Use when this KID replaces or absorbs older KIDs.

`source_refs`

Use for provenance references if needed.

`legacy_path`

Only for migrated items or backward compatibility.

Body section guidance

`# Summary`

Must answer:

- what is this
- when should it be used
- why does it matter

Keep it short and readable.

`# Purpose`

Explain the reason the item exists.
This helps prevent duplicate KIDs.

Scope

This is important because it keeps KIDs from becoming giant junk drawers.

Core content

This is the actual reusable knowledge payload.

It should be:

- concrete
- reusable
- internally coherent
- not padded with filler

Rules / guidance

Good for:

- guardrails
- standards
- decision rules
- expected behavior

Examples

Examples make retrieval much more useful.
Especially valuable for:

- patterns

- checklists
- workflows
- integrations

Related knowledge

This is the human-readable complement to structured relationship fields.

Usage notes

Useful when the same KID is consumed differently by:

- agents
- operators
- control plane
- verification systems

Maintenance notes

Important for volatile domains and future cleanup.

Variant templates by content type

You should not use one totally identical body shape for every content type.

Use the shared frontmatter, but vary the body slightly.

Concept template

Summary
Purpose
Scope
Core concept

Key properties
Common misunderstandings
Examples
Related knowledge
Maintenance notes

Pattern template

Summary
Purpose
Scope
Pattern
When to use
When not to use
Implementation notes
Examples
Related knowledge
Maintenance notes

Checklist template

Summary
Purpose
Scope
Checklist
Pass / fail criteria
Common failure modes
Related knowledge
Maintenance notes

Workflow template

Summary
Purpose
Scope
Preconditions
Workflow steps
Outputs
Failure handling
Related knowledge
Maintenance notes

Standard template

Summary
Purpose
Scope
Standard
Required rules
Exceptions
Verification notes
Related knowledge
Maintenance notes

Example filled-out frontmatter

---
kid: KID-SOL-AUTH-PATTERN-0012
title: OAuth Integration Pattern
content_type: pattern
primary_domain: authentication
secondary_domains:
 - identity
industry_refs: []
stack_family_refs: []
pillar_refs:
 - solution_patterns
status: active
authority_tier: reviewed
freshness_class: versioned_stable
bundle_refs:
 - FULLSTACK_SAAS
tags:
 - oauth
 - identity
 - session-management
aliases:
 - oauth-pattern
depends_on:
 - KID-SOL-IDENTITY-CONCEPT-0004
related_kids:
 - KID-SOL-AUTH-CHECKLIST-0007
supersedes: []
source_refs: []
legacy_path:
PILLARS/IT_END_TO_END/03_backend_and_apis/authentication/oauth-pattern.md
---

Authoring rules

Rule 1

One KID should represent one coherent reusable unit.

Rule 2

Do not create a new KID if the content is just a tiny note that belongs inside an existing item.

Rule 3

Do not overload one KID with multiple unrelated concerns.

Rule 4

Do not rely on path location to explain what the item is.

Rule 5

Do not leave placeholder frontmatter values in committed canonical content.

Rule 6

If semantic identity changes significantly, make a new KID instead of mutating the old one invisibly.

Quality checklist for a new KID

Before accepting a new item, verify:

- filename matches `kid`
- title is specific
- content type is valid

- primary domain is not vague garbage
- pillar refs make sense
- tags are useful
- body has real content, not a stub
- relationships are declared where relevant
- registry build passes

Recommended companion templates

After this, the next templates you should have are:

- `collection_descriptor.template.md`
- `bundle_definition.template.md`
- `taxonomy_change_request.md`
- `deprecation_request.md`

collection descriptor template

Use this as the collection descriptor template.

This is the template for the files that replace the old buried `_meta.md` files and give browseable groupings a clean, explicit structure.

Purpose

A collection descriptor explains a grouping of KIDs such as:

- an industry
- a domain
- a stack family
- a pillar
- a bundle-facing collection

It is not a canonical knowledge item itself.
It is contextual metadata for browsing, navigation, and grouping.

Where this should live

Put the template in:

Axion/libraries/knowledge/SYSTEM/templates/collection_descriptor.template.md

Collection descriptor files themselves should live in:

Axion/libraries/knowledge/CONTENT/META/collections/

The template

```
---
collection_key: domain.replace_me
collection_type: domain
title: Replace Me
status: active
```

legacy_path: null
scope:
 primary_domain: replace_me
 industry_refs: []
 stack_family_refs: []
 pillar_refs: []
related_views: []
related_kids: []
notes: []

Summary
One short paragraph describing what this collection represents and how it should be used for browsing or grouping.

Purpose
Explain why this collection exists and what it groups together.

Scope
Describe what belongs in this collection and what does not.

In scope
-

Out of scope
-

Inclusion guidance
Explain the rules for which KIDs should be associated with this collection.

Navigation guidance
Describe how users, agents, or operators should use this collection when browsing the library.

Related collections
List nearby or overlapping collections.

Maintenance notes
Document anything relevant for future updates:
- migration notes
- legacy mapping
- classification cautions
- expected future changes

Required frontmatter fields

These should always exist:

- `collection_key`
- `collection_type`
- `title`
- `status`
- `scope`

Recommended defaults

status: active
legacy_path: null
related_views: []
related_kids: []
notes: []

Field guidance

`collection_key`

Stable identifier for the collection.

Format:

- `industry.financial_services`
- `domain.authentication`
- `stack.react`

- `pillar.solution_patterns`

Rules:

- lowercase
- dot-separated
- stable over time
- should reflect semantic grouping, not folder history

`collection_type`

Use controlled values.

Recommended:

- `industry`
- `domain`
- `stack_family`
- `pillar`
- `bundle_context`
- `custom`

`title`

Human-readable label.

Examples:

- `Financial Services`

- `Authentication`
- `React`
- `Solution Patterns`

status

Recommended values:

- `active`
- `deprecated`
- `archived`

legacy_path

Only needed when the collection was migrated from an old `_meta.md` path.

scope

This is the most important field.

It defines what metadata the collection corresponds to.

Examples:

Domain collection

```
scope:
 primary_domain: authentication
 industry_refs: []
 stack_family_refs: []
 pillar_refs:
   - solution_patterns
```

Industry collection

```
scope:
 primary_domain: null
 industry_refs:
   - financial_services
 stack_family_refs: []
 pillar_refs:
   - industry_knowledge
```

Stack collection

```
scope:
 primary_domain: null
 industry_refs: []
 stack_family_refs:
   - react
 pillar_refs:
   - technology_knowledge
```

`related_views`

Use this to point to generated browse pages.

Examples:

- `VIEWS/by_domain/authentication.md`
- `VIEWS/by_industry/financial_services.md`

`related_kids`

Optional curated list of especially important KIDs for the collection.

This should be selective, not exhaustive.

`notes`

Freeform structured notes for maintainers.

Collection authoring rules

Rule 1

A collection descriptor describes a grouping. It does not replace item metadata.

Rule 2

A collection must be derivable from item metadata.
If it cannot be described through metadata scope, it is probably not a stable collection.

Rule 3

Do not use collections to smuggle in a second taxonomy system.

Rule 4

Do not copy large amounts of item content into the collection file.

Rule 5

If a collection overlaps heavily with another collection, explain the distinction clearly in `Scope` or `Related collections`.

Recommended body guidance

`# Summary`

State what the grouping is and why someone would browse it.

`# Purpose`

Explain why the grouping exists at all.

`# Scope`

This prevents collections from becoming vague and overlapping.

Inclusion guidance

This is where you explain what kinds of KIDs belong here.

Good examples:

- “Use this collection for KIDs whose primary domain is authentication.”
- “Use this collection for KIDs that materially serve financial services workflows or compliance.”
- “Use this collection for KIDs centered on React-specific implementation guidance.”

Navigation guidance

Explain how this collection should help:

- operators browsing the library
- agents retrieving relevant KIDs
- maintainers deciding classification

Related collections

Useful for overlaps like:

- `domain.authentication` and `stack.react`
- `industry.financial_services` and `domain.compliance`
- `pillar.solution_patterns` and `domain.api_design`

Maintenance notes

Capture migration assumptions or future cleanup notes.

Example: domain collection

---
collection_key: domain.authentication
collection_type: domain
title: Authentication
status: active
legacy_path: PILLARS/IT_END_TO_END/03_backend_and_apis/authentication/_meta.md
scope:
 primary_domain: authentication
 industry_refs: []
 stack_family_refs: []
 pillar_refs:
   - solution_patterns
related_views:
 - VIEWS/by_domain/authentication.md
related_kids: []
notes: []

Summary
This collection groups KIDs whose primary focus is authentication design, implementation patterns, controls, and operational guidance.

Purpose
Use this collection to browse canonical items related to authentication as a domain, independent of industry or specific stack.

Scope
Authentication mechanisms, login/session flows, identity integration patterns, and related operational guidance.

In scope
- OAuth patterns
- session management
- authentication decision rules
- auth-related checklists

Out of scope
- general authorization unless tightly coupled
- unrelated identity governance topics
- framework-specific implementation details unless auth is primary

Inclusion guidance

Associate KIDs here when `primary_domain: authentication`.

Navigation guidance
Use this collection when browsing for auth-specific guidance before narrowing by stack or industry.

Related collections
- domain.identity
- stack.react
- pillar.solution_patterns

Maintenance notes
Review overlap with identity-related collections if the taxonomy expands.

Example: industry collection

```
---
collection_key: industry.financial_services
collection_type: industry
title: Financial Services
status: active
legacy_path: PILLARS/INDUSTRY_PLAYBOOKS/finance/_meta.md
scope:
 primary_domain: null
 industry_refs:
   - financial_services
 stack_family_refs: []
 pillar_refs:
   - industry_knowledge
related_views:
 - VIEWS/by_industry/financial_services.md
related_kids: []
notes: []
---
```

Summary
This collection groups KIDs relevant to financial-services products, workflows, controls, and domain expectations.

Purpose
Use this collection to browse knowledge specific to financial-services environments.

Scope

Financial-services domain knowledge, compliance-aware workflows, sector-specific data and operational patterns.

In scope
- payments
- reconciliation
- financial compliance guidance
- banking-adjacent system patterns

Out of scope
- generic SaaS guidance
- industry-neutral API patterns
- stack-specific docs with no sector relevance

Inclusion guidance
Associate KIDs here when they materially apply to financial-services workflows, controls, or sector-specific requirements.

Navigation guidance
Use this collection to narrow a search from general system guidance into industry-specific constraints and expectations.

Related collections
- industry.payments
- domain.compliance
- pillar.industry_knowledge

Maintenance notes
Keep boundaries clear between general compliance and sector-specific compliance.

Naming rules

Use these filename patterns:

- `domain.authentication.md`
- `industry.financial_services.md`
- `stack.react.md`
- `pillar.solution_patterns.md`

That keeps collection files easy to scan and deterministic.

Acceptance checklist

A collection descriptor is valid when:

- `collection_key` is stable and correctly typed
- `scope` is explicit
- it describes a real reusable grouping
- it does not duplicate canonical item content
- it helps browsing and classification
- it can be matched from item metadata

bundle definition template

Use this as the bundle definition template.

This standardizes the other major grouping layer so bundles stay clean, metadata-driven, and separate from canonical content.

Purpose

A bundle defines a curated set of KIDs for a repeatable use case, such as:

- an executor pack
- a risk-class pack
- a run-profile pack
- a solution starter set
- an operator-focused working set

A bundle is not a canonical content home.
It is a selection mechanism over canonical KIDs.

Where this should live

Put the template in:

Axion/libraries/knowledge/SYSTEM/templates/bundle_definition.template.md

Actual bundle definitions can live as markdown or JSON under:

Axion/libraries/knowledge/BUNDLES/

Since your registries are machine-driven, I recommend:

- human-readable bundle design note in markdown
- machine-readable bundle entry in `bundles.index.json`

The markdown template

```
---
bundle_id: BUNDLE-REPLACE_ME
title: Replace Me
status: active
bundle_type: run_profile
selection_mode: explicit
kid_refs: []
filters:
 primary_domain: []
 industry_refs: []
 stack_family_refs: []
 pillar_refs: []
 content_type: []
 status: []
 authority_tier: []
 tags: []
legacy_path: null
notes: []
---

# Summary
One short paragraph describing what this bundle is for and when it should be used.

# Purpose
Explain why this bundle exists and what repeatable use case it supports.

# Scope
Describe what belongs in the bundle and what does not.

## In scope
-

## Out of scope
-

# Selection logic
Explain how items enter this bundle.

# Usage guidance
Describe how operators, agents, or control-plane systems should use this bundle.

# Included KIDs
```

List especially important included KIDs if useful.

Related bundles
List nearby or overlapping bundles.

Maintenance notes
Document future update concerns, drift risks, or migration notes.

Required frontmatter fields

These should always exist:

- `bundle_id`
- `title`
- `status`
- `bundle_type`
- `selection_mode`

Recommended defaults

status: active
selection_mode: explicit
kid_refs: []
legacy_path: null
notes: []
filters:
 primary_domain: []
 industry_refs: []
 stack_family_refs: []
 pillar_refs: []
 content_type: []
 status: []
 authority_tier: []
 tags: []

Field guidance

`bundle_id`

Stable identifier.

Examples:

- `FULLSTACK_SAAS`
- `RUN_PROFILE_PRODUCTION`
- `EXECUTOR_OPERATOR_CORE`
- `RISK_CLASS_HIGH_GOVERNANCE`

Rules:

- stable
- uppercase with underscores is fine
- should represent the bundle's operating purpose, not a folder name

`title`

Human-readable label.

Examples:

- `Fullstack SaaS`
- `Production Run Profile`
- `Operator Core`
- `High Governance Risk Class`

status

Recommended values:

- `active`
- `deprecated`
- `archived`

bundle_type

Recommended values:

- `executor`
- `risk_class`
- `run_profile`
- `solution_pack`
- `starter_set`
- `custom`

selection_mode

Use one of:

- `explicit`
- `filter`
- `hybrid`

`explicit`

Manual `kid_refs` list only.

`filter`

Pure metadata-based selection only.

`hybrid`

Base `kid_refs` plus metadata-driven additions.

`kid_refs`

Explicit included KIDs.

Use this when:

- certain KIDs must always be in the bundle
- the bundle is curated intentionally
- filter logic alone is not enough

`filters`

Metadata-driven selection rules.

Supported keys should mirror canonical metadata:

- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `content_type`

- `status`
- `authority_tier`
- `tags`

`legacy_path`

Only if migrated from an older bundle organization.

`notes`

Maintainer notes only.

Bundle authoring rules

Rule 1

A bundle must select from canonical KIDs only.

Rule 2

A bundle must never depend on folder location.

Rule 3

A bundle must not redefine KID identity.

Rule 4

A bundle should exist only for repeatable selection needs, not one-off convenience piles.

Rule 5

If a grouping is really a semantic category, it probably belongs in taxonomy or a collection, not a bundle.

Selection guidance

Use `explicit` when:

- bundle membership is intentionally curated
- included KIDs are few and stable
- precision matters more than broad discovery

Use `filter` when:

- membership follows stable metadata rules
- the grouping is broad and systematic
- you want automatic inclusion of future matching KIDs

Use `hybrid` when:

- some KIDs are mandatory anchors
- others should flow in from metadata rules

Example: explicit bundle

```
---
bundle_id: EXECUTOR_OPERATOR_CORE
title: Operator Core
status: active
bundle_type: executor
selection_mode: explicit
kid_refs:
 - KID-SOL-AUTH-PATTERN-0012
 - KID-OPS-MONITORING-STANDARD-0006
 - KID-GOV-CHANGE-CHECKLIST-0003
```

filters:
 primary_domain: []
 industry_refs: []
 stack_family_refs: []
 pillar_refs: []
 content_type: []
 status: []
 authority_tier: []
 tags: []
legacy_path: null
notes: []

Summary
This bundle provides the core KIDs an operator should reach for during normal system operation and review.

Purpose
Use this bundle to give operators a reliable working set without requiring full-library browsing.

Scope
Core operational, governance, and practical run-support knowledge.

In scope
- operational standards
- monitoring guidance
- decision-support patterns
- practical run checklists

Out of scope
- deep stack-specific references
- niche industry knowledge unless essential
- deprecated KIDs

Selection logic
This bundle is curated explicitly for operator workflows.

Usage guidance
Use this bundle as the default operator browsing pack and baseline retrieval context.

Included KIDs
- KID-SOL-AUTH-PATTERN-0012
- KID-OPS-MONITORING-STANDARD-0006
- KID-GOV-CHANGE-CHECKLIST-0003

Related bundles
- RUN_PROFILE_PRODUCTION
- RISK_CLASS_HIGH_GOVERNANCE

Maintenance notes
Review when operator workflows materially change.

Example: filter bundle

---
bundle_id: RUN_PROFILE_PRODUCTION
title: Production Run Profile
status: active
bundle_type: run_profile
selection_mode: filter
kid_refs: []
filters:
 primary_domain: []
 industry_refs: []
 stack_family_refs: []
 pillar_refs:
   - execution_and_operations
   - governance_and_quality
 content_type:
   - standard
   - checklist
   - workflow
 status:
   - active
 authority_tier:
   - reviewed
   - verified
   - golden
 tags:
   - production
legacy_path: null
notes: []

Summary
This bundle selects KIDs relevant to production-grade operation, governance, and execution.

Purpose
Use this bundle to assemble a production-oriented working set from canonical metadata.

Scope
Production-facing operations, quality, and governance knowledge.

In scope
- production operational standards
- run checklists
- governance rules
- execution workflows

Out of scope
- draft-only concepts
- low-confidence exploratory notes
- highly niche stack references without production relevance

Selection logic
This bundle is generated from metadata filters over pillar, content type, status, authority, and tags.

Usage guidance
Use this bundle as a default pack for production workflows and higher-confidence retrieval.

Included KIDs
Generated from filters.

Related bundles
- EXECUTOR_OPERATOR_CORE
- RISK_CLASS_HIGH_GOVERNANCE

Maintenance notes
Keep filter logic aligned with authority tier policy.

Recommended machine-readable shape

In `bundles.index.json`, each entry should follow a structure like:

```
{
 "RUN_PROFILE_PRODUCTION": {
   "title": "Production Run Profile",
   "status": "active",
   "bundle_type": "run_profile",
```

```
  "selection_mode": "filter",
  "kid_refs": [],
  "filters": {
    "primary_domain": [],
    "industry_refs": [],
    "stack_family_refs": [],
    "pillar_refs": ["execution_and_operations", "governance_and_quality"],
    "content_type": ["standard", "checklist", "workflow"],
    "status": ["active"],
    "authority_tier": ["reviewed", "verified", "golden"],
    "tags": ["production"]
  }
 }
}
```


Bundle acceptance checklist

A bundle is valid when:

- it has a stable `bundle_id`
- it has a clear operating purpose
- it selects only canonical KIDs
- it uses metadata, not paths
- its selection logic is understandable
- it is not duplicating taxonomy
- it is not acting as a hidden storage category

Bundle anti-patterns to avoid

Do not:

- use bundles as new folder trees

- pack random unrelated KIDs together
- rely on old `PILLARS/` structure
- make bundles the only way to find essential content
- create too many near-duplicate bundles

taxonomy change request template

Use this as the taxonomy change request template.

This is the control point that keeps taxonomy from turning into a dumping ground for every new idea.

Purpose

A taxonomy change request is required whenever you want to:

- add a new controlled vocabulary value
- rename an existing taxonomy value
- split one taxonomy value into multiple values
- merge overlapping taxonomy values
- deprecate a taxonomy value
- change the meaning or intended usage of a taxonomy value

Without this, taxonomy grows in an ad hoc way and starts causing classification drift, duplicate categories, and messy retrieval.

Where this should live

Put the template in:

Axion/libraries/knowledge/SYSTEM/templates/taxonomy_change_request.md

The template

```
# Taxonomy Change Request

## 1. Change summary
- Change title:
- Change owner:
- Date:
- Taxonomy area:
```

- [ ] content_type
- [ ] pillar
- [ ] primary_domain
- [ ] industry_refs
- [ ] stack_family_refs
- [ ] status
- [ ] authority_tier
- [ ] freshness_class
- [ ] other

- Change type:
- [ ] add value
- [ ] rename value
- [ ] split value
- [ ] merge values
- [ ] deprecate value
- [ ] redefine usage guidance

2. Objective
What problem does this taxonomy change solve?

3. Current state
Describe the current taxonomy value(s) and why they are insufficient, confusing, overlapping, or obsolete.

4. Proposed change
Describe the exact proposed taxonomy change.

- Existing value(s):
- Proposed new value(s):
- Affected taxonomy file(s):
- Affected documentation:
- Affected KIDs:

5. Justification
Why should this be taxonomy instead of:
- a tag
- a bundle
- a collection descriptor
- a relationship
- a one-off metadata note

6. Reuse test
Answer all of these:

- Will more than one KID use this value?
- Is this concept stable over time?
- Is it distinct from existing values?
- Is it likely to remain useful for retrieval and browsing?
- Is it likely to be used consistently?

7. Alternatives considered
List alternatives you considered and why they were rejected.

8. Impact analysis
- Affected KIDs:
- Affected collections:
- Affected bundles:
- Affected resolver/search behavior:
- Affected views:
- Migration required:
- Backward compatibility required:

9. Collision / overlap review
Which existing taxonomy values are closest to this proposal?
Explain why this is not a duplicate, synonym, or unnecessary variant.

10. Migration plan
If approved, how will existing KIDs and registries be updated?

- Metadata updates needed:
- Collection updates needed:
- Bundle updates needed:
- Alias or compatibility updates needed:
- Registry rebuild required:
- Legacy support needed:

11. Validation checklist
- [ ] This is not better represented as a tag
- [ ] This is not better represented as a bundle
- [ ] This is not better represented as a collection
- [ ] This value will be used by more than one KID
- [ ] This value is distinct from existing values
- [ ] Overlap with neighboring values has been reviewed
- [ ] Migration impact is understood
- [ ] Registry build impact is understood

12. Risk check

What could go wrong if this taxonomy change is wrong?

- Retrieval risk:
- Classification drift risk:
- Duplicate-category risk:
- Migration risk:
- Search relevance risk:

13. Decision
- Status:
 - [ ] Approved
 - [ ] Approved with revisions
 - [ ] Rejected
- Reviewer:
- Notes:

When this template is required

Use it for:

- new `content_type`
- new `pillar`
- new domain families
- new industry values
- new stack-family values
- renaming or splitting existing taxonomy
- deprecating taxonomy values

Do not require it for:

- normal `tags`
- single-item aliases

- bundle membership changes
- simple content edits

Approval rules by taxonomy area

`content_type`

Very strict.

A new content type should only be added when it represents a genuinely different knowledge-unit form, not just a different topic.

Bad examples:

- `frontend_pattern`
- `security_concept`
- `payments_checklist`

Those are topic-specific, not type-specific.

Good examples:

- `pattern`
- `checklist`
- `workflow`
- `standard`

`pillar`

Very strict.

A pillar should represent a durable semantic role in the library, not a short-term organizational convenience.

`primary_domain`

Moderately strict.

Domains are useful, but they can sprawl if every narrow topic becomes a domain.

Ask:

- is this actually a domain?
- or is it a tag / stack / industry / subtopic?

`industry_refs`

Moderately strict.

Use when the category reflects a real industry context, not just a business feature area.

`stack_family_refs`

Moderately strict.

Use for durable technology families, not tiny version-specific distinctions.

`status`, `authority_tier`, `freshness_class`

Very strict.

These are system-control taxonomies and should change rarely.

Review questions to enforce

Before approving any taxonomy change, ask:

- Is this a reusable controlled classification or just a convenient label?

- Will multiple items need this?
- Is this distinct from a neighboring value?
- Will users retrieve better with this added?
- Does it create more clarity than complexity?

Examples

Good taxonomy change

Adding `eventing` as a primary domain if multiple KIDs cover:

- event design
- event delivery guarantees
- event schema strategy
- event-driven workflows

That is a reusable domain.

Bad taxonomy change

Adding `stripe-webhooks` as a primary domain.

That is probably:

- a tag
- a stack-specific reference
- maybe a KID topic
  but not a durable domain taxonomy value.

Good stack-family change

Adding `fastapi` when there are multiple KIDs around:

- FastAPI patterns
- FastAPI deployment guidance
- FastAPI integration notes

Bad stack-family change

Adding `fastapi-0-115`.

That is too version-specific for taxonomy.

Strong anti-drift rules

Do not approve taxonomy changes that are:

- single-use only
- synonyms of existing values
- overly narrow
- path-driven leftovers from old folder structures
- temporary migration hacks
- really just tags in disguise

Acceptance standard

A taxonomy change should only be approved if it improves:

- classification clarity
- search/retrieval quality

- browsing coherence
- long-term maintainability

and does not just increase category count.

Practical policy statement

You can use this as the operating rule:

Taxonomy is for durable, reusable classification.
If a concept is narrow, temporary, synonymous, or one-off, it should be a tag, bundle, collection note, or content detail instead.

deprecation request template

Use this as the deprecation request template.

This gives you a controlled way to retire KIDs, bundles, collections, or taxonomy values without creating broken references or silent drift.

Purpose

A deprecation request is required whenever you want to mark something as no longer recommended for active use.

Use it when you need to:

- deprecate a KID
- deprecate a bundle
- deprecate a collection descriptor
- deprecate a taxonomy value
- replace an older canonical item with a newer one
- retire legacy compatibility behavior

Deprecation should be explicit, traceable, and reversible if needed.

Where this should live

Put the template in:

Axion/libraries/knowledge/SYSTEM/templates/deprecation_request.md

The template

```
# Deprecation Request

## 1. Change summary
- Request title:
- Owner:
```

- Date:
- Deprecation target type:
 - [ ] KID
 - [ ] bundle
 - [ ] collection descriptor
 - [ ] taxonomy value
 - [ ] legacy compatibility rule
 - [ ] other

2. Target
- Target ID / key:
- Current location:
- Current status:
- Affected zone:
 - [ ] CONTENT/ITEMS
 - [ ] CONTENT/META/collections
 - [ ] BUNDLES
 - [ ] SYSTEM/taxonomy
 - [ ] SYSTEM/registries
 - [ ] Legacy compatibility

3. Objective
Why is this target being deprecated?

4. Deprecation reason
Choose one or more:

- [ ] replaced by a stronger canonical item
- [ ] duplicate / overlapping content
- [ ] obsolete guidance
- [ ] taxonomy cleanup
- [ ] migration complete
- [ ] legacy compatibility no longer needed
- [ ] bundle no longer justified
- [ ] other

5. Replacement / successor
- Replacement ID / key:
- Is there a direct replacement?
 - [ ] yes
 - [ ] no
- If no, explain why no replacement is needed.

6. Impact analysis

- Affected KIDs:
- Affected bundles:
- Affected collections:
- Affected taxonomy:
- Affected aliases:
- Affected relationships:
- Affected resolver/search behavior:
- Affected views:
- Backward compatibility impact:

7. Metadata / registry changes required
- Status change required:
- `replaced_by` update required:
- `supersedes` update required:
- Alias updates required:
- Relationship updates required:
- Deprecation registry update required:
- View or collection cleanup required:
- Bundle cleanup required:

8. Compatibility plan
How will old references continue to work, if needed?

- Alias redirect needed:
- Legacy path redirect needed:
- Resolver fallback needed:
- Time-limited compatibility needed:
- Safe removal date:

9. Migration plan
What concrete updates are required to complete the deprecation safely?

- Canonical item changes:
- Registry changes:
- Bundle changes:
- Collection changes:
- View changes:
- Taxonomy changes:
- Cleanup steps:

10. Validation checklist
- [ ] Deprecation target is clearly identified
- [ ] Reason for deprecation is explicit
- [ ] Replacement path is clear, if applicable

- [ ] Compatibility impact has been reviewed
- [ ] Alias / redirect impact has been reviewed
- [ ] Relationship changes have been reviewed
- [ ] Registry build impact has been reviewed
- [ ] No active dependency is being silently broken
- [ ] Search / retrieval impact has been considered

11. Risk check
What could go wrong if this deprecation is handled badly?

- Broken lookup risk:
- Search relevance risk:
- Migration confusion risk:
- Duplicate-content risk:
- Compatibility risk:

12. Decision
- Status:
 - [ ] Approved
 - [ ] Approved with revisions
 - [ ] Rejected
- Reviewer:
- Notes:

When to use it

Use this template for:

- retiring an old KID after replacement
- merging duplicate KIDs
- deprecating a bundle that no longer serves a distinct purpose
- retiring collection descriptors created only for migration
- removing old taxonomy values after a split or merge
- removing legacy fallback logic once the migration is complete

KID-specific deprecation rules

When deprecating a KID:

- set `status: deprecated`
- add or update deprecation tracking
- set `replaced_by` where applicable
- preserve aliases or redirects if old references still exist
- do not silently delete the item from canonical history

A deprecated KID should remain traceable even if it stops being recommended.

Taxonomy-specific deprecation rules

When deprecating a taxonomy value:

- identify all affected KIDs
- map each one to replacement values
- rebuild collections/views/bundles affected by the old value
- preserve compatibility only if needed for a limited transition

Taxonomy deprecation is usually higher risk than item deprecation because it affects classification broadly.

Bundle-specific deprecation rules

When deprecating a bundle:

- explain why the bundle no longer deserves to exist
- identify replacement bundle or workflow if any

- remove path-based assumptions if they still exist
- ensure no operator workflow depends on the retired bundle name without notice

Legacy compatibility deprecation rules

When deprecating legacy behavior such as old path fallback:

- confirm the canonical registry path is working
- confirm aliases/redirects cover remaining real usage
- confirm no active workflow still depends on the legacy path
- set a removal point, not indefinite “temporary” support

Recommended companion registry shape

You already have `deprecations.json`. A good entry shape is:

```
{
 "KID-SOL-AUTH-PATTERN-0005": {
   "status": "deprecated",
   "replaced_by": "KID-SOL-AUTH-PATTERN-0012",
   "reason": "Merged into canonical OAuth pattern rewrite.",
   "effective_date": "2026-03-11"
 }
}
```

You can use similar shapes for bundles or taxonomy values if you later expand it.

Review rules

Approve deprecation only when:

- the target is clearly identified

- the reason is real and not just cosmetic cleanup
- replacement handling is clear
- compatibility consequences are understood
- registry rebuild consequences are understood

Do not approve deprecation that creates hidden breakage.

Short-form version

For simple KID deprecations, you can use this compact form:

```
# Deprecation Request — Short Form

## Target
- ID:
- Type:
- Current status:

## Reason
-

## Replacement
- Replaced by:
- [ ] no replacement needed

## Impact
- [ ] aliases reviewed
- [ ] relationships reviewed
- [ ] bundles reviewed
- [ ] registry build reviewed

## Decision
- [ ] Approved
- [ ] Rejected
```

Practical policy statement

Nothing should be silently retired.
If a KID, bundle, collection, taxonomy value, or compatibility rule is no longer active, its deprecation should be explicit, reviewable, and reflected in the registries.

library review checklist2

Use this as the library review checklist.

This is the periodic audit layer that tells you whether the knowledge system is still healthy or drifting back into clutter.

Purpose

A library review checklist is for scheduled audits of the whole knowledge system.

Use it to check whether:

- canonical content is clean
- taxonomy is still coherent
- registries are healthy
- aliases are controlled
- relationships are useful
- bundles still make sense
- legacy compatibility is shrinking instead of growing
- the library is still easy to expand

This is not for one small change.
It is for reviewing the system as a system.

Where this should live

Put it in:

Axion/libraries/knowledge/SYSTEM/templates/library_review_checklist.md

The template

Knowledge Library Review Checklist

1. Review summary
- Review title:
- Reviewer:
- Date:
- Review scope:
 - [ ] full library
 - [ ] canonical content
 - [ ] taxonomy
 - [ ] registries
 - [ ] bundles
 - [ ] collections
 - [ ] migration / legacy compatibility
 - [ ] other

2. Canonical content health
- [ ] All active knowledge items live in `CONTENT/ITEMS`
- [ ] No new canonical items are being added under `PILLARS`
- [ ] KID filenames match canonical KIDs
- [ ] Required frontmatter is present
- [ ] Placeholder metadata is not lingering in active items
- [ ] Duplicate or near-duplicate KIDs have been reviewed
- [ ] Deprecated KIDs are explicitly marked and traceable

Notes:
-

3. Taxonomy health
- [ ] Taxonomy values are still distinct and non-overlapping
- [ ] No obvious one-off taxonomy values have been added
- [ ] Domains are not drifting into tags
- [ ] Tags are not doing the job of taxonomy
- [ ] Pillars still reflect semantic roles, not storage habits
- [ ] Deprecated taxonomy values are being cleaned up properly

Notes:
-

4. Registry health
- [ ] `knowledge.index.json` is generated, not hand-maintained
- [ ] `aliases.index.json` is generated, not drifting
- [ ] `relationships.index.json` is generated, not drifting
- [ ] Registry build pipeline passes
- [ ] No unexplained registry errors are accumulating

- [ ] Warning volume is acceptable and understood

Notes:
-

5. Alias health
- [ ] Aliases exist only where retrieval or compatibility justifies them
- [ ] Alias collisions are reviewed
- [ ] Vanity or low-value aliases are not accumulating
- [ ] Legacy alias behavior is still justified
- [ ] Alias sprawl is under control

Notes:
-

6. Relationship health
- [ ] `depends_on` is used where true dependencies exist
- [ ] `related_kids` is meaningful and not random
- [ ] `supersedes` is used for replacement flows
- [ ] Relationship references point to real KIDs
- [ ] Orphaned items have been reviewed
- [ ] Important structural knowledge is not relationship-empty by neglect

Notes:
-

7. Collection health
- [ ] Collection descriptors are explicit and useful
- [ ] Collections do not duplicate canonical item content
- [ ] Collection scope is clear
- [ ] Overlapping collections are distinguished clearly
- [ ] Collections are still derivable from item metadata
- [ ] Old `_meta.md` migration leftovers are being retired

Notes:
-

8. Bundle health
- [ ] Bundles still serve real repeatable use cases
- [ ] Bundles do not depend on path structure
- [ ] Bundle logic is understandable
- [ ] Near-duplicate bundles have been reviewed
- [ ] Deprecated bundles are explicitly tracked
- [ ] Bundle sprawl is under control

Notes:
-

9. Legacy compatibility health
- [ ] `PILLARS` is shrinking in operational importance
- [ ] Old path fallback is still justified where it remains
- [ ] Legacy redirects are documented
- [ ] Temporary compatibility is not becoming permanent by neglect
- [ ] Migration support files are still accurate
- [ ] Removal candidates have been identified

Notes:
-

10. Search / retrieval health
- [ ] Search works from metadata, not path assumptions
- [ ] Common lookup terms resolve cleanly
- [ ] Important KIDs are discoverable by title, domain, tags, or aliases
- [ ] Search relevance is not being harmed by taxonomy drift
- [ ] Browse views still reflect canonical metadata accurately

Notes:
-

11. Governance health
- [ ] New KIDs are entering through the canonical process
- [ ] Taxonomy changes are reviewed instead of ad hoc
- [ ] Deprecations are explicit
- [ ] Change requests are being used where appropriate
- [ ] Generated files are not being hand-patched as the real fix
- [ ] The library still has a clear source-of-truth hierarchy

Notes:
-

12. Risk findings
List the biggest current risks.

- Structural risk:
- Taxonomy risk:
- Retrieval risk:
- Compatibility risk:
- Maintenance risk:

13. Recommended actions
List specific follow-up actions.

- Immediate:
- Near-term:
- Later:

14. Review outcome
- Overall status:
 - [ ] healthy
 - [ ] healthy with cleanup needed
 - [ ] drifting
 - [ ] high risk / needs structural intervention

- Reviewer conclusion:
- Follow-up owner:
- Follow-up date:

How to use it

Run this checklist on a cadence, not just when something is already broken.

Good triggers are:

- after major migration batches
- after taxonomy changes
- after bundle growth
- after lots of new KIDs are added
- before relying on the library for broader automation

Recommended review cadence

Use different cadences for different layers.

Monthly

Review:

- registry health
- alias health
- legacy compatibility health

Quarterly

Review:

- taxonomy health
- bundle health
- collection health
- governance health

After major restructuring

Review:

- canonical content health
- search/retrieval health
- migration health

What a good review should catch

This checklist is meant to catch problems like:

- new KIDs sneaking back into old tree structures
- taxonomy growing with duplicate meanings

- generated registries being patched manually
- orphaned or duplicated items
- bundle sprawl
- permanent “temporary” legacy compatibility
- search relevance weakening because metadata got sloppy

Suggested scoring model

If you want, you can add a simple score per section:

- `0 = broken`
- `1 = weak`
- `2 = acceptable`
- `3 = strong`

Then total the sections for a quick system health score.

That is optional, but useful once the library gets bigger.

Practical policy statement

A clean structure is not enough by itself.
The library must be reviewed as a living system so drift, duplication, and compatibility debt are caught before they become the new normal.

Knowledge Library Operating Manual.

Use this as the consolidated Knowledge Library Operating Manual.

This pulls the structure, governance, workflows, and templates into one operating document so the library has a single clear way of working.

Purpose

The Knowledge Library Operating Manual defines:

- how the library is structured
- what each zone is for
- what counts as canonical content
- how metadata works
- how registries are built
- how changes are governed
- how deprecations and migration are handled
- how periodic review works

This should become the main reference for running the library cleanly over time.

Where this should live

Put it in:

Axion/libraries/knowledge/SYSTEM/doctrine/knowledge_library_operating_manual.md

The operating manual

Knowledge Library Operating Manual

1. Purpose

The Axion Knowledge Library is the canonical store for reusable knowledge units used for retrieval, guidance, packaging, and system support.

Its operating goals are:

- keep canonical knowledge stable and explicit
- make classification metadata-driven
- keep registries generated and trustworthy
- support browsing without letting browse structures become storage truth
- preserve compatibility during migration without letting legacy structure govern the system forever

---

2. Source-of-truth hierarchy
The library is governed in this order:

1. canonical item content in `CONTENT/ITEMS`
2. controlled taxonomy in `SYSTEM/taxonomy`
3. generated registries in `SYSTEM/registries`
4. collection descriptors in `CONTENT/META/collections`
5. generated views in `VIEWS`
6. legacy compatibility in `PILLARS` only as fallback during migration

No update is considered valid if it bypasses this hierarchy.

---

3. Library zones

3.1 Canonical content
Path:
`CONTENT/ITEMS/`

Purpose:
Holds all actual reusable canonical KIDs.

Rules:
- every canonical item lives here
- filename matches `kid`
- item identity is defined by frontmatter, not folder location

3.2 System control
Path:

`SYSTEM/`

Purpose:
Holds doctrine, taxonomy, registries, contracts, policies, and templates.

Subzones:
- `SYSTEM/doctrine`
- `SYSTEM/taxonomy`
- `SYSTEM/registries`
- `SYSTEM/contracts`
- `SYSTEM/policies`
- `SYSTEM/templates`

3.3 Collection metadata
Path:
`CONTENT/META/collections/`

Purpose:
Defines browsing and grouping context for domains, industries, stacks, pillars, and related collection concepts.

3.4 Views
Path:
`VIEWS/`

Purpose:
Generated navigation surfaces only.

Rules:
- no canonical content authored here
- views do not define truth

3.5 Bundles
Path:
`BUNDLES/`

Purpose:
Curated packs for executor, risk-class, run-profile, and other repeatable working sets.

Rules:
- bundles select canonical KIDs
- bundles must not depend on path structure

3.6 Reuse and outputs

Paths:
- `REUSE/`
- `OUTPUTS/`

Purpose:
Reuse/compliance tracking and generated output artifacts.

3.7 Legacy compatibility
Path:
`PILLARS/`

Purpose:
Temporary migration compatibility only.

Rules:
- no new canonical content here
- path logic here is fallback only
- operational dependence should decrease over time

---

4. Canonical item standard

4.1 Every canonical item must have:
- `kid`
- `title`
- `content_type`
- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `pillar_refs`
- `status`
- `authority_tier`

4.2 Recommended optional fields
- `secondary_domains`
- `freshness_class`
- `bundle_refs`
- `tags`
- `aliases`
- `depends_on`
- `related_kids`
- `supersedes`
- `source_refs`

- `legacy_path`

4.3 Canonical content rules
- one KID = one coherent reusable unit
- semantic identity should not silently change
- if identity changes materially, create a new KID
- path must not carry the meaning the metadata should carry

---

5. Taxonomy rules

Taxonomy is controlled vocabulary for durable reusable classification.

5.1 Taxonomy areas
- `content_type`
- `pillar`
- `primary_domain`
- `industry_refs`
- `stack_family_refs`
- `status`
- `authority_tier`
- `freshness_class`

5.2 Taxonomy rules
- taxonomy values must be durable
- taxonomy values must be reusable
- taxonomy values must be distinct from neighbors
- one-off concepts should usually be tags, not taxonomy

5.3 Taxonomy anti-drift rule
Do not add taxonomy values to solve narrow or temporary problems.

Use:
- tags
- bundles
- collections
- content detail

instead where appropriate.

---

6. Registry rules

Generated registries live in:
`SYSTEM/registries/`

Core generated registries:
- `knowledge.index.json`
- `aliases.index.json`
- `relationships.index.json`

6.1 Registry rules
- generated registries are not hand-maintained
- if a registry is wrong, fix metadata, override input, or builder logic
- registries must be deterministic and rebuildable

6.2 Registry build pipeline
The standard order is:

1. validate canonical items
2. build `knowledge.index.json`
3. build `aliases.index.json`
4. build `relationships.index.json`
5. write summary reports

No update is valid until the registry build passes.

---

7. Alias rules

Aliases exist only for retrieval and compatibility.

7.1 Allowed alias reasons
- legacy filename support
- common search term support
- migrated naming support
- established shorthand lookup

7.2 Alias restrictions
- an alias must resolve to one KID only
- aliases are not metadata identity
- vanity aliases are discouraged
- collision-prone aliases must be reviewed carefully

---

8. Relationship rules

Relationships make structural meaning explicit.

8.1 Relationship types
- `depends_on`
- `related_kids`
- `supersedes`
- `replaced_by` through deprecation handling
- `legacy_path` compatibility mapping
- `collection_keys` generated from item metadata

8.2 Relationship rules
- use `depends_on` where actual dependency exists
- use `related_kids` for meaningful non-dependent links
- use `supersedes` when a new item replaces an older one
- relationship fields should not be left empty by neglect on structurally important items

---

9. Collection rules

Collection descriptors describe groupings. They do not replace canonical items.

9.1 Common collection types
- industry
- domain
- stack_family
- pillar

9.2 Collection rules
- each collection must be derivable from item metadata
- collections should clarify browsing, not duplicate item content
- overlapping collections must explain their distinction

---

10. Bundle rules

Bundles are curated selection mechanisms over canonical items.

10.1 Bundle types
- executor
- risk_class

- run_profile
- solution_pack
- starter_set
- custom

10.2 Selection modes
- `explicit`
- `filter`
- `hybrid`

10.3 Bundle rules
- bundles select by KID or metadata, never path
- bundles do not define semantic identity
- near-duplicate bundles should be reviewed aggressively

---

11. Resolver model

Resolution order is:

1. canonical item metadata
2. generated registries
3. collection descriptors
4. generated views
5. legacy path fallback only during migration

The resolver must not rely on folder structure as the primary semantic system.

---

12. Migration rules

Migration from legacy structure to canonical structure is governed.

12.1 Migration principles
- canonical copies are safer than destructive moves at first
- old paths must be preserved through `legacy_path` and redirects during transition
- `_meta.md` files are converted into collection descriptors
- `PILLARS` is frozen during migration

12.2 Migration completion rule
Legacy fallback should shrink over time and eventually stop being part of the normal operating path.

---

13. Deprecation rules

Nothing should be silently retired.

13.1 Deprecation applies to
- KIDs
- bundles
- collection descriptors
- taxonomy values
- legacy compatibility rules

13.2 Deprecation requirements
- mark status explicitly
- record replacement if applicable
- update aliases/relationships as needed
- preserve traceability
- rebuild registries

Deprecated does not automatically mean deleted.

---

14. Change governance

All meaningful updates should be classed as one or more of:

- Type A — content update
- Type B — new KID
- Type C — taxonomy update
- Type D — relationship update
- Type E — alias update
- Type F — bundle update
- Type G — deprecation update
- Type H — migration/compatibility update

Appropriate change-request templates should be used for higher-risk changes.

---

15. Validation gates

A valid library update must pass:

- canonical frontmatter validation
- duplicate KID checks
- taxonomy value validation
- alias collision checks
- relationship reference checks
- registry build pipeline

Higher-confidence review should also check:
- placeholder titles
- unknown domains
- weak tags
- missing relationships where they matter

---

16. Review cadence

Monthly
Review:
- registry health
- alias health
- legacy compatibility health

Quarterly
Review:
- taxonomy health
- bundle health
- collection health
- governance health

After major restructuring
Review:
- canonical content health
- retrieval quality
- migration status

---

17. Forbidden patterns

The following are not allowed as normal operating behavior:

- adding canonical KIDs under `PILLARS`
- hand-editing generated registries as the permanent fix
- creating one-off taxonomy values
- encoding meaning only in paths
- using views as canonical storage
- using bundles as storage categories
- silently mutating a KID's semantic identity
- leaving migrated items under-specified because the path "already explains it"

---

18. Standard operating workflow

For a normal content update
1. update or add canonical item(s) in `CONTENT/ITEMS`
2. update metadata and relationships as needed
3. run registry build pipeline
4. inspect reports
5. fix issues
6. accept only after pipeline passes

For taxonomy changes
1. submit taxonomy change request
2. approve taxonomy change
3. update taxonomy files
4. update affected KIDs
5. rebuild registries
6. review retrieval/browse impact

For deprecations
1. submit deprecation request
2. update item or taxonomy status
3. update replacement relationships
4. update compatibility handling if needed
5. rebuild registries
6. review search and bundle impact

---

19. Templates and control documents

Core templates:
- `KID.template.md`
- `collection_descriptor.template.md`

- `bundle_definition.template.md`
- `knowledge_change_request.md`
- `taxonomy_change_request.md`
- `deprecation_request.md`
- `library_review_checklist.md`

These documents should be treated as part of the library's operating system.

---

20. Definition of done
The Knowledge Library is operating correctly when:

- canonical content is centralized
- taxonomy is controlled and coherent
- registries are generated and trusted
- search uses metadata, not path assumptions
- bundles and collections remain non-canonical
- deprecations are explicit
- migration compatibility is shrinking
- the system can grow without structural surgery every time

Why this matters

This gives you one document that explains the whole operating model instead of forcing you to piece it together from separate templates and rules.

That matters because once the library gets bigger, structural drift usually happens when:

- the rules exist, but are scattered
- nobody remembers the source-of-truth order
- people start patching symptoms instead of following the system

Recommended companion files

Once this manual exists, the clean supporting set is:

- `knowledge_library_operating_manual.md`
- `KID.template.md`
- `collection_descriptor.template.md`
- `bundle_definition.template.md`
- `knowledge_change_request.md`
- `taxonomy_change_request.md`
- `deprecation_request.md`
- `library_review_checklist.md`

phased execution roadmap

Use this as the phased execution roadmap.

This turns the reorganization into an actual rollout sequence so you are not trying to clean, migrate, rewrite, and govern everything at once.

Goal

Move from the current state:

- legacy tree dominates
- structure is path-heavy
- registries can drift
- resolver logic depends too much on folder shape

to the target state:

- canonical content lives in `CONTENT/ITEMS`
- metadata drives classification
- registries are generated
- resolver is metadata-first
- governance prevents relapse

Phase 1 — structural cleanup

Objective

Create the new operating structure without yet doing the risky full-content migration.

Deliverables

Create:

- `SYSTEM/`
- `CONTENT/`
- `VIEWS/`

Move:

- root `KNO-*` → `SYSTEM/doctrine/`
- `INDEX/` → split into `SYSTEM/taxonomy/` and `SYSTEM/registries/`
- `contracts/` → `SYSTEM/contracts/`
- `POLICIES/` → `SYSTEM/policies/`
- `TEMPLATES/` → `SYSTEM/templates/`

Keep but normalize:

- `BUNDLES/`
- `REUSE/`
- `OUTPUTS/`

Freeze:

- `PILLARS/`

Success condition

The library root is clean and the non-content systems are no longer mixed into the same layer as canonical knowledge.

Phase 2 — canonical content migration

Objective

Move actual knowledge items out of the legacy tree into canonical storage.

Deliverables

Create:

- `CONTENT/ITEMS/`
- `CONTENT/META/collections/`
- `CONTENT/META/aliases/`
- `CONTENT/META/redirects/`

Run migration work for:

- `INDUSTRY_PLAYBOOKS`
- `IT_END_TO_END`
- `LANGUAGES_AND_LIBRARIES`

Do two parallel conversions:

- KIDs → `CONTENT/ITEMS/`
- `_meta.md` → `CONTENT/META/collections/`

Use:

- collision-safe KID migration script

- `_meta.md` conversion script

Add explicit frontmatter to moved KIDs.

Preserve:

- `legacy_path`
- alias/redirect information

Success condition

New canonical items are living in `CONTENT/ITEMS`, and the old tree is no longer the only place the library can function from.

Phase 3 — registry stabilization

Objective

Make generated registries the real machine-readable backbone.

Deliverables

Lock and generate:

- `knowledge.index.json`
- `aliases.index.json`
- `relationships.index.json`

Add:

- `aliases.manual.json` if needed
- `relationships.manual.json` if needed

Implement builders:

- `build_knowledge_index.py`
- `build_aliases_index.py`
- `build_relationships_index.py`
- `build_registries.py`

Write reports into:

- `OUTPUTS/selections/`

Stop hand-editing generated registries.

Success condition

Canonical items plus builders can regenerate the registry layer consistently and deterministically.

Phase 4 — resolver rewrite

Objective

Remove the old path-first lookup model and make resolution metadata-first.

Deliverables

Implement resolver modules:

- `types.ts`
- `paths.ts`
- `registry.ts`

- `resolveKid.ts`
- `loadItem.ts`
- `loadRelationships.ts`
- `loadCollections.ts`
- `searchItems.ts`
- `legacyFallback.ts`
- `index.ts`

Resolution order becomes:

1. canonical item metadata
2. generated registries
3. collection descriptors
4. generated views
5. legacy path fallback

Update callers to use the new resolver.

Keep `legacyFallback` only as temporary migration support.

Success condition

Normal retrieval no longer depends on `PILLARS/` or folder inference.

Phase 5 — governance rollout

Objective

Make the clean structure durable so it does not degrade again.

Deliverables

Install and use:

- Knowledge Library Operating Manual
- KID template
- collection descriptor template
- bundle definition template
- knowledge change request
- taxonomy change request
- deprecation request
- library review checklist

Adopt rules for:

- new KIDs
- taxonomy changes
- aliases
- relationships
- deprecations
- review cadence

Success condition

New updates follow the governed process instead of improvising structure ad hoc.

Phase 6 — bundle and view normalization

Objective

Standardize the non-canonical browsing and packaging layers.

Deliverables

Normalize:

- bundle definitions
- collection descriptors
- generated browse views

Ensure:

- bundles select by KID or metadata only
- views are generated from metadata
- collections are explicit and useful
- none of these layers act like canonical storage

Success condition

Browsing and packaging are useful, but they do not compete with canonical content.

Phase 7 — legacy retirement

Objective

Shrink and eventually retire old-tree dependence.

Deliverables

Audit and reduce:

- path-based fallback logic
- old redirects that are no longer needed
- reliance on `PILLARS/`
- leftover `_meta.md` artifacts
- migration-only compatibility scaffolding

Move toward:

- archival snapshot of `PILLARS/`
  or
- generated legacy-reference layer only

Success condition

The library runs primarily from canonical items, registries, and metadata-driven resolution, with little or no operational reliance on the old tree.

Recommended order of execution

Do the phases in this order:

1. structural cleanup
2. canonical migration
3. registry stabilization
4. resolver rewrite
5. governance rollout
6. bundle/view normalization

7. legacy retirement

That order matters because each phase reduces risk for the next one.

What not to do

Do not try to:

- rewrite the resolver before canonical items exist
- retire `PILLARS/` before registries and redirects work
- normalize bundles before metadata is trustworthy
- enforce governance before the operating structure exists

That would create churn without real stability.

Practical milestone version

Milestone A

Root cleanup complete

Milestone B

Canonical KID migration working

Milestone C

Registry pipeline working

Milestone D

Resolver switched to metadata-first

Milestone E

Governance and templates in active use

Milestone F

Legacy tree no longer operationally central

Definition of full completion

You are done when:

- all active KIDs live in `CONTENT/ITEMS`
- registries are generated, not hand-maintained
- resolver works from metadata-first logic
- bundles and views are non-canonical
- governance controls new changes
- `PILLARS/` is legacy-only or retired

task board

Here's the task board version of the roadmap.

This breaks each phase into concrete work items so you can track progress without losing the bigger structure.

Phase 1 — structural cleanup

1.1 Create the new root structure

- Create `SYSTEM/`
- Create `CONTENT/`
- Create `VIEWS/`
- Create `CONTENT/META/`
- Create `SYSTEM/doctrine/`
- Create `SYSTEM/taxonomy/`
- Create `SYSTEM/registries/`
- Create `SYSTEM/contracts/`
- Create `SYSTEM/policies/`
- Create `SYSTEM/templates/`

1.2 Relocate non-content system files

- Move root `KNO-*` files into `SYSTEM/doctrine/`
- Split `INDEX/` into `SYSTEM/taxonomy/` and `SYSTEM/registries/`

- Move `contracts/` into `SYSTEM/contracts/`
- Move `POLICIES/` into `SYSTEM/policies/`
- Move `TEMPLATES/` into `SYSTEM/templates/`

1.3 Normalize non-canonical zones

- Confirm `BUNDLES/` stays separate
- Confirm `REUSE/` stays separate
- Confirm `OUTPUTS/` stays separate
- Add placeholder README files where needed

1.4 Freeze the legacy tree

- Add migration notice to `PILLARS/`
- Set rule: no new canonical content under `PILLARS/`
- Record legacy-tree freeze in operating notes

Exit criteria

- New root exists
- Non-content systems moved
- `PILLARS/` clearly marked legacy

Phase 2 — canonical content migration

2.1 Create canonical content zones

- Create `CONTENT/ITEMS/`
- Create `CONTENT/META/collections/`
- Create `CONTENT/META/aliases/`
- Create `CONTENT/META/redirects/`

2.2 Prepare migration tooling

- Create collision-safe KID migration script
- Create `_meta.md` conversion script
- Define migration report outputs
- Define collision handling rules
- Define missing-KID handling rules

2.3 Migrate `INDUSTRY_PLAYBOOKS`

- Scan for KID files
- Detect collisions
- Copy canonical KIDs into `CONTENT/ITEMS/`
- Convert `_meta.md` into collection descriptors
- Add `legacy_path`
- Review ambiguous industry mappings

2.4 Migrate `IT_END_TO_END`

- Scan for KID files
- Detect collisions
- Copy canonical KIDs into `CONTENT/ITEMS/`
- Convert `_meta.md` into collection descriptors
- Add `legacy_path`
- Review ambiguous domain mappings

2.5 Migrate `LANGUAGES_AND_LIBRARIES`

- Scan for KID files
- Detect collisions
- Copy canonical KIDs into `CONTENT/ITEMS/`
- Convert `_meta.md` into collection descriptors
- Add `legacy_path`
- Review ambiguous stack-family mappings

2.6 Normalize moved KIDs

- Ensure filename matches `kid`
- Add required frontmatter
- Add recommended metadata defaults
- Clean placeholder metadata

- Record unresolved metadata gaps

Exit criteria

- Canonical KIDs exist in `CONTENT/ITEMS/`
- Converted collection descriptors exist
- `legacy_path` is preserved
- Migration reports exist

Phase 3 — registry stabilization

3.1 Lock registry contracts

- Lock `knowledge.index.json` shape
- Lock `aliases.index.json` shape
- Lock `relationships.index.json` shape
- Lock report shapes
- Lock enum expectations

3.2 Build knowledge index tooling

- Implement `build_knowledge_index.py`
- Add required-field validation
- Add taxonomy validation

- Add duplicate-KID detection
- Add warning and error reports

3.3 Build alias index tooling

- Implement `build_aliases_index.py`
- Generate aliases from title slug
- Generate aliases from legacy filename
- Support frontmatter aliases
- Support `aliases.manual.json`
- Add alias collision reporting

3.4 Build relationships index tooling

- Implement `build_relationships_index.py`
- Generate `current_path`
- Generate `collection_keys`
- Pull `depends_on`
- Pull `related_kids`
- Pull `supersedes`
- Support `relationships.manual.json`
- Validate referenced KIDs

3.5 Build the pipeline orchestrator

- Implement `build_registries.py`
- Run builders in correct order
- Produce `registry_build_summary.json`
- Add `--check`
- Add `--write`
- Add `--strict`

3.6 Make registries generated-only

- Stop hand-editing generated registries
- Document build workflow
- Add registry pipeline to maintenance routine

Exit criteria

- All three registries generate successfully
- Reports are written
- Registry build is repeatable

Phase 4 — resolver rewrite

4.1 Create resolver module structure

- Create `types.ts`

- Create `paths.ts`
- Create `registry.ts`
- Create `resolveKid.ts`
- Create `loadItem.ts`
- Create `loadRelationships.ts`
- Create `loadCollections.ts`
- Create `searchItems.ts`
- Create `legacyFallback.ts`
- Create `index.ts`

4.2 Implement direct resolution

- Resolve direct KID input
- Load item from `CONTENT/ITEMS/`
- Load registry data
- Return canonical path and legacy path where applicable

4.3 Implement alias and legacy resolution

- Resolve alias through `aliases.index.json`
- Resolve old path through `relationships.index.json`
- Resolve legacy filename fallback

- Return resolution source metadata

4.4 Implement metadata search

- Search by title
- Search by `primary_domain`
- Search by `industry_refs`
- Search by `stack_family_refs`
- Search by `pillar_refs`
- Search by tags
- Search by aliases

4.5 Implement collection enrichment

- Match items to collection descriptors
- Load related collection context
- Keep collection logic non-authoritative

4.6 Transition callers

- Identify old resolver callers
- Switch them to metadata-first resolver
- Keep legacy fallback temporary
- Remove path-first assumptions from normal flow

Exit criteria

- KID lookup works from canonical metadata
- Alias lookup works
- Legacy-path redirect works
- Search no longer depends on folder traversal

Phase 5 — governance rollout

5.1 Install governance documents

- Add Knowledge Library Operating Manual
- Add KID template
- Add collection descriptor template
- Add bundle definition template
- Add knowledge change request template
- Add taxonomy change request template
- Add deprecation request template
- Add library review checklist

5.2 Define update process

- Define new KID process
- Define taxonomy change process

- Define alias update process
- Define relationship update process
- Define deprecation process
- Define migration-change process

5.3 Define acceptance gates

- Require valid frontmatter
- Require registry pipeline pass
- Require taxonomy review where applicable
- Require deprecation traceability
- Require no direct generated-registry patching

5.4 Roll governance into normal workflow

- Make templates discoverable
- Add guidance in doctrine
- Use change-request templates for structural changes
- Record review cadence

Exit criteria

- Governance templates exist
- Update rules are explicit
- Structural changes follow the process

Phase 6 — bundle and view normalization

6.1 Normalize bundle definitions

- Review all existing bundles
- Remove path-based assumptions
- Reframe bundles as KID/metadata selectors
- Convert unclear bundles to explicit/hybrid/filter logic
- Add or update bundle descriptors

6.2 Normalize collection descriptors

- Review converted collection files
- Remove weak or duplicate collections
- Tighten collection scopes
- Add related collection links
- Ensure scope maps cleanly to metadata

6.3 Generate browse views

- Build `VIEWS/by_pillar/`
- Build `VIEWS/by_domain/`
- Build `VIEWS/by_content_type/`
- Build `VIEWS/by_industry/`
- Build `VIEWS/by_stack_family/`

- Build `VIEWS/by_status/`
- Build `VIEWS/by_authority/`
- Build `VIEWS/by_bundle/`

6.4 Validate browse behavior

- Check that views reflect canonical metadata
- Check that views do not act like storage
- Check that bundle discovery remains useful
- Check that collection and bundle overlap is understandable

Exit criteria

- Bundles are metadata/KID-driven
- Views are generated and useful
- Collections are explicit and stable

Phase 7 — legacy retirement

7.1 Audit remaining legacy dependence

- Identify unresolved `PILLARS/` dependencies
- Identify unresolved path-based resolver behavior
- Identify remaining `_meta.md` leftovers

- Identify old redirects still actively needed

7.2 Reduce compatibility debt

- Remove dead redirects
- Remove old-path assumptions no longer used
- Retire migration-only scaffolding where safe
- Tighten legacy fallback scope

7.3 Decide end-state for `PILLARS/`

- Archive as historical snapshot
- Or keep as legacy reference only
- Or regenerate limited compatibility views if needed

7.4 Final cleanup pass

- Remove empty legacy directories where safe
- Remove outdated migration notes
- Confirm canonical pipeline works without legacy-tree dependence

Exit criteria

- Legacy tree is no longer operationally central
- Old path fallback is minimal or gone
- Canonical content and registries run the system

Cross-phase support tasks

A. Documentation

- Add migration README
- Add builder usage docs
- Add resolver usage docs
- Add governance quickstart
- Add review cadence note

B. Validation

- Add dry-run checks for migration scripts
- Add strict-mode builds
- Add spot-check lists for migrated KIDs
- Add sampling review for collections and bundles

C. Risk management

- Keep non-destructive copy-first migration
- Keep legacy redirects until verified safe to remove
- Track collisions and ambiguous mappings explicitly
- Do not delete originals until canonical copies are verified

---

Suggested board columns

Use these columns:

- Backlog
- Ready
- In Progress
- Review
- Blocked
- Done

Suggested first sprint

If you want the cleanest starting slice, do this first:

Sprint 1

- Create new root structure
- Move doctrine/index/contracts/policies/templates
- Freeze `PILLARS/`
- Create `CONTENT/ITEMS/` and `CONTENT/META/collections/`
- Draft migration scripts
- Lock registry JSON contracts

That gives you the base without yet doing the highest-risk moves.